M2.991_20251_PAC2_Enunciat

November 19, 2025

M2.991 · Aprenentatge automàtic · PAC1

2025-1 · Màster universitari en Ciència de dades (Data science)

Estudis d'informàtica, multimèdia i telecomunicació

1 PAC 2: Mètodes no supervisats

En aquesta pràctica explorarem l'aprenentatge no supervisat amb un cas real: l'anàlisi i agrupament de milers de receptes. A partir de dades textuals i d'ingredients, crearem diferents representacions (des d'embeddings semàntics de models de llenguatge fins a vectors one-hot) per descobrir patrons sense etiquetes. Usarem UMAP per reduir dimensionalitat i diferents tècniques de clustering per segmentar, observant com l'elecció de representació i mètrica de distància condiciona les estructures detectades i la seva interpretació.

La PAC consta de dos exercicis:

- **Clustering amb diferents estratègies**: k-means i regla del colze, basades en densitat i jeràrquiques.
- **Aplicació**: clustering i reducció de dimensionalitat culinària, diferents maneres de descobrir grups de receptes sobre la base de les seves metadades.

Important: cadascun dels exercicis pot suposar diversos minuts d'execució, per la qual cosa el lliurament s'ha de fer en format notebook i en format html, on es vegi el codi, els resultats i els comentaris de cada exercici. Es pot exportar el notebook a html des del menú File → Download as → HTML.

Important: existeix un tipus de cel · la especial per albergar text. Aquest tipus de cel · la us serà molt útil per respondre a les diferents preguntes teòriques plantejades al llarg de cada PAC. Per canviar el tipus de cel · la a aquest tipus, trieu al menú: Cell → Cell Type → Markdown.

Important: la solució plantejada no ha d'utilitzar mètodes, funcions o paràmetres declarats "deprecated" en futures versions.

Important: és convenient que utilitzeu una llavor amb un valor fix (en aquest Notebook se us proposa la variable seed inicialitzada a 100) en tots aquells mètodes o funcions que continguin algun component aleatori per assegurar-vos que obtindreu sempre el mateix resultat en les diferents execucions del vostre codi.

Important: si l'execució d'una cel · la triga més de 20 minuts pots reduir el nombre de files del dataframe per agilitzar el procés.

Important: no oblideu posar el vostre nom i cognoms a la cel · la següent.

Nom i cognoms: Jordi Baiges Ferré —

Per això necessitarem les següents llibreries:

[1]:
```
import ast
from collections import Counter
import random

import hdbscan
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.spatial.distance import pdist, squareform, cdist
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sentence_transformers import SentenceTransformer
from sklearn import cluster
from sklearn.cluster import KMeans, DBSCAN, MeanShift, estimate_bandwidth,␣
↪AgglomerativeClustering
from sklearn import datasets
from sklearn import manifold
from sklearn.metrics import silhouette_score, pairwise_distances
from sklearn.metrics.pairwise import cosine_distances, euclidean_distances
from sklearn.preprocessing import MultiLabelBinarizer
import tqdm
import umap.umap_ as umap

# Inicialitzem la variable global a 100 (llavor)
SEED = 100
```

1.1 1. Mètodes de *clustering* (4 punts)

Aquest exercici tracta d'explorar diferents tècniques d'agrupament ajustant-les a diferents conjunts de dades.

L'objectiu és doble: entendre la influència dels paràmetres en el seu comportament, i conèixer les seves limitacions en la cerca d'estructures de dades.

1.1.1 Generació dels conjunts de dades

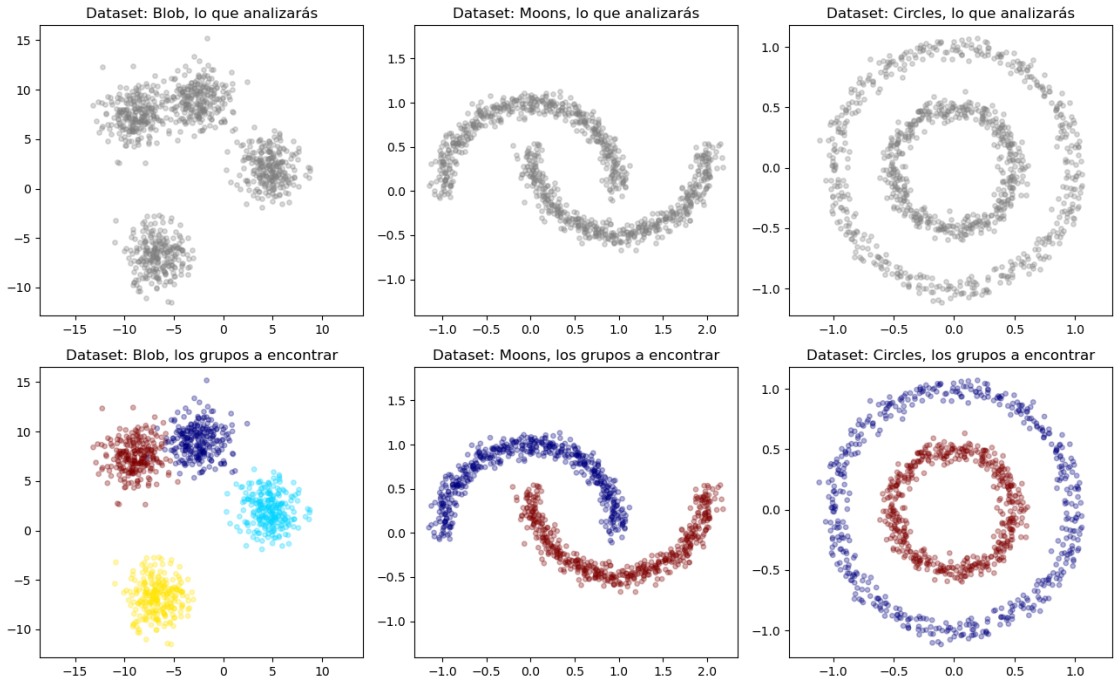
[2]:
```
X_blobs, y_blobs = datasets.make_blobs(n_samples=1000, n_features=2, centers=4,␣
↪cluster_std=1.6, random_state=42)
X_moons, y_moons = datasets.make_moons(n_samples=1000, noise=.07,␣
↪random_state=42)
X_circles, y_circles = datasets.make_circles(n_samples=1000, factor=.5, noise=.
↪05, random_state=42)
```

Cada dataset té 2 variables: una variable X que conté 2 *features* (columnes) i tantes files com mostres. I una variable y que alberga les etiquetes que identifiquen cada clúster.

Al llarg de l'exercici no s'usarà la variable y (només amb l'objectiu de visualitzar). L'objectiu és a través dels diferents models de *clustering* aconseguir trobar les estructures descrites per les variables y.

[3]:
```
fig, axis = plt.subplots(2, 3, figsize=(13, 8))
for i, (X, y, ax, name) in enumerate(zip([X_blobs, X_moons, X_circles] * 2,
                                          [None] * 3 + [y_blobs, y_moons,␣
 ↪y_circles],
                                          axis.reshape(-1),
                                          ['Blob', 'Moons', 'Circles'] * 2)):
    ax.set_title('Dataset: {}, '.format(name) + ('lo que analizarás' if i < 3␣
 ↪else 'los grupos a encontrar'))
    if y is not None:
        ax.scatter(X[:,0], X[:,1], s=15, c=y, alpha=.3, cmap='jet')
    else:
        ax.scatter(X[:,0], X[:,1], s=15, c='gray', alpha=.3, cmap='jet')
    ax.axis('equal')
plt.tight_layout()
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_17856\339150881.py:10: UserWarning:
No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  ax.scatter(X[:,0], X[:,1], s=15, c='gray', alpha=.3, cmap='jet')
```



1.1.2 1 a. K-means

En aquest apartat es demana provar l'algoritme *k-means* sobre els tres datasets presentats anteriorment ajustant amb els paràmetres adequats i analitzar els seus resultats.

[4]:
```
X, y = X_blobs, y_blobs
```

Per estimar el nombre de clústers a detectar per *k-means*. Una tècnica per estimar k és, com s'explica a la teoria: > Els criteris anteriors (minimització de distàncies intra grup o maximització de distàncies inter grup) es poden fer servir per establir un valor adequat per al paràmetre k. Valors k pels quals ja no s'aconsegueixen millores significatives en l'homogeneïtat interna dels segments o l'heterogeneïtat entre segments diferents, s'haurien de descartar.

El que popularment es coneix com a *regla del colze*.

Primer cal calcular la suma dels errors quadràtics (*SSE*) que consisteix en la suma de tots els errors (distància de cada punt al seu centroide assignat) al quadrat.

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} euclidean(x, c_i)^2$$

On K és el nombre de clústers a buscar per *k-means*, $x \in C_i$ són els punts que pertanyen a l'i-èssim clúster, c_i és el centroide del clúster C_i (al qual pertany el punt x), i *euclidean* és la distància euclidiana.

Aquest procediment realitzat per a cada possible valor k, resulta en una funció monòtona decreixent, on l'eix x representa els diferents valors de k, i l'eix y l'SSE. Intuïtivament es podrà observar un significatiu descens de l'error, que indicarà el valor idoni de k.

Es demana realitzar la representació gràfica de la regla del colze juntament amb la seva interpretació, utilitzant la llibreria `matplotlib` i la implementació a scikit-learn de *k-means*.

Implementació: Càlcul i visualització de la regla del colze al dataset Blobs.

1) Convertim a pandas i deixem-ho tot indexat net

[5]:
```
# ---------------------------------------------------------------------
# CONVERSIÓ A PANDAS
# ---------------------------------------------------------------------
# Estructures de partida:
#   - X_blobs : np.ndarray de forma (n, 2) amb les coordenades (x1, x2)
#   - y_blobs : np.ndarray de forma (n,) amb l'etiqueta "etiqueta_real"
#
# Estructures resultants (pandas):
#   - df_blobs : DataFrame amb columnes ['x1','x2','etiqueta_real']
#   - L'index de pandas manté l'ordenació dels punts (0..n-1)
# ---------------------------------------------------------------------
df_blobs = (
    pd.DataFrame(X_blobs, columns=['x1', 'x2'])              # DataFrame (2␣
 ↪columnes float)
```

```
        .assign(etiqueta_real=pd.Series(y_blobs, index=None))  # Afegim␣
 ↪l'etiqueta_real com a Series
)

# Mostrem per pantalla les 5 primeres entrades
print(df_blobs.head(5))

# A partir d'aquí treballarem amb df_blobs (pandas) per coherència amb la PAC2.
```

```
         x1        x2  etiqueta_real
0 -8.385067  6.909275              3
1 -5.692275 -6.401241              2
2 -5.986304 -6.758501              2
3  3.904901  0.613419              1
4  4.224533 -0.431859              1
```

2) Regla del colze amb k-means (SSE/inertia) + silueta (diagnòstic extra)

[6]:
```
# ----------------------------------------------------------------------
# FUNCIÓ D'ANÀLISI DEL COLZE
# ----------------------------------------------------------------------

def elbow_table_kmeans(df, ks=range(1, 11), n_init=20, seed=SEED):
    """
    Itera k = 1..Kmax, entrena KMeans i recull:
      - SSE (inertia_ a sklearn)  -> mesura la suma de distàncies quadrades al␣
 ↪centroide.
      - Silhouette (k >= 2)       -> mesura cohesió/separació dels clústers.

    Parameters:
      - df: DataFrame amb columnes ['x1', 'x2'] (podria tenir més columnes;␣
 ↪aquí en fem servir 2)
      - ks: rang de valors de k a provar (iterable d'enters)
      - n_init: nº d'inicis aleatoris (k-means++). Ajuda a estabilitzar␣
 ↪resultats.
      - seed: llavor de random per fer el procés reproduïble.

    Returns:
      - res_df: DataFrame indexat per k amb columnes:
          ['SSE', 'ΔSSE', 'curv', 'silhouette']
          on ΔSSE = SSE(k-1) - SSE(k) i curv és la 2a diferència discreta␣
 ↪(aprox. "inflexió")
      - elbow_k: enter amb la proposta de k pel colze (màxima curvatura,␣
 ↪ignorant k molt petits)
      - best_sil_k: enter amb el k que maximitza la silueta (k>=2)

    """
    # 1) Secció de dades d'entrada:
```

```
    #    Extraiem només les columnes numèriques que volem agrupar (aquí, x1 i␣
↪x2).
    X = df[['x1', 'x2']].to_numpy()  # np.ndarray (n,2). Fem servir NumPy per a␣
↪sklearn.

    # 2) Diccionaris on anirem acumulant mètriques per a cada k.
    sse_dict = {}   # dict int -> float
    sil_dict = {}   # dict int -> float (o np.nan per k=1)

    # 3) Bucle principal sobre els k candidats

    for k in ks:
        km = KMeans(n_clusters=k, n_init=n_init, random_state=SEED)

        '''
        - K-means el que farà és particionar els punts de X en k grups, i per a␣
↪cada grup calcularà
        un centroide (la mitjana dels punts del clúster).
        - K-means necessita triar uns centroids inicials abans de començar les␣
↪iteracions.
        Si aquests centroids inicials surten "mal donats", el resultat final␣
↪pot ser un mínim local dolent.
        - Per reduir aquest risc, scikit-learn:
            Repeteix el k-means diverses vegades amb inicialitzacions diferents.
            es queda amb la millor solució segons la SSE (inertia_ més baixa).
            n_init vol dir: Quantes vegades vols que es torni a executar␣
↪k-means amb diferents punts inicials.
        Per cada valor de k, k-means:
            S'inicialitza 20 vegades amb centroids diferents, s'executa fins a␣
↪convergir, calcula la suma de
            distàncies quadrades (SSE), tria la configuració amb SSE més baixa.
            random_state=SEED vol dir: Aquest paràmetre controla l'aleatorietat␣
↪de l'algoritme. Serveix per
            fer el resultat reproduïble. Si es torna a executar el codi amb el␣
↪mateix seed, s'obtindràn els mateixos clústers.

        L'objecte km porta també altres paràmetres amb valors per defecte, com:
            init='k-means++' → tria centroids inicials de manera "llesta", no␣
↪totalment aleatòria.
            max_iter=300 → màxim 300 iteracions per cada execució de k-means.
            tol=1e-4 → criteri de parada: si el canvi en SSE és molt petit, el␣
↪procés es dona per convergit.

        Per a cada inicialització, es fa:
            Assignar cada punt al centroide més proper (distància euclidiana),
            Recalcular els centroids com la mitjana dels punts de cada clúster,
```

```
            Repetir fins que "no canvii gaire" o arribi a max_iter.
        Es tria la millor solució i es guarden atributs a km com:
            km.cluster_centers_ → coordenades dels centroids.
            km.labels_ → etiqueta de clúster de cada punt.
            km.inertia_ → SSE final (que es guarda a sse_dict[k]).
        '''

        # Entrenem i obtenim etiquetes predites com a np.ndarray (n,)
        labels = km.fit_predict(X)

        # SSE (inertia_) és un escalar float: suma de distàncies quadràtiques␣
↪punt-centroide
        sse_dict[k] = float(km.inertia_)

        # Silueta: només definida per k >= 2. Guardem np.nan per k=1.
        sil_dict[k] = (
            float(silhouette_score(X, labels, metric='euclidean'))
            if k >= 2 else np.nan
        )

    # 4) Convertim els diccionaris a Series per aprofitar l'index per k i fer␣
↪càlculs vectorials:

    sse_s = pd.Series(sse_dict, name='SSE').sort_index()           # Series␣
↪(index=k, val=SSE)
    dSSE  = sse_s.shift(1) - sse_s                                 # 1a␣
↪diferència (caiguda de SSE)
    curv  = dSSE.shift(1) - dSSE                                   # 2a␣
↪diferència (aprox. curvatura)
    sil_s = pd.Series(sil_dict, name='silhouette').sort_index()    # Series␣
↪(index=k, val=silhouette)

    # 5) Taula final: DataFrame amb totes les mètriques alineades per index k.
    res_df = pd.concat(
        [sse_s, dSSE.rename('ΔSSE'), curv.rename('curv'), sil_s],
        axis=1
    )

    # 6) Propostes automàtiques:
    #    - "Colze": triem el k amb màxima curvatura, però ignorem els 2 primers␣
↪k per evitar soroll.
    elbow_k = curv.iloc[2:].idxmax()
    #    - "Millor silueta": k (>=2) que maximitza silhouette
    best_sil_k = sil_s.iloc[1:].idxmax()

    return res_df, elbow_k, best_sil_k
```

```
# Executem l'anàlisi del colze sobre Blobs llançan la funció elbow_table_kmeans␣
↪preparada abans.
elbow_df, elbow_k, best_sil_k = elbow_table_kmeans(df_blobs, ks=range(1, 11),␣
↪n_init=20, seed=42)

# Inspeccionem la taula (DataFrame): cada fila és un k; columnes = mètriques
print(elbow_df.round(3))

print(f"\nProposta pel colze (curvatura màxima): k = {elbow_k}")
print(f"Millor k per silueta: k = {best_sil_k}")
```

```
          SSE       ΔSSE       curv  silhouette
1   69844.440        NaN        NaN         NaN
2   34688.496  35155.944        NaN       0.549
3   10326.863  24361.633  10794.311       0.681
4    4943.411   5383.452  18978.182       0.668
5    4478.587    464.824   4918.628       0.570
6    4015.612    462.975      1.849       0.453
7    3563.752    451.861     11.114       0.363
8    3176.536    387.215     64.646       0.324
9    2893.864    282.672    104.543       0.331
10   2647.480    246.385     36.287       0.331

Proposta pel colze (curvatura màxima): k = 4
Millor k per silueta: k = 3
```

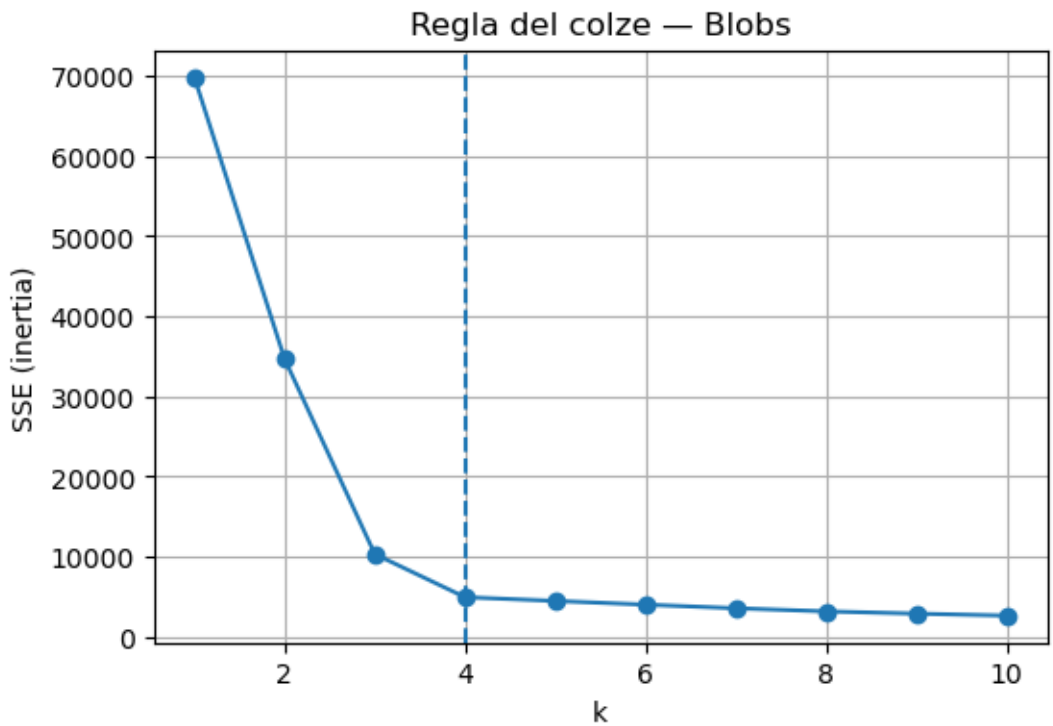
Què acabem de fer (resum conceptual)

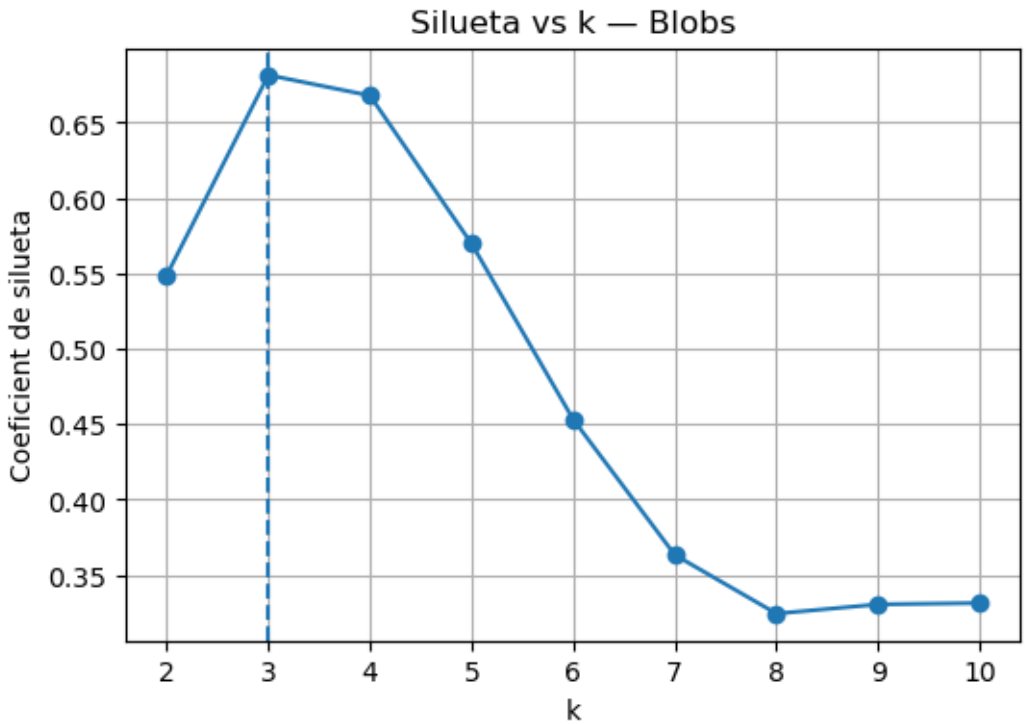
- Diccionaris per acumular mètriques {k: valor} → fàcil convertir a Series (pandas) i operar per index.
- Series per fer diferències (shift) i derivades discretes → generem ΔSSE i curv.
- DataFrame final (elbow_df) per tenir tot plegat (SSE, caiguda, curvatura, silueta) i poder justificar la tria de k.

3) Gràfics: SSE (colze) i Silueta (contrast)

[7]:
```
# ----------------------------------------------------------------------
# VISUALITZACIÓ (matplotlib ja importat)
#   - Gràfic 1: SSE vs k (regla del colze) amb línia vertical al k suggerit
#   - Gràfic 2 (opcional però útil): Silueta vs k amb línia vertical al màxim
# ----------------------------------------------------------------------

# Gràfic del colze (SSE)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(elbow_df.index, elbow_df['SSE'], marker='o')
ax.set_xlabel('k')
ax.set_ylabel('SSE (inertia)')
```

```
ax.set_title('Regla del colze - Blobs')
ax.grid(True)
ax.axvline(elbow_k, linestyle='--')  # marquem la nostra proposta
plt.show()

# Gràfic de Silueta (diagnòstic extra)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(elbow_df.index, elbow_df['silhouette'], marker='o')
ax.set_xlabel('k')
ax.set_ylabel('Coeficient de silueta')
ax.set_title('Silueta vs k - Blobs')
ax.grid(True)
ax.axvline(best_sil_k, linestyle='--')
plt.show()
```





Anàlisi: Què s'interpreta a la gràfica? Com es podria millorar l'elecció de k?.

Lectura dels gràfics

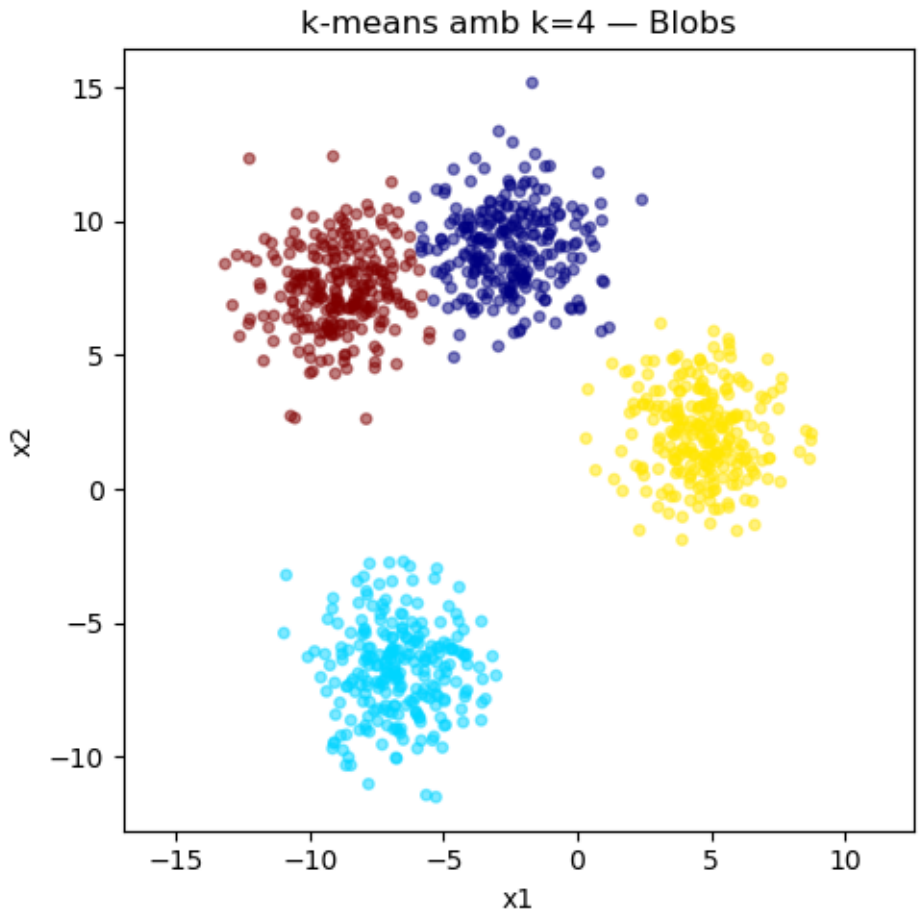
- SSE ha de decréixer quan creix k. El colze és on la corba "s'aplana": millores molt menors en SSE tot i augmentar k..
- Silueta dóna una segona opinió: valors més alts indiquen clústers més densos i separats..

Implementació: Càlcul i visualització dels grups al dataset Blobs.

4) Model final i prediccions com a Series dins del DataFrame

[8]:
```
# ----------------------------------------------------------------------
# ENTRENAMENT FINAL AMB EL k TRIAT
# ----------------------------------------------------------------------
# Estructures:
#   - km_final : objecte KMeans entrenat (conté centroides i labels_)
#   - labels_s : Series amb les etiquetes predites alineades amb l'index del␣
↪DataFrame
#   - df_blobs : DataFrame original + nova columna 'cluster_kmeans'
# ----------------------------------------------------------------------

k_final = int(elbow_k)  # o bé int(best_sil_k) si prefereixes basar-te en␣
↪silueta
km_final = KMeans(n_clusters=k_final, n_init=50, random_state=42)
```

```python
labels_pred = km_final.fit_predict(df_blobs[['x1', 'x2']].to_numpy())

# Convertim a Series per mantenir l'index i poder-la enganxar fàcilment
labels_s = pd.Series(labels_pred, index=df_blobs.index, name='cluster_kmeans')

# Afegim la predicció com a nova columna (transformació DataFrame <- concat/
 ↪assign)
df_blobs = df_blobs.assign(cluster_kmeans=labels_s)

# Visualització ràpida del resultat amb prediccions
fig, ax = plt.subplots(figsize=(5, 5))
scatter = ax.scatter(df_blobs['x1'], df_blobs['x2'],␣
 ↪c=df_blobs['cluster_kmeans'],
                     s=15, alpha=.5, cmap='jet')
ax.set_title(f'k-means amb k={k_final} - Blobs')
ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.axis('equal'); plt.tight_layout()
plt.show()

# Centroides (els traiem com a DataFrame per coherència d'estil)
centroids_df = pd.DataFrame(km_final.cluster_centers_, columns=['x1_centroid',␣
 ↪'x2_centroid'])
print(centroids_df.round(3))
```




```
   x1_centroid  x2_centroid
0       -2.527        9.028
1       -6.665       -6.774
2        4.668        2.059
3       -8.929        7.460
```

Què hem fet aquí:

- Ens quedem amb el k escollit i entrenem el model definitiu (KMeans).
- Convertim les etiquetes predites a pd.Series i les enganxem al nostre DataFrame.
- Deixem els centroides com un DataFrame separat (molt útil si volem conservarlos "com a model" per a ús predictiu posterior).

Notes teòriques per memòria

- SSE (inertia) és la suma de distàncies euclidianes al quadrat de cada punt al seu centroide de clúster. A k-means el centroide és la mitjana del clúster.
- La regla del colze tria el k on la millora marginal (caiguda de SSE) deixa de ser significativa.

- La silueta complementa el criteri: valors alts impliquen clústers cohesionats i separats.
- K-means funciona millor quan els clústers són convexos i d'escala comparable; és sensible a l'escala i a la inicialització (d'aquí n_init i random_state).

Anàlisi: Què ha passat? Explica els motius pels quals creus que s'ha produït aquest resultat.

L'algoritme K-means ha identificat correctament els clusters amb la regla del colze al dataset blobs perquè els clusters són convexos.

Implementació: Càlcul i visualització de la regla del colze al dataset Moons.

1) Convertim a pandas i deixem-ho tot indexat net

[9]:
```
# ----------------------------------------------------------------------
# Muntem el DataFrame de treball per a "Moons"
#    - Estructura: df_moons = ['x1','x2','etiqueta_real']
#    - Nota: 'etiqueta_real' s'usarà només per a visualitzar/avaluar, no per␣
↪entrenar.
# ----------------------------------------------------------------------
df_moons = (
    pd.DataFrame(X_moons, columns=['x1', 'x2'])
      .assign(etiqueta_real=pd.Series(y_moons, index=None))
)
print(df_moons.head(3))
```

```
         x1        x2  etiqueta_real
0 -0.033411  0.421391              1
1  0.998827 -0.442890              1
2  0.889592 -0.327843              1
```

[10]:
```
# ----------------------------------------------------------------------
# FUNCIÓ D'ANÀLISI DEL COLZE (VERSIÓ "MOONS")
# ----------------------------------------------------------------------

def elbow_table_kmeans_moons(df, ks=range(1, 11), n_init=20, seed=SEED):
    """
    Itera k = 1..Kmax, entrena KMeans sobre 'Moons' i recull:
      - SSE (inertia_ a sklearn)  -> suma de distàncies quadrades al centroide.
      - Silhouette (k >= 2)       -> cohesió/separació dels clústers.

    Context específic Moons:
      - La forma és NO CONVEXA (dues mitges llunes). El "colze" pot no ser molt␣
↪marcat
        i la silueta pot preferir valors de k > 2 perquè k-means tendeix a␣
↪"trossejar" les llunes.

    Parameters:
      - df: DataFrame amb columnes ['x1', 'x2'] (podria tenir més columnes;␣
↪aquí n'usem 2).
```

```
    - ks: rang de valors de k a provar (iterable d'enters).
    - n_init: nº d'inicis aleatoris (k-means++). Ajuda a estabilitzar␣
↪resultats.
    - seed: llavor de random per fer el procés reproduïble (es farà servir␣
↪SEED).

  Returns:
    - res_df: DataFrame indexat per k amb columnes:
        ['SSE', 'ΔSSE', 'curv', 'silhouette']
        on ΔSSE = SSE(k-1) - SSE(k) i curv és la 2a diferència discreta␣
↪(aprox. "inflexió")
    - elbow_k: enter amb la proposta de k pel colze (màxima curvatura,␣
↪ignorant k molt petits)
    - best_sil_k: enter amb el k que maximitza la silueta (k>=2)
  """
  # 1) Secció de dades d'entrada:
  #    Extraiem només les columnes numèriques que volem agrupar (aquí, x1 i␣
↪x2).
  X = df[['x1', 'x2']].to_numpy()  # np.ndarray (n,2). Fem servir NumPy per a␣
↪sklearn.

  # 2) Diccionaris on anirem acumulant mètriques per a cada k.
  sse_dict = {}   # dict int -> float
  sil_dict = {}   # dict int -> float (o np.nan per k=1)

  # 3) Bucle principal sobre els k candidats
  for k in ks:
      km = KMeans(n_clusters=k, n_init=n_init, random_state=SEED)

      '''
      - K-means particiona els punts de X en k grups, i per a cada grup␣
↪calcula un
        centroide (la mitjana dels punts del clúster).
      - Necessita centroids inicials; si són "dolents", pots caure en un␣
↪mínim local.
      - Per reduir-ho, scikit-learn:
          * repeteix k-means diverses vegades amb inicialitzacions diferents␣
↪(n_init),
          * es queda amb la millor solució segons SSE (inertia_ mínima).
        n_init indica quantes execucions amb diferents punts inicials.
        random_state=SEED fa que el procés sigui reproduïble.

      Paràmetres per defecte rellevants:
        init='k-means++'  → inicialització "llesta" (redueix mals inicis).
        max_iter=300      → límit d'iteracions per execució.
        tol=1e-4          → criteri de parada (canvi mínim admissible).
```

```
        Algorisme (per cada execució):
          1) Assignar cada punt al centroide més proper (distància euclidiana).
          2) Recalcular centroide com la mitjana del clúster.
          3) Repetir fins convergència / max_iter.
        Es tria la millor execució i es guarden:
          km.cluster_centers_ → coordenades dels centroids.
          km.labels_          → etiqueta de clúster de cada punt.
          km.inertia_         → SSE final (la guardem a sse_dict[k]).
        '''

        # Entrenem i obtenim etiquetes predites com a np.ndarray (n,)
        labels = km.fit_predict(X)

        # SSE (inertia_) és un escalar float: suma de distàncies quadràtiques␣
↪punt-centroide
        sse_dict[k] = float(km.inertia_)

        # Silueta: només definida per k >= 2. Guardem np.nan per k=1.
        sil_dict[k] = (
            float(silhouette_score(X, labels, metric='euclidean'))
            if k >= 2 else np.nan
        )

    # 4) Convertim els diccionaris a Series per aprofitar l'index per k i fer␣
↪càlculs vectorials:
    sse_s = pd.Series(sse_dict, name='SSE').sort_index()           # Series␣
↪(index=k, val=SSE)
    dSSE  = sse_s.shift(1) - sse_s                                  # 1a␣
↪diferència (caiguda de SSE)
    curv  = dSSE.shift(1) - dSSE                                    # 2a␣
↪diferència (aprox. curvatura)
    sil_s = pd.Series(sil_dict, name='silhouette').sort_index()    # Series␣
↪(index=k, val=silhouette)

    # 5) Taula final: DataFrame amb totes les mètriques alineades per index k.
    res_df = pd.concat(
        [sse_s, dSSE.rename('ΔSSE'), curv.rename('curv'), sil_s],
        axis=1
    )

    # 6) Propostes automàtiques:
    #    - "Colze": triem el k amb màxima curvatura, però ignorem els 2 primers␣
↪k per evitar soroll.
    elbow_k = curv.iloc[2:].idxmax()
    #    - "Millor silueta": k (>=2) que maximitza silhouette
```

```
    best_sil_k = sil_s.iloc[1:].idxmax()

    return res_df, elbow_k, best_sil_k

# ---------------------------------------------------------------------
# EXECUCIÓ SOBRE "MOONS" (llançant la funció anterior)
# ---------------------------------------------------------------------
elbow_df_moons, elbow_k_moons, best_sil_k_moons = elbow_table_kmeans_moons(
    df_moons, ks=range(1, 11), n_init=20, seed=SEED
)

# Inspeccionem la taula (DataFrame): cada fila és un k; columnes = mètriques
print(elbow_df_moons.round(3))
print(f"\nProposta pel colze (curvatura màxima): k = {elbow_k_moons}")
print(f"Millor k per silueta: k = {best_sil_k_moons}")
```

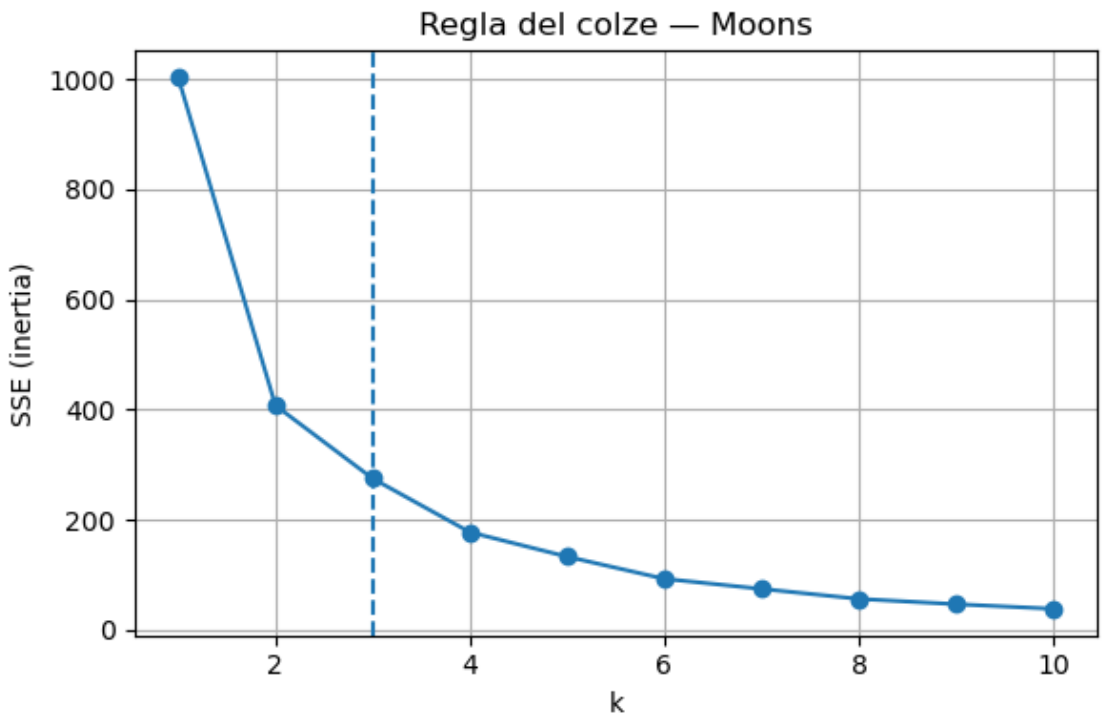
```
         SSE     ΔSSE     curv  silhouette
1   1003.469      NaN      NaN         NaN
2    407.856  595.613      NaN       0.486
3    275.155  132.701  462.911       0.423
4    177.664   97.491   35.211       0.457
5    133.248   44.416   53.075       0.482
6     92.980   40.268    4.148       0.511
7     74.669   18.310   21.958       0.514
8     56.552   18.117    0.193       0.514
9     46.930    9.622    8.495       0.516
10    38.561    8.370    1.252       0.509

Proposta pel colze (curvatura màxima): k = 3
Millor k per silueta: k = 9
```

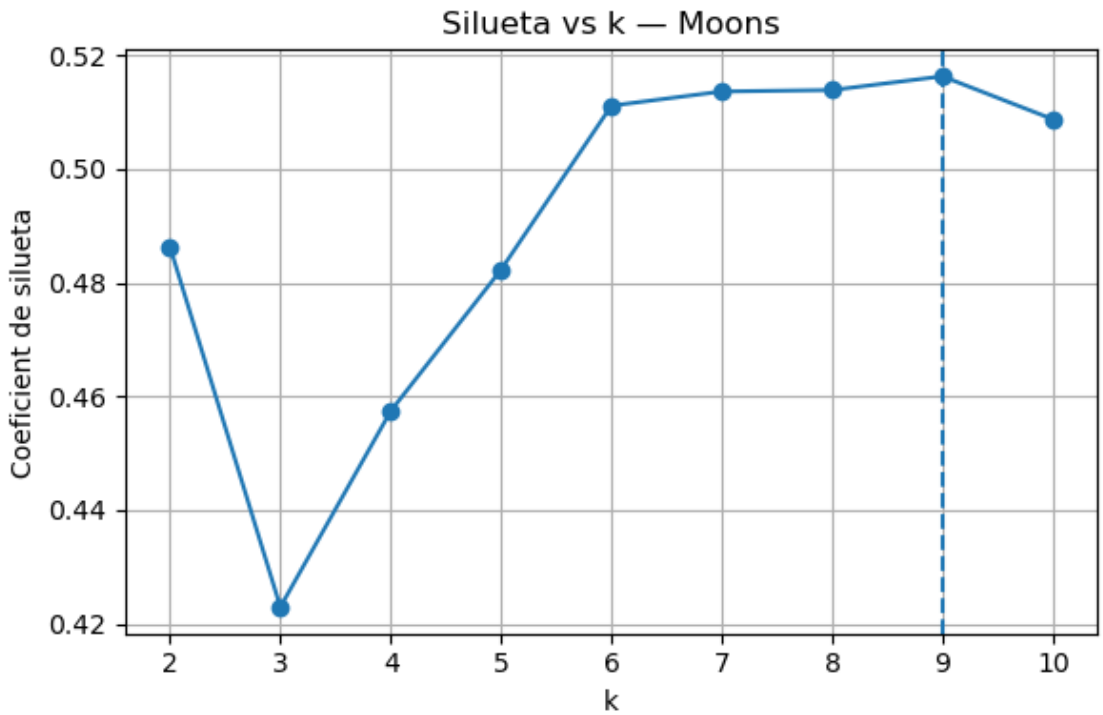
Què acabem de fer (resum conceptual)

- Diccionaris per acumular mètriques {k: valor} → fàcil convertir a Series (pandas) i operar per index.
- Series per fer diferències (shift) i derivades discretes → generem ΔSSE i curv.
- DataFrame final (elbow_df) per tenir tot plegat (SSE, caiguda, curvatura, silueta) i poder justificar la tria de k.

[11]:
```
# ---------------------------------------------------------------------
# VISUALITZACIÓ DEL COLZE I LA SILUETA PER "MOONS"
# ---------------------------------------------------------------------
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(elbow_df_moons.index, elbow_df_moons['SSE'], marker='o')
ax.set_xlabel('k'); ax.set_ylabel('SSE (inertia)')
ax.set_title('Regla del colze - Moons'); ax.grid(True)
ax.axvline(elbow_k_moons, linestyle='--'); plt.tight_layout(); plt.show()
```

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(elbow_df_moons.index, elbow_df_moons['silhouette'], marker='o')
ax.set_xlabel('k'); ax.set_ylabel('Coeficient de silueta')
ax.set_title('Silueta vs k - Moons'); ax.grid(True)
ax.axvline(best_sil_k_moons, linestyle='--'); plt.tight_layout(); plt.show()
```





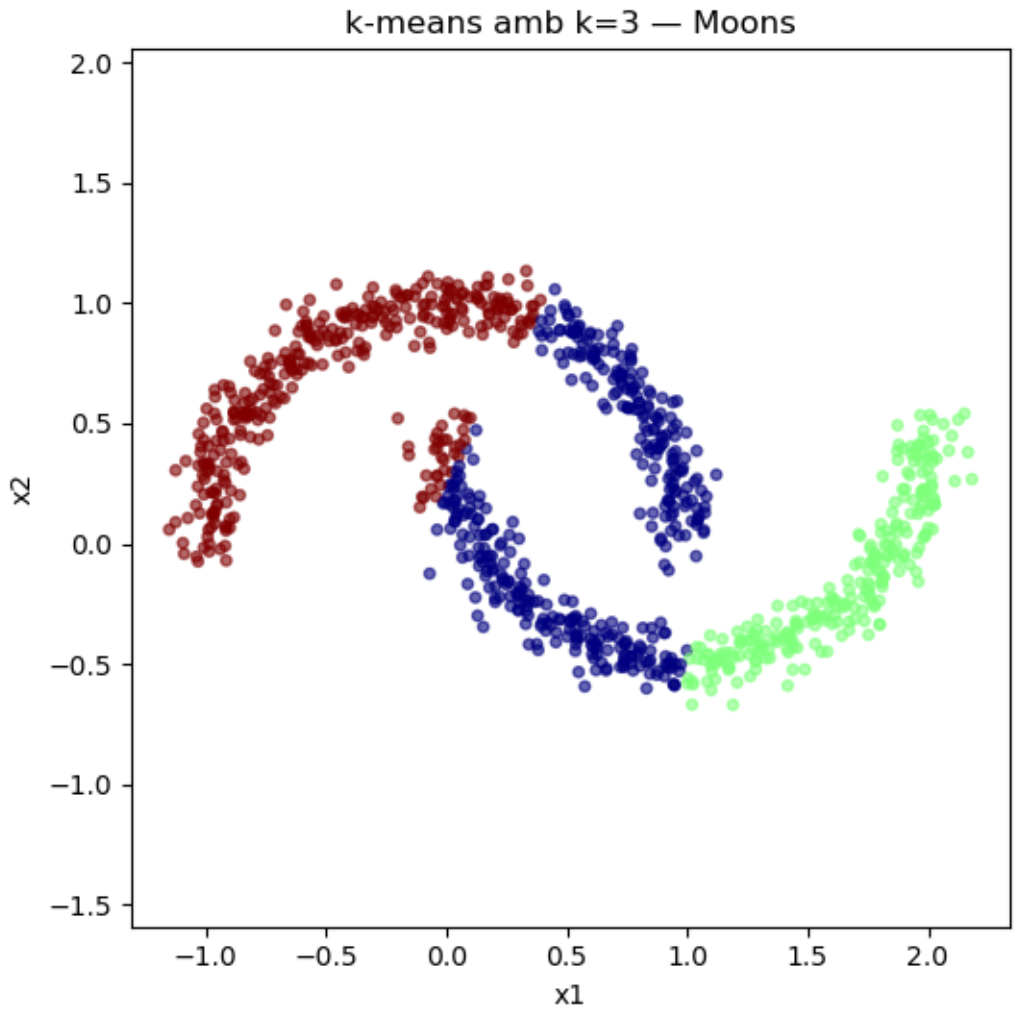
Anàlisi: Què s'interpreta a la gràfica? Com es podria millorar l'elecció de k?.

El colze suggereix k=3 però la silueta és màxima a k=9 i que aquesta discrepància s'explica perquè k-means no modela bé formes no convexes. Una millora és canviar d'algorisme (DBSCAN/HDBSCAN o espectral amb k=2)

Implementació: Càlcul i visualització dels grups al dataset Moons.

[12]:
```
# ----------------------------------------------------------------------
# ENTRENAMENT FINAL AMB EL k TRIAT - MOONS
# ----------------------------------------------------------------------
# Estructures:
#   - km_final_moons : objecte KMeans entrenat (centroides i labels_ internes)
#   - labels_s       : Series amb les etiquetes predites alineades amb l'index␣
↪del DataFrame
#   - df_moons_pred  : DataFrame original + nova columna 'cluster_kmeans'
# ----------------------------------------------------------------------

# Triem k (es pots canviar a best_sil_k_moons per comparar, el resultat és mes␣
↪omogeni però no represente adecuadament
# els clusters)
k_final_moons = int(elbow_k_moons)
# k_final_moons = int(best_sil_k_moons)
```

```python
# 1) Ajust del model
km_final_moons = KMeans(n_clusters=k_final_moons, n_init=50, random_state=SEED)

# 2) Entrenem i prediem
labels_pred = km_final_moons.fit_predict(df_moons[['x1', 'x2']].to_numpy())

# 3) Convertim a Series per conservar index i enganxar al DataFrame
labels_s = pd.Series(labels_pred, index=df_moons.index, name='cluster_kmeans')

# 4) Afegim la predicció com a nova columna
df_moons_pred = df_moons.assign(cluster_kmeans=labels_s)

# 5) Visualització: NOMÉS punts.
fig, ax = plt.subplots(figsize=(5.5, 5.5))
ax.scatter(df_moons_pred['x1'], df_moons_pred['x2'],
           c=df_moons_pred['cluster_kmeans'], s=15, alpha=.6, cmap='jet')
ax.set_title(f'k-means amb k={k_final_moons} - Moons')
ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.axis('equal'); plt.tight_layout()
plt.show()

# 6) Centroides (els traiem com a DataFrame per coherència d'estil) - es␣
↪MOSTREN PER PANTALLA
centroids_moons_df = pd.DataFrame(
    km_final_moons.cluster_centers_,
    columns=['x1_centroid', 'x2_centroid']
)
print(centroids_moons_df.round(3))
```




```
   x1_centroid  x2_centroid
0        0.590        0.125
1        1.639       -0.135
2       -0.428        0.668
```

Anàlisi: Què ha passat? Explica els motius pels quals creus que s'ha produït aquest resultat.

Amb k=3, k-means ha partit la lluna superior en 2 trams i la lluna inferior en tres. El motiu és que k-means usa fronteres lineals i busca clústers convexos; per a formes corbades, el millor que pot fer és tallar-les en trossos. Això també explica que la silueta prefereixi k més grans: cada tros és més compacte i la mètrica puja lleugerament, però no reflecteix l'estructura "natural" de 2 llunes.

[13]:
```
X, y = X_circles, y_circles
```

Implementació: Càlcul i visualització de la regla del colze al dataset Circles.

[14]:
```
# ----------------------------------------------------------------------
# DATAFRAME DE TREBALL PER A "CIRCLES"
#    - Estructura: df_circles = ['x1','x2','etiqueta_real']
#    - Nota: 'etiqueta_real' s'usa només per visualitzar/avaluar, NO per␣
↪entrenar.
# ----------------------------------------------------------------------
df_circles = (
    pd.DataFrame(X, columns=['x1', 'x2'])
      .assign(etiqueta_real=pd.Series(y, index=None))
)
print(df_circles.head(3))
```

```
         x1        x2  etiqueta_real
0  0.452592  0.168433              1
1 -0.438027  0.119900              1
2 -0.532224  0.184359              1
```

[15]:
```
# ----------------------------------------------------------------------
# FUNCIÓ D'ANÀLISI DEL COLZE (VERSIÓ "CIRCLES")
# ----------------------------------------------------------------------

def elbow_table_kmeans_circles(df, ks=range(1, 11), n_init=20, seed=SEED):
    """
    Itera k = 1..Kmax, entrena KMeans sobre 'Circles' i recull:
      - SSE (inertia_ a sklearn)  -> suma de distàncies quadrades al centroide.
      - Silhouette (k >= 2)       -> cohesió/separació dels clústers.

    Context específic Circles
    -------------------------
    - Les classes naturals són **anells concèntrics** (no convexos).
    - K-means assumeix clústers convexos amb fronteres lineals (Voronoi); per␣
↪minimitzar SSE,
      acostuma a **partir els anells en sectors** quan k creix. El "colze" pot␣
↪ser difús i la
      silueta pot millorar lleugerament amb k grans per sobre-segmentació.

    Parameters
    ----------
    df : DataFrame amb columnes ['x1','x2'] (podria tenir més columnes; aquí␣
↪n'usem 2).
    ks : iterable d'enters, rang de valors de k a provar.
    n_init : int, nº d'inicis aleatoris (k-means++) per estabilitat.
    seed : int, llavor de random per fer el procés reproduïble (es farà servir␣
↪SEED).

    Returns
    -------
```

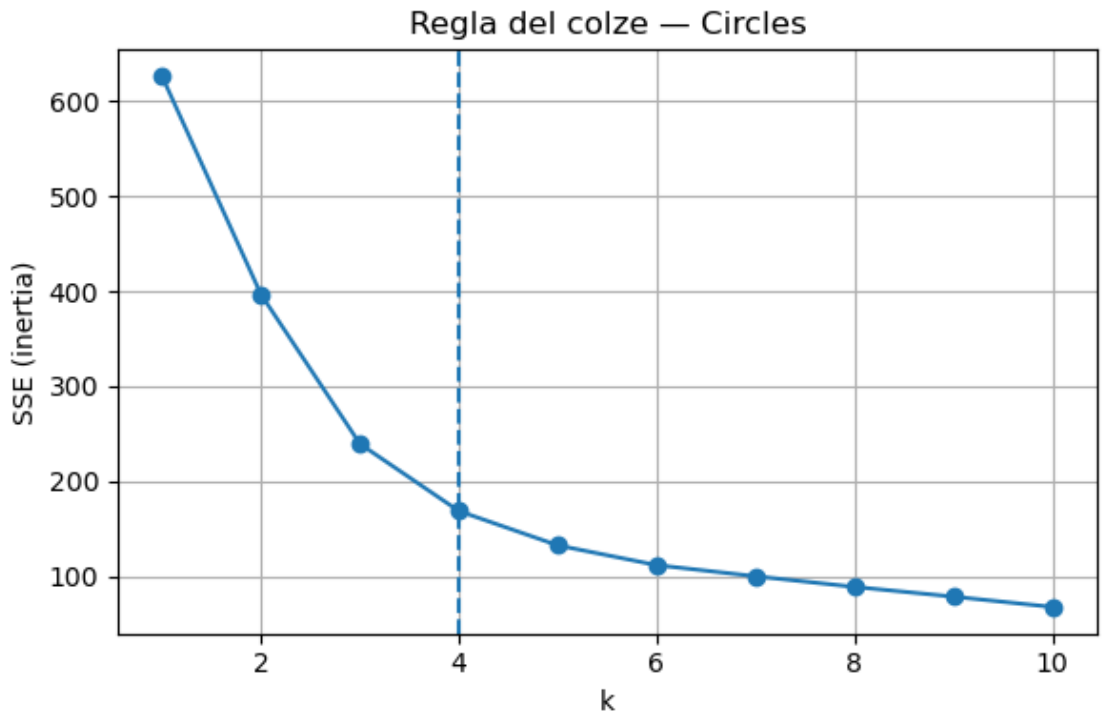
```
    res_df : DataFrame indexat per k amb columnes:
        ['SSE', 'ΔSSE', 'curv', 'silhouette']
        on ΔSSE = SSE(k-1) - SSE(k) i 'curv' és la 2a diferència discreta␣
↪(aprox. "inflexió").
    elbow_k : int
        Proposta de k pel colze (màxima curvatura, ignorant k molt petits).
    best_sil_k : int
        k que maximitza la silueta (k>=2).
    """
    # 1) Secció de dades d'entrada:
    #    Extraiem només les columnes numèriques que volem agrupar (aquí, x1 i␣
↪x2).
    X = df[['x1', 'x2']].to_numpy()  # np.ndarray (n,2). Fem servir NumPy per a␣
↪sklearn.

    # 2) Diccionaris on anirem acumulant mètriques per a cada k.
    sse_dict = {}   # dict int -> float
    sil_dict = {}   # dict int -> float (o np.nan per k=1)

    # 3) Bucle principal sobre els k candidats
    for k in ks:
        km = KMeans(n_clusters=k, n_init=n_init, random_state=SEED)

        '''
        - K-means particiona X en k grups i calcula el centroide (mitjana) de␣
↪cada grup.
        - Repeteix el procés amb inicialitzacions diferents (n_init) i es queda␣
↪amb la millor
          solució segons SSE (km.inertia_ mínima). random_state=SEED garanteix␣
↪reproductibilitat.

        Paràmetres rellevants per defecte:
          init='k-means++'  → inicialització "llesta".
          max_iter=300, tol=1e-4 → criteri de parada.

        Algorisme (per execució):
          1) Assignar cada punt al centroide més proper (euclidiana),
          2) Recalcular centroides com a mitjanes,
          3) Repetir fins convergència/max_iter.
        Desem:
          km.cluster_centers_ (centroides), km.labels_ (etiquetes), km.inertia_␣
↪(SSE).
        '''

        # Entrenem i obtenim etiquetes predites com a np.ndarray (n,)
        labels = km.fit_predict(X)
```

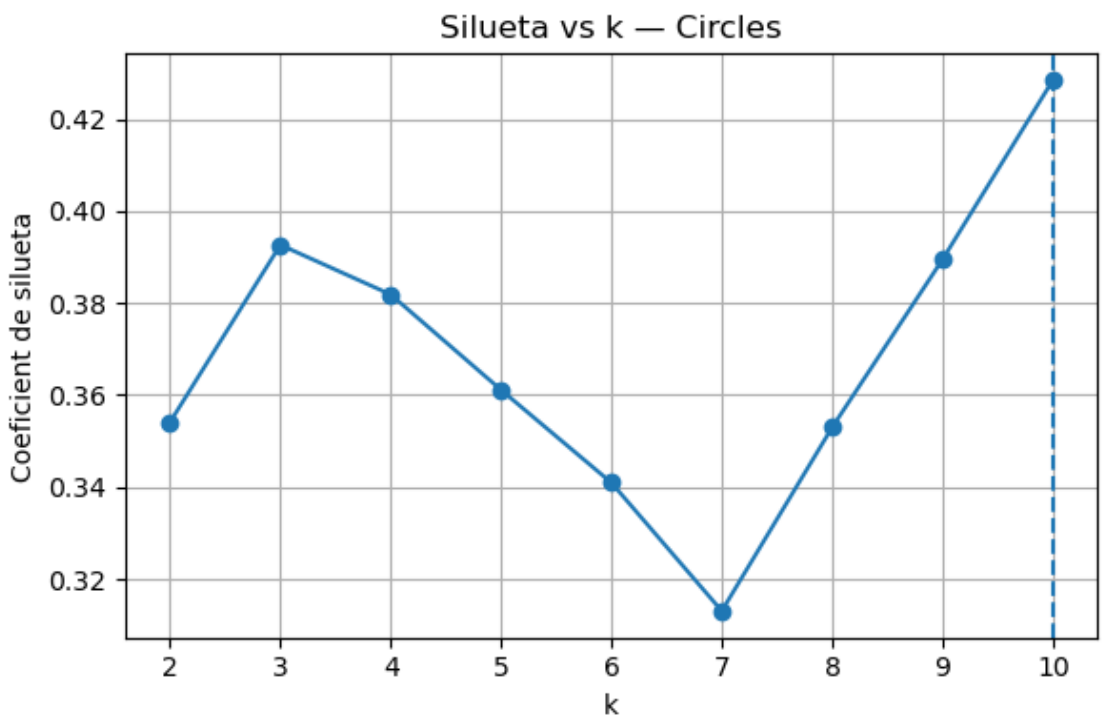
```
        # SSE (inertia_) és un escalar float: suma de distàncies quadrades␣
↪punt-centroide
        sse_dict[k] = float(km.inertia_)

        # Silueta: només definida per k >= 2. Guardem np.nan per k=1.
        sil_dict[k] = (
            float(silhouette_score(X, labels, metric='euclidean'))
            if k >= 2 else np.nan
        )

    # 4) Convertim els diccionaris a Series per aprofitar l'index per k i fer␣
↪càlculs vectorials:
    sse_s = pd.Series(sse_dict, name='SSE').sort_index()           # Series␣
↪(index=k, val=SSE)
    dSSE  = sse_s.shift(1) - sse_s                                 # 1a␣
↪diferència (caiguda de SSE)
    curv  = dSSE.shift(1) - dSSE                                   # 2a␣
↪diferència (aprox. curvatura)
    sil_s = pd.Series(sil_dict, name='silhouette').sort_index()    # Series␣
↪(index=k, val=silhouette)

    # 5) Taula final: DataFrame amb totes les mètriques alineades per index k.
    res_df = pd.concat(
        [sse_s, dSSE.rename('ΔSSE'), curv.rename('curv'), sil_s],
        axis=1
    )

    # 6) Propostes automàtiques:
    #    - "Colze": triem el k amb màxima curvatura, però ignorem els 2 primers␣
↪k per evitar soroll.
    elbow_k = curv.iloc[2:].idxmax()
    #    - "Millor silueta": k (>=2) que maximitza silhouette
    best_sil_k = sil_s.iloc[1:].idxmax()

    return res_df, elbow_k, best_sil_k
```

[16]:
```
# ----------------------------------------------------------------------
# EXECUCIÓ SOBRE "CIRCLES" (càlcul + visualització)
# ----------------------------------------------------------------------
elbow_df_circ, elbow_k_circ, best_sil_k_circ = elbow_table_kmeans_circles(
    df_circles, ks=range(1, 11), n_init=20, seed=SEED
)

# Taula de mètriques per k
print(elbow_df_circ.round(3))
```

```python
print(f"\nProposta pel colze (curvatura màxima): k = {elbow_k_circ}")
print(f"Millor k per silueta: k = {best_sil_k_circ}")

# Gràfic del colze (SSE)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(elbow_df_circ.index, elbow_df_circ['SSE'], marker='o')
ax.set_xlabel('k'); ax.set_ylabel('SSE (inertia)')
ax.set_title('Regla del colze - Circles'); ax.grid(True)
ax.axvline(elbow_k_circ, linestyle='--')
plt.tight_layout(); plt.show()

# Gràfic de Silueta
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(elbow_df_circ.index, elbow_df_circ['silhouette'], marker='o')
ax.set_xlabel('k'); ax.set_ylabel('Coeficient de silueta')
ax.set_title('Silueta vs k - Circles'); ax.grid(True)
ax.axvline(best_sil_k_circ, linestyle='--')
plt.tight_layout(); plt.show()
```

```
        SSE     ΔSSE    curv  silhouette
1   626.241      NaN     NaN         NaN
2   396.612  229.629     NaN       0.354
3   239.271  157.341  72.288       0.393
4   168.516   70.755  86.586       0.382
5   132.377   36.139  34.617       0.361
6   111.559   20.818  15.320       0.341
7    99.810   11.749   9.069       0.313
8    88.637   11.172   0.577       0.353
9    78.364   10.273   0.899       0.389
10   67.510   10.854  -0.581       0.428

Proposta pel colze (curvatura màxima): k = 4
Millor k per silueta: k = 10
```


Regla del colze — Circles
SSE (inertia)
600
500
400
300
200
100
2
4
6
8
10
k


Silueta vs k — Circles
Coeficient de silueta
0.42
0.40
0.38
0.36
0.34
0.32
2
3
4
5
6
7
8
9
10
k

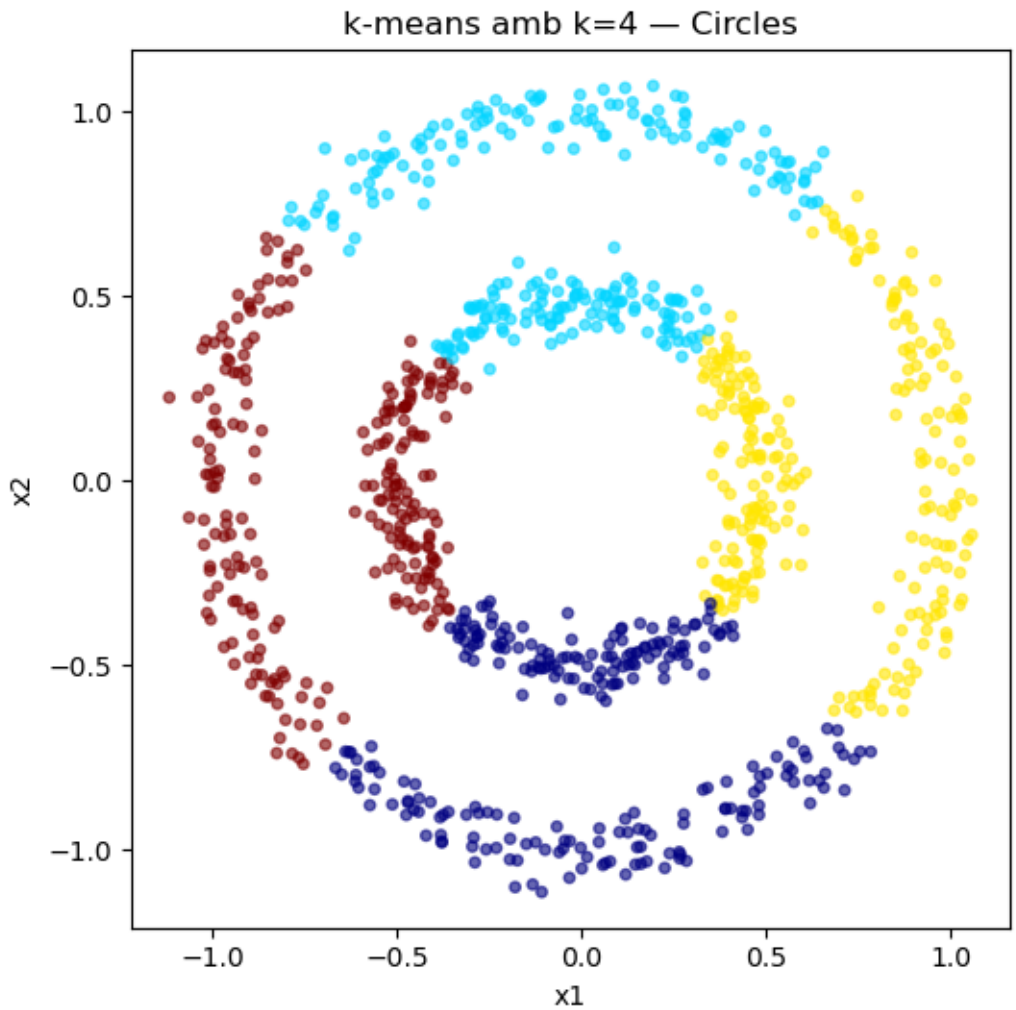
Anàlisi: Què s'interpreta a la gràfica? Com es podria millorar l'elecció de k?.

El colze suggereix k=4 però la silueta és màxima a k=10, aquesta discrepància s'explica perquè k-means no modela bé formes no convexes. Una millora és canviar d'algorisme (DBSCAN/HDBSCAN o espectral amb k=2)

Implementació: Càlcul i visualització dels grups al dataset Circles.

[17]:
```
# ---------------------------------------------------------------------
# ENTRENAMENT FINAL AMB EL k TRIAT - CIRCLES
# ---------------------------------------------------------------------
# Estructures:
#   - km_final_circ  : objecte KMeans entrenat (centroides i labels_ internes)
#   - labels_s       : Series amb les etiquetes predites alineades amb l'index␣
 ↪del DataFrame
#   - df_circles_pred: DataFrame original + nova columna 'cluster_kmeans'
# Notes:
#   - Per a "circles" (formes no convexes) sol ser més prudent usar el k del␣
 ↪colze.
#     Si vols comparar amb silueta, canvia la línia de selecció de k.
# ---------------------------------------------------------------------

# Triem k (pots provar també best_sil_k_circ si vols comparar)

k_final_circ = int(elbow_k_circ)
# k_final_circ = int(best_sil_k_circ)

# 1) Ajust del model
km_final_circ = KMeans(n_clusters=k_final_circ, n_init=50, random_state=SEED)

# 2) Entrenem i prediem (labels: np.ndarray (n,))
X_circ_np = df_circles[['x1', 'x2']].to_numpy()   # DataFrame -> ndarray (n,2)␣
 ↪per a sklearn
labels_pred = km_final_circ.fit_predict(X_circ_np)

# 3) Convertim les etiquetes a Series per conservar l'index i enganxar-les al␣
 ↪DataFrame
labels_s = pd.Series(labels_pred, index=df_circles.index, name='cluster_kmeans')

# 4) Construïm el DataFrame de sortida amb la nova columna de clúster
df_circles_pred = df_circles.assign(cluster_kmeans=labels_s)

# 5) Visualització: només punts acolorits pel clúster predit (SENSE centroides)
fig, ax = plt.subplots(figsize=(5.5, 5.5))
ax.scatter(df_circles_pred['x1'], df_circles_pred['x2'],
           c=df_circles_pred['cluster_kmeans'], s=15, alpha=.6, cmap='jet')
ax.set_title(f'k-means amb k={k_final_circ} - Circles')
ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.axis('equal'); plt.tight_layout()
```

```
plt.show()

# 6) Centroides (els traiem com a DataFrame per coherència d'estil) - es␣
↪MOSTREN PER PANTALLA
centroids_circ_df = pd.DataFrame(
    km_final_circ.cluster_centers_,
    columns=['x1_centroid', 'x2_centroid']
)
print(centroids_circ_df.round(3))
```




```
   x1_centroid  x2_centroid
0        0.024       -0.679
1       -0.043        0.677
2        0.665        0.028
3       -0.681       -0.039
```

Anàlisi: Què ha passat? Explica els motius pels quals creus que s'ha produït aquest resultat.

Amb k=4, k-means ha partit cadascún dels cercles en 4 porcions. El motiu és que k-means usa fronteres lineals i busca clústers convexos; per a formes corbades, el millor que pot fer és tallar-les en trossos. Això també explica que la silueta prefereixi k més grans: cada tros és més compacte i la mètrica puja lleugerament, però no reflecteix l'estructura "natural" de 2 cercles concèntrics.

1.1.3 1 b. Algorismes basats en densitat: DBScan

En aquest apartat es demana aplicar clustering per densitat com DBSCAN als datasets anteriors per detectar els grups subjacents.

[18]:
```
X, y = X_blobs, y_blobs
```

Implementació: Prova la implementació de DBSCAN a scikit-learn jugant amb els paràmetres eps i min_samples per trobar els grups (i outliers) del dataset Blobs.

Crearem 3 funcions per aplicar el mètode de classificació DBSCAN:

- plot_k_distance(df, min_samples): La regla del colze tria el k on la millora marginal (caiguda de SSE) deixa de ser significativa. Calcula la distància al k-èssim veí (k=min_samples) per a cada punt, les ordena i en fa el k-dist plot. Per a què serveix: Localitzar el genoll de la corba i obtenir un rang inicial d'eps raonable. Output clau: vector de distàncies ordenades (per inspeccionar el genoll).
- sweep_dbscan(df, eps_list, min_samples_list): Prova una graella de parelles (eps, min_samples) amb DBSCAN i en recull: clústers, soroll i silhouette (només punts no soroll). Per a què serveix: Refinar la tria al voltant del rang d'eps vist al k-dist i comparar diferents min_samples.
- dbscan_fit_predict_plot(df, eps, min_samples): Ajusta DBSCAN amb els paràmetres escollits, afegeix cluster_dbscan al DataFrame i plota punts (clústers en colors, soroll en gris). Calcula la silhouette dels punts no soroll. Per a què serveix: Resultat final i validació visual/numèrica. Output clau: df_out (amb etiquetes) i counts (recompte per etiqueta)

[19]:
```
# ---------------------------------------------------------------------
# 1) K-DIST PLOT (EINA PER TRIAR eps DONAT min_samples)
# ---------------------------------------------------------------------
def plot_k_distance(df, min_samples=5, ds_name=None):
    """
    Dibuixa la corba de 'k-distància' (distància al k-èssim veí més proper)
    per ajudar a triar 'eps' a DBSCAN. Fem servir distàncies euclidianes.

    Idea
    ----
    Ordenem les distàncies al k-èssim veí (k = min_samples) de tots els punts.
    El "genoll" (elbow) d'aquesta corba és un bon punt de partida per 'eps'.

    Paràmetres
    ----------
    df : pandas.DataFrame amb columnes ['x1','x2'] (les que s'agrupen)
```

```
    min_samples : int
        Valor de k per al veí-èssim (mateix que a DBSCAN).
    ds_name : str | None
        Nom del dataset per mostrar al títol (p. ex. "Blobs", "Moons",␣
↪"Circles").

    Sortida
    -------
    Mostra un gràfic; retorna també el vector de distàncies ordenades (np.
↪ndarray)
    per si es vol inspeccionar numèricament.

    Com s'interpreta
    ----------------
    Tram pla (pendent suau): punts "normals" dins de zones d'alta densitat.
    "Genoll"/salt sobtat: separa punts densos dels bordes/soroll.
    Tria d'eps: agafa un valor a l'alçada del genoll o lleugerament per sota.
    """
    # DataFrame -> ndarray (n,2)
    X = df[['x1', 'x2']].to_numpy()

    # Matriu de distàncies euclidianes (n x n)
    D = pairwise_distances(X, metric='euclidean')

    # Per a cada fila (punt), ordenem distàncies i agafem la del k-èssim veí
    k = int(min_samples)
    kth = np.sort(D, axis=1)[:, k]  # columna k (0=self, 1=1r veí, ...,␣
↪k=k-èssim)

    # Ordenem globalment per veure el genoll
    kth_sorted = np.sort(kth)

    # Plot
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(kth_sorted)
    title_ds = f" - {ds_name}" if ds_name else ""
    ax.set_title(f'k-dist (k=min_samples={min_samples}){title_ds}')
    ax.set_xlabel('Punts ordenats'); ax.set_ylabel('Distància al k-èssim veí')
    ax.grid(True); plt.tight_layout(); plt.show()

    return kth_sorted


# ---------------------------------------------------------------------
# 2) EXPLORACIÓ D'EPS × MIN_SAMPLES (AMB SILUETA I CONTEIGS)
# ---------------------------------------------------------------------
def sweep_dbscan(df, eps_list, min_samples_list):
```

```
    """
    Prova una graella d'(eps, min_samples) i recull mètriques per ajudar a␣
↪escollir.

    Mètriques
    ---------
    - n_clusters : nº de clústers trobats (exclou soroll -1)
    - n_noise    : nº d'outliers (etiqueta -1)
    - silhouette : silueta sobre punts NO-SOROLL (si hi ha com a mínim 2␣
↪clústers)

    Retorna
    -------
    pandas.DataFrame amb files per combinació de paràmetres i les mètriques␣
↪anteriors.
    """
    X = df[['x1', 'x2']].to_numpy()

    rows = []
    for ms in min_samples_list:
        for eps in eps_list:
            # Ajust DBSCAN
            db = DBSCAN(eps=float(eps), min_samples=int(ms), metric='euclidean')
            labels = db.fit_predict(X)

            # Soroll i clústers
            noise_mask = labels == -1
            labels_nonoise = labels[~noise_mask]
            unique_clusters = np.unique(labels_nonoise)

            # Mètriques
            n_clusters = int(len(unique_clusters))
            n_noise    = int(noise_mask.sum())

            # Silueta (només si hi ha >=2 clústers i >=2 punts sense soroll)
            if n_clusters >= 2 and labels_nonoise.size > 1:
                sil = float(silhouette_score(X[~noise_mask], labels_nonoise,␣
↪metric='euclidean'))
            else:
                sil = np.nan

            rows.append({
                'eps': float(eps),
                'min_samples': int(ms),
                'n_clusters': n_clusters,
                'n_noise': n_noise,
                'silhouette': sil
```

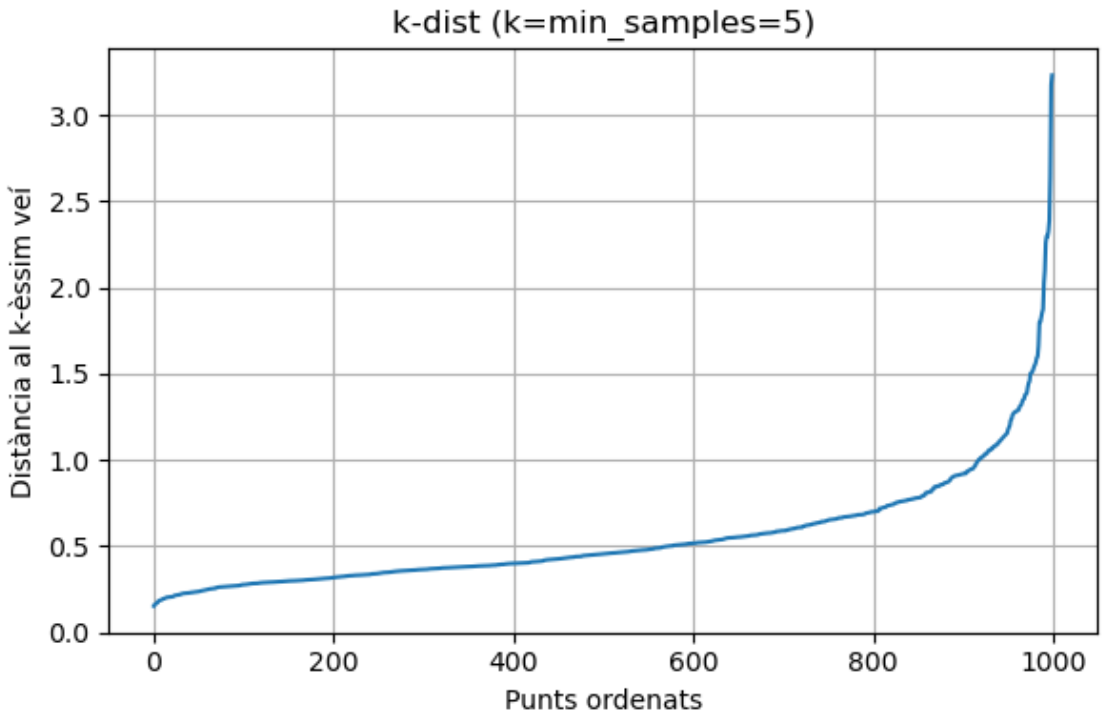
```
            })

    res_df = pd.DataFrame(rows).sort_values(['min_samples','eps']).
↪reset_index(drop=True)
    return res_df


# ----------------------------------------------------------------------
# 3) AJUST FINAL + VISUALITZACIÓ (PUNTS I OUTLIERS EN GRIS)
# ----------------------------------------------------------------------
def dbscan_fit_predict_plot(df, eps, min_samples):
    """
    Ajusta DBSCAN amb (eps, min_samples), retorna etiquetes i mostra el gràfic.

    Visualització
    -------------
    - Punts de clúster: acolorits per etiqueta (0..C-1)
    - Outliers (-1): color gris i alpha més baix
    - No es dibuixen "centroides" (DBSCAN no en té). S'imprimeix un resum.

    Retorna
    -------
    df_out : DataFrame = df + 'cluster_dbscan'
    counts : dict amb {etiqueta: recompte} (inclou -1 si hi ha soroll)
    """
    X = df[['x1','x2']].to_numpy()

    db = DBSCAN(eps=float(eps), min_samples=int(min_samples),␣
↪metric='euclidean')
    labels = db.fit_predict(X)

    # Afegim etiquetes al DataFrame
    df_out = df.assign(cluster_dbscan=pd.Series(labels, index=df.index))

    # Resum comptatges
    counts = dict(Counter(labels))

    # Visualització
    fig, ax = plt.subplots(figsize=(5.5, 5.5))

    # Soroll en gris
    noise_mask = labels == -1
    ax.scatter(df_out.loc[noise_mask, 'x1'], df_out.loc[noise_mask, 'x2'],
               c='lightgray', s=12, alpha=.35, label=f'soroll (-1): {counts.
↪get(-1,0)}')

    # Clústers (0..C-1) amb cmap
```

```
    clusters = sorted([c for c in np.unique(labels) if c != -1])
    if clusters:
        ax.scatter(df_out.loc[~noise_mask, 'x1'], df_out.loc[~noise_mask, 'x2'],
                   c=df_out.loc[~noise_mask, 'cluster_dbscan'], s=15, alpha=.7,␣
↪cmap='jet')
    ax.set_title(f'DBSCAN - eps={eps:.2f}, min_samples={min_samples}')
    ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.axis('equal'); ax.grid(False)
    ax.legend(loc='upper right'); plt.tight_layout(); plt.show()

    # Informe curt
    print("Recompte d'etiquetes (inclou -1 com a soroll):", counts)

    # Silueta sobre NO-soroll (si aplicable)
    if len(clusters) >= 2 and (~noise_mask).sum() > 1:
        sil = silhouette_score(X[~noise_mask], labels[~noise_mask],␣
↪metric='euclidean')
        print(f"Silueta (sense soroll): {sil:.3f}")
    else:
        print("Silueta no definida (menys de 2 clústers sense soroll).")

    return df_out, counts
```

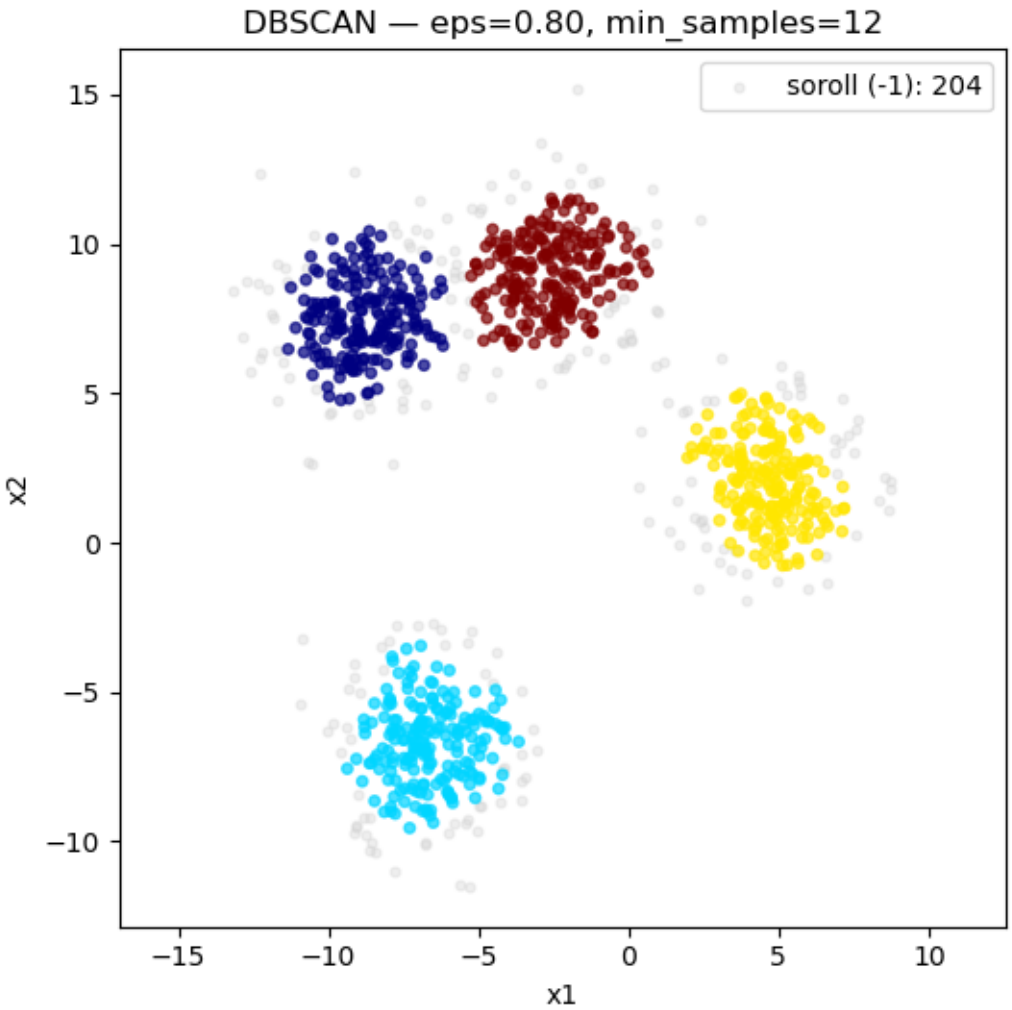
Ajustem els valors de eps_cadidates i de ms_candidateper obtenir 4 clusters

[20]:
```
# (A) Mira el k-dist plot per fer-te una idea d'eps (pren min_samples de␣
↪partida, p.ex. 5)
# plot_k_distance(df_blobs, min_samples=5)
plot_k_distance(df_blobs, min_samples=5)

# (B) Explora una graella petita (adapta rangs segons el que vegis al k-dist)
eps_candidates = np.linspace(0.8, 0.95, 5)    # exemple; ajusta segons el␣
↪genoll observat
ms_candidates  = [8, 10, 12]

res_grid = sweep_dbscan(df_blobs, eps_candidates, ms_candidates)
print(res_grid)

# (C) Tria una combinació raonable (pocs outliers, >=2 clústers i silueta␣
↪acceptable)
#     Exemple orientatiu: escull la millor silueta entre les files amb␣
↪n_clusters>=2
res_valid = res_grid[res_grid['n_clusters'] >= 2].copy()
best_row = res_valid.sort_values('silhouette', ascending=False).head(1)
eps_star = float(best_row['eps'].iloc[0])
ms_star  = int(best_row['min_samples'].iloc[0])
print(f"Paràmetres triats: eps={eps_star}, min_samples={ms_star}")
```

```python
# (D) Ajust final i visualització
df_blobs_db, counts = dbscan_fit_predict_plot(df_blobs, eps=eps_star,␣
↪min_samples=ms_star)
```




```
       eps  min_samples  n_clusters  n_noise  silhouette
0   0.8000            8           3      106    0.713969
1   0.8375            8           3       93    0.711435
2   0.8750            8           3       85    0.709413
3   0.9125            8           3       71    0.705017
4   0.9500            8           3       66    0.703959
5   0.8000           10           3      146    0.718382
6   0.8375           10           3      119    0.713759
7   0.8750           10           3      104    0.712012
8   0.9125           10           3       90    0.707909
9   0.9500           10           3       79    0.706989
10  0.8000           12           4      204    0.729630
11  0.8375           12           4      161    0.713345
12  0.8750           12           4      134    0.593480
13  0.9125           12           3      111    0.712423
14  0.9500           12           3       99    0.710523
Paràmetres triats: eps=0.8, min_samples=12
```




```
Recompte d'etiquetes (inclou -1 com a soroll): {np.int64(0): 200, np.int64(1):
194, np.int64(2): 199, np.int64(-1): 204, np.int64(3): 203}
Silueta (sense soroll): 0.730
```

Anàlisi: Què ha passat? Explica els motius pels quals creus que s'ha produït aquest resultat.

El k-dist mostra un genoll clar cap a eps 0.7–0.85 (k=5).

A la taula, hem triat eps=0.85, min_samples=12 perquè dona silueta alta (0.73). Amb aquests valors:

Tenim 4 clústers

204 outliers: amb min_samples=12 es demana la densitat; els punts perifèrics no arriben a 12 veïns en el radi i queden com a soroll (-1).

Que passa: tenim més soroll (-1), però recuperem els 4 clústers.

Implementació: Prova la implementació de DBSCAN jugant amb els paràmetres eps i min_samples

per trobar els grups (i outliers) del dataset Moons.

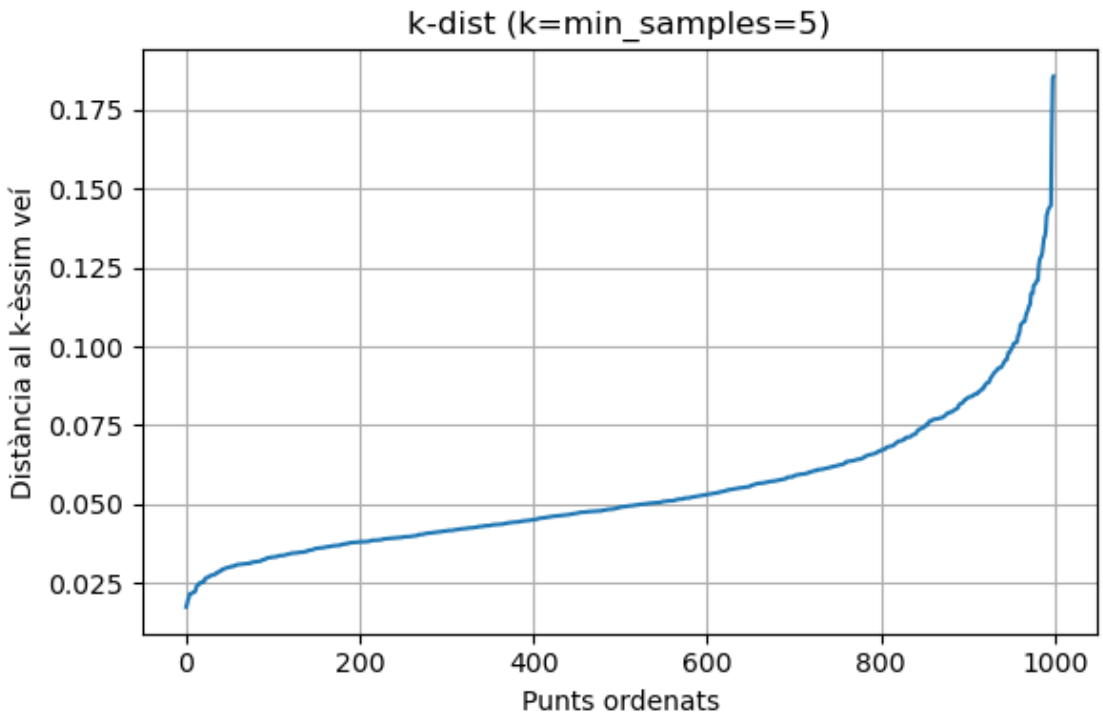
Reutilitzarem les funcions genèriques plot_k_distance, sweep_dbscan i dbscan_fit_predict_plot que ja tenim.

[21]:
```
# ---------------------------------------------------------------------
# DATAFRAME DE TREBALL PER A "MOONS"
# ---------------------------------------------------------------------
df_moons = (
    pd.DataFrame(X_moons, columns=['x1', 'x2'])
      .assign(etiqueta_real=pd.Series(y_moons, index=None))
)
print(df_moons.head(3))
```

```
         x1        x2  etiqueta_real
0 -0.033411  0.421391              1
1  0.998827 -0.442890              1
2  0.889592 -0.327843              1
```

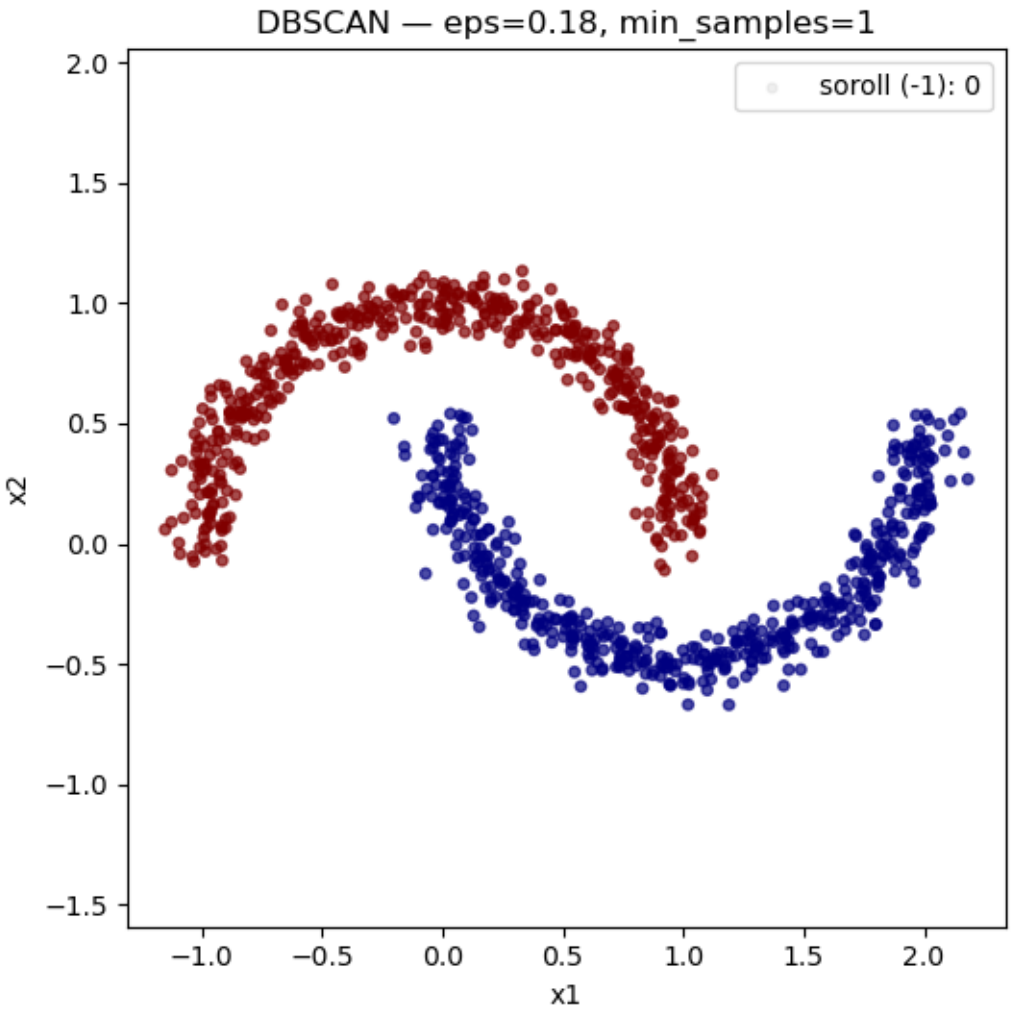
provem a utilitzar les mateixes funcions i estratégies que la part incial de bloobs

[22]:
```
# (A) Mira el k-dist plot per fer-te una idea d'eps (pren min_samples de␣
↪partida, p.ex. 5)
# plot_k_distance(df_moons, min_samples=5)
plot_k_distance(df_moons, min_samples=5)

# (B) Explora una graella petita (adapta rangs segons el que vegis al k-dist)
eps_candidates = np.linspace(0.1, 0.2, 11)   # ajustem segons el genoll observat
ms_candidates  = [1]

res_grid = sweep_dbscan(df_moons, eps_candidates, ms_candidates)
print(res_grid)

# (C) Tria una combinació raonable (pocs outliers, >=2 clústers i silueta␣
↪acceptable)
#     Exemple orientatiu: escull la millor silueta entre les files amb␣
↪n_clusters>=2
res_valid = res_grid[res_grid['n_clusters'] >= 2].copy()
best_row = res_valid.sort_values('silhouette', ascending=False).head(1)
eps_star = float(best_row['eps'].iloc[0])
ms_star  = int(best_row['min_samples'].iloc[0])
print(f"Paràmetres triats: eps={eps_star}, min_samples={ms_star}")

# (D) Ajust final i visualització
df_moons_db, counts = dbscan_fit_predict_plot(df_moons, eps=eps_star,␣
↪min_samples=ms_star)
```



	eps	min_samples	n_clusters	n_noise	silhouette
0	0.10	1	6	0	-0.137002
1	0.11	1	5	0	-0.136419
2	0.12	1	5	0	-0.136419
3	0.13	1	3	0	0.067755
4	0.14	1	3	0	0.067755
5	0.15	1	3	0	0.067755
6	0.16	1	2	0	0.331596
7	0.17	1	2	0	0.331596
8	0.18	1	2	0	0.331596
9	0.19	1	2	0	0.331596
10	0.20	1	2	0	0.331596

Paràmetres triats: eps=0.18, min_samples=1




```
Recompte d'etiquetes (inclou -1 com a soroll): {np.int64(0): 500, np.int64(1):
500}
Silueta (sense soroll): 0.332
```

Anàlisi: Què ha passat? Explica els motius pels quals creus que s'ha produït aquest resultat.

Amb DBSCAN i un eps al voltant de 0.1–0.2 amb min_samples 11, s'obtenen 2 clústers (les dues llunes) i cap outlier.

Si eps és massa petit → molts outliers (-1) i clústers trencats.

Si eps és massa gran → les dues llunes es fusionen en un sol clúster.

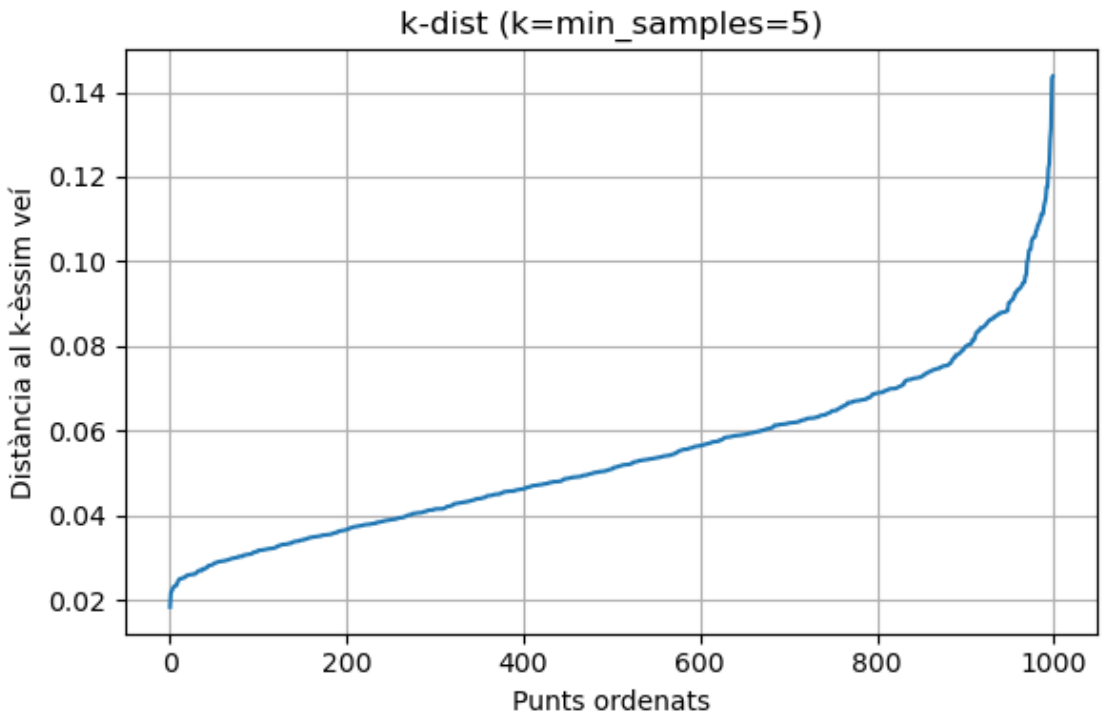
Implementació: Prova la implementació de DBSCAN jugant amb els paràmetres eps i min_samples per trobar els grups (i outliers) del dataset Circles.

provem a utilitzar les mateixes funcions i estratégies que la part incial de bloobs

[23]:
```
# ----------------------------------------------------------------------
# DATAFRAME DE TREBALL PER A "CIRCLES"
#     - Estructura: df_circles = ['x1','x2','etiqueta_real']
#     - Nota: 'etiqueta_real' s'usa només per visualitzar/avaluar, NO per␣
↪entrenar.
# ----------------------------------------------------------------------
df_circles = (
    pd.DataFrame(X_circles, columns=['x1', 'x2'])
      .assign(etiqueta_real=pd.Series(y_circles, index=None))
)
print(df_circles.head(3))
```

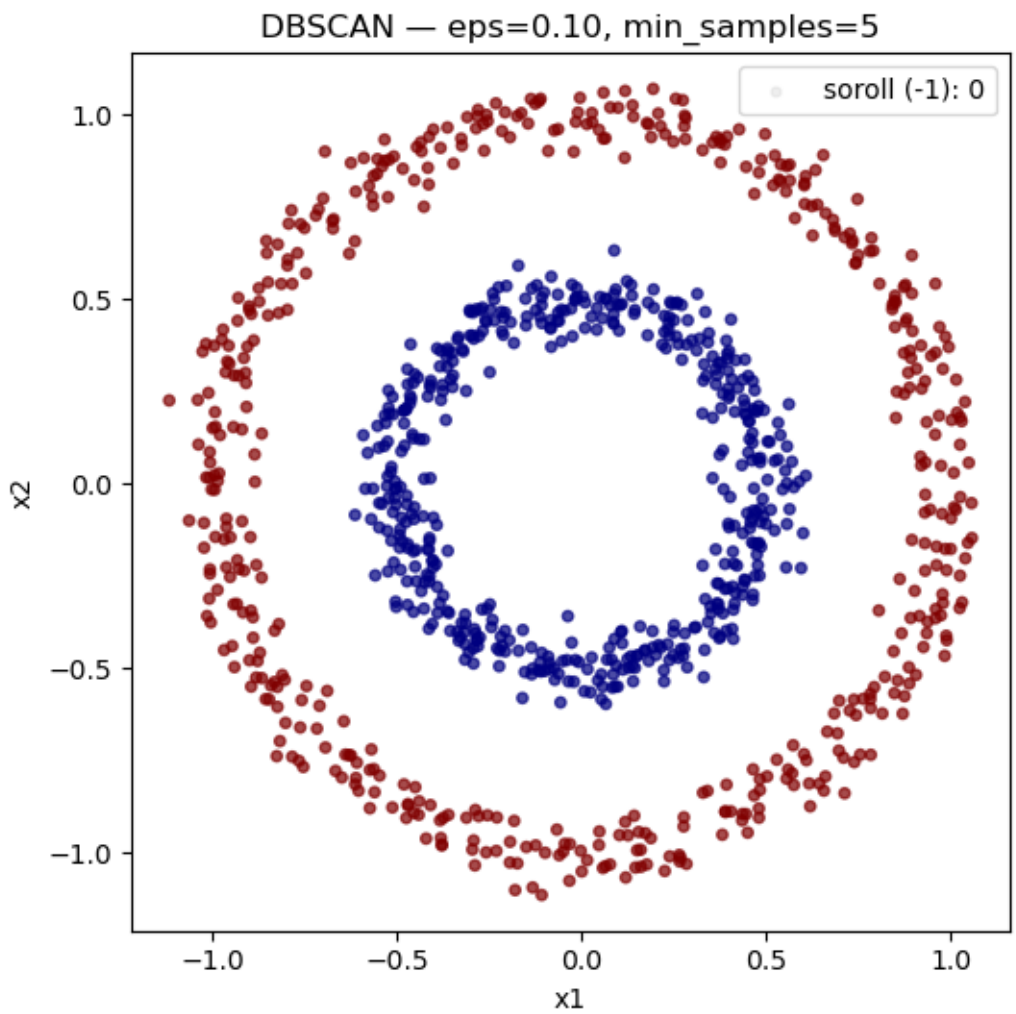
```
         x1        x2  etiqueta_real
0  0.452592  0.168433              1
1 -0.438027  0.119900              1
2 -0.532224  0.184359              1
```

[24]:
```
# (A) Mira el k-dist plot per fer-te una idea d'eps (pren min_samples de␣
↪partida, p.ex. 5)
# plot_k_distance(df_circles, min_samples=5)
plot_k_distance(df_circles, min_samples=5)

# (B) Explora una graella petita (adapta rangs segons el que vegis al k-dist)
eps_candidates = np.linspace(0.1, 0.2, 11)   # ajustem segons el genoll observat
ms_candidates  = [5]

res_grid = sweep_dbscan(df_circles, eps_candidates, ms_candidates)
print(res_grid)

# (C) Tria una combinació raonable (pocs outliers, >=2 clústers i silueta␣
↪acceptable)
#     Exemple orientatiu: escull la millor silueta entre les files amb␣
↪n_clusters>=2
res_valid = res_grid[res_grid['n_clusters'] >= 2].copy()
best_row = res_valid.sort_values('silhouette', ascending=False).head(1)
eps_star = float(best_row['eps'].iloc[0])
ms_star  = int(best_row['min_samples'].iloc[0])
print(f"Paràmetres triats: eps={eps_star}, min_samples={ms_star}")

# (D) Ajust final i visualització
df_circles_db, counts = dbscan_fit_predict_plot(df_circles, eps=eps_star,␣
↪min_samples=ms_star)
```



	eps	min_samples	n_clusters	n_noise	silhouette
0	0.10	5	2	0	0.112462
1	0.11	5	2	0	0.112462
2	0.12	5	2	0	0.112462
3	0.13	5	2	0	0.112462
4	0.14	5	2	0	0.112462
5	0.15	5	2	0	0.112462
6	0.16	5	2	0	0.112462
7	0.17	5	2	0	0.112462
8	0.18	5	2	0	0.112462
9	0.19	5	2	0	0.112462
10	0.20	5	2	0	0.112462

Paràmetres triats: eps=0.1, min_samples=5




```
Recompte d'etiquetes (inclou -1 com a soroll): {np.int64(0): 500, np.int64(1):
500}
Silueta (sense soroll): 0.112
```

Anàlisi: Què ha passat? Explica els motius pels quals creus que s'ha produït aquest resultat.

Amb DBSCAN i un eps al voltant de 0.1–0.2 amb min_samples 5, s'obtenen 2 clústers els dos cercles i cap outlier.

Si eps és massa petit → molts outliers (-1) i clústers trencats.

Si eps és massa gran → els dos cercles es fusionen en un sol clúster.

1.1.4 1 c. Algorismes jeràrquics

En aquest apartat es demana visualitzar mitjançant un dendrograma la construcció progressiva dels grups mitjançant un algorisme jeràrquic aglomeratiu (estratègia *bottom-up*). Amb això es pretén

trobar un mètode gràfic per entendre el comportament de l'algorisme i trobar els *clústers* desitjats a cada dataset.

[25]:
```
X, y = X_blobs, y_blobs
```

Implementació: Prova la implementació de clustering jeràrquic de scipy provant diferents criteris d'enllaç o linkage permetent identificar els clústers subjacents (mostrant-ne el resultat) i el seu dendrograma per al dataset Blobs. Pots importar les llibreries necessàries per a això.

Creem les funcions que utilitzarem per cadascún dels datasets

[26]:
```
# ---------------------------------------------------------------------
# JERÀRQUIC AGLOMERATIU (SCIPY): DENDROGRAMA + CLÚSTERS
#   * Només criteris d'enllaç: single, complete, average, centroid
# ---------------------------------------------------------------------
# 1) LINKAGE
# ---------------------------------------------------------------------
def fit_linkage(df, method='average', metric='euclidean'):
    """
    Calcula l'enllaç jeràrquic (linkage) per a un DataFrame amb ['x1','x2'].

    Què fa
    ------
    - X = df[['x1','x2']].to_numpy() -> (n,2)
    - linkage(X, method=..., metric=...)

    Notes
    -----
    - Els mètodes permesos aquí: 'single', 'complete', 'average', 'centroid'.
    - 'centroid' requereix mètrica euclidiana (forcem 'euclidean').
    """
    allowed = {'single', 'complete', 'average', 'centroid'}
    if method not in allowed:
        raise ValueError(f"method='{method}' no permès; usa {allowed}")

    X = df[['x1', 'x2']].to_numpy() # transformem a matriu numpy
    metric_eff = 'euclidean' if method == 'centroid' else metric # Forcem␣
↪metriques de distància
    # Cridem scipy linkage per anar fusionant mostres en clusters fins quedar␣
↪un sol grup.
    Z = linkage(X, method=method, metric=metric_eff)
    return Z

# ---------------------------------------------------------------------
# 2) DENDROGRAMA
# ---------------------------------------------------------------------
def plot_dendro(Z, k=None, max_d=None, title='Dendrograma'):
    """
```

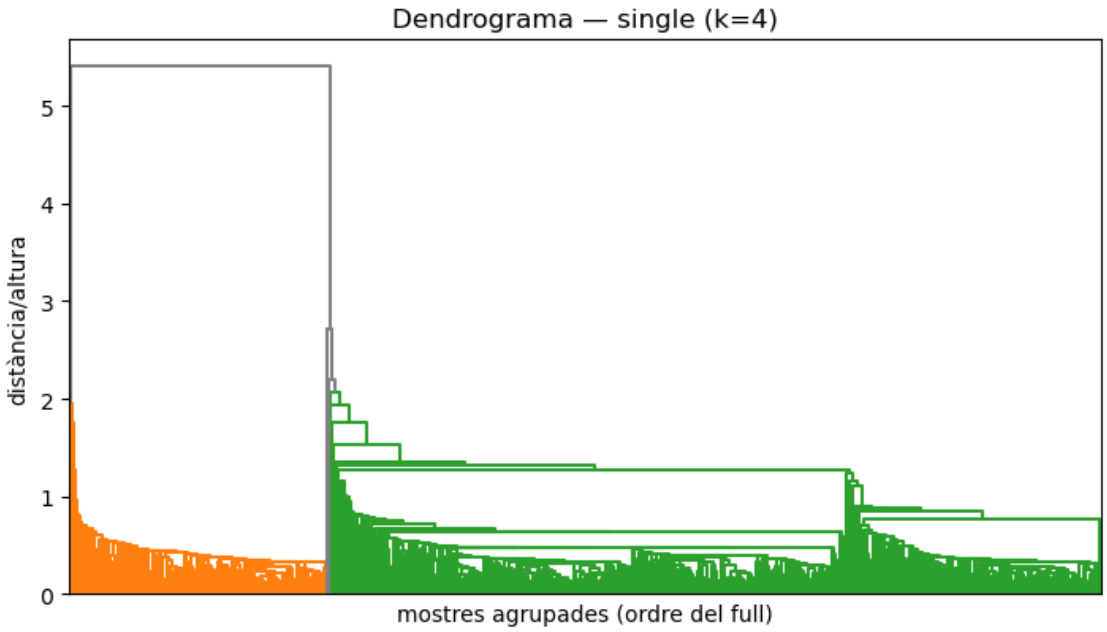
```
    Dibuixa el dendrograma. Si s'indica k, colora el tall per obtenir k␣
↪clústers.

    Tall
    ----
    - Si k: color_threshold = Z[-(k-1), 2]
    - Si max_d: color_threshold = max_d
    """
    color_threshold = None
    if k is not None:
        color_threshold = Z[-(k-1), 2]
    elif max_d is not None:
        color_threshold = max_d

    plt.figure(figsize=(7.2, 4.2))
    dendrogram(
        Z,
        color_threshold=color_threshold,
        above_threshold_color='gray',
        no_labels=True,
        distance_sort='ascending'
    )
    plt.title(title)
    plt.xlabel('mostres agrupades (ordre del full)')
    plt.ylabel('distància/altura')
    plt.tight_layout()
    plt.show()

# ---------------------------------------------------------------------
# 3) TALLAR L'ARBRE I PLOTEJAR CLÚSTERS
# ---------------------------------------------------------------------
def cut_and_plot_clusters(df, Z, k=2, title_prefix='Dataset', cmap='jet'):
    """
    Retalla l'arbre a k clústers, fa un scatter i calcula mètriques.

    Estructures
    -----------
    - labels : np.ndarray (n,)
    - labels_s : Series alineada amb df
    - df_out = df.assign(cluster_hclust=labels_s)
    """
    labels = fcluster(Z, t=int(k), criterion='maxclust')
    labels_s = pd.Series(labels, index=df.index, name='cluster_hclust')
    df_out = df.assign(cluster_hclust=labels_s)

    # Mètriques ràpides
    X = df[['x1','x2']].to_numpy()
```

```
    sil = silhouette_score(X, labels, metric='euclidean') if len(np.
 ↪unique(labels)) > 1 else np.nan
    counts = dict(Counter(labels))
    print(f"Recompte per clúster (k={k}, {title_prefix}): {counts}")
    print(f"Silueta (euclidiana): {sil:.3f}")

    # Scatter
    fig, ax = plt.subplots(figsize=(5.6, 5.6))
    ax.scatter(df_out['x1'], df_out['x2'],
               c=df_out['cluster_hclust'], s=15, alpha=.6, cmap=cmap)
    ax.set_title(f'{title_prefix} - jeràrquic (k={k})')
    ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.axis('equal'); plt.
 ↪tight_layout()
    plt.show()

    return df_out, sil, counts

# ---------------------------------------------------------------------
# 4) Funció de fluxe a on apliquem les funciopns abans creades (GENÈRIC PER A␣
 ↪QUALSEVOL DATASET)
# ---------------------------------------------------------------------
def run_hierarchical(df, k=2,␣
 ↪methods=('single','complete','average','centroid'),
                     metric='euclidean', name='Dataset'):
    """
    Prova els diferents 'linkage' especificats, mostra dendrograma i resultat.

    Flux
    ----
    per a cada m a methods:
        Z = fit_linkage(df, method=m, metric=metric)
        plot_dendro(Z, k=k, title=f"Dendrograma - {m} (k={k})")
        df_pred, sil, counts = cut_and_plot_clusters(df, Z, k=k,␣
 ↪title_prefix=f'{name} ({m})')
    """
    results = {}
    for m in methods:
        print("\n" + "-"*70)
        print(f"[{m.upper()}] construint enllaç i dendrograma…")
        Z = fit_linkage(df, method=m, metric=metric)
        plot_dendro(Z, k=k, title=f"Dendrograma - {m} (k={k})")
        df_pred, sil, counts = cut_and_plot_clusters(df, Z, k=k,␣
 ↪title_prefix=f'{name} ({m})')
        results[m] = {'Z': Z, 'df_pred': df_pred, 'silhouette': sil, 'counts':␣
 ↪counts}
    return results
```

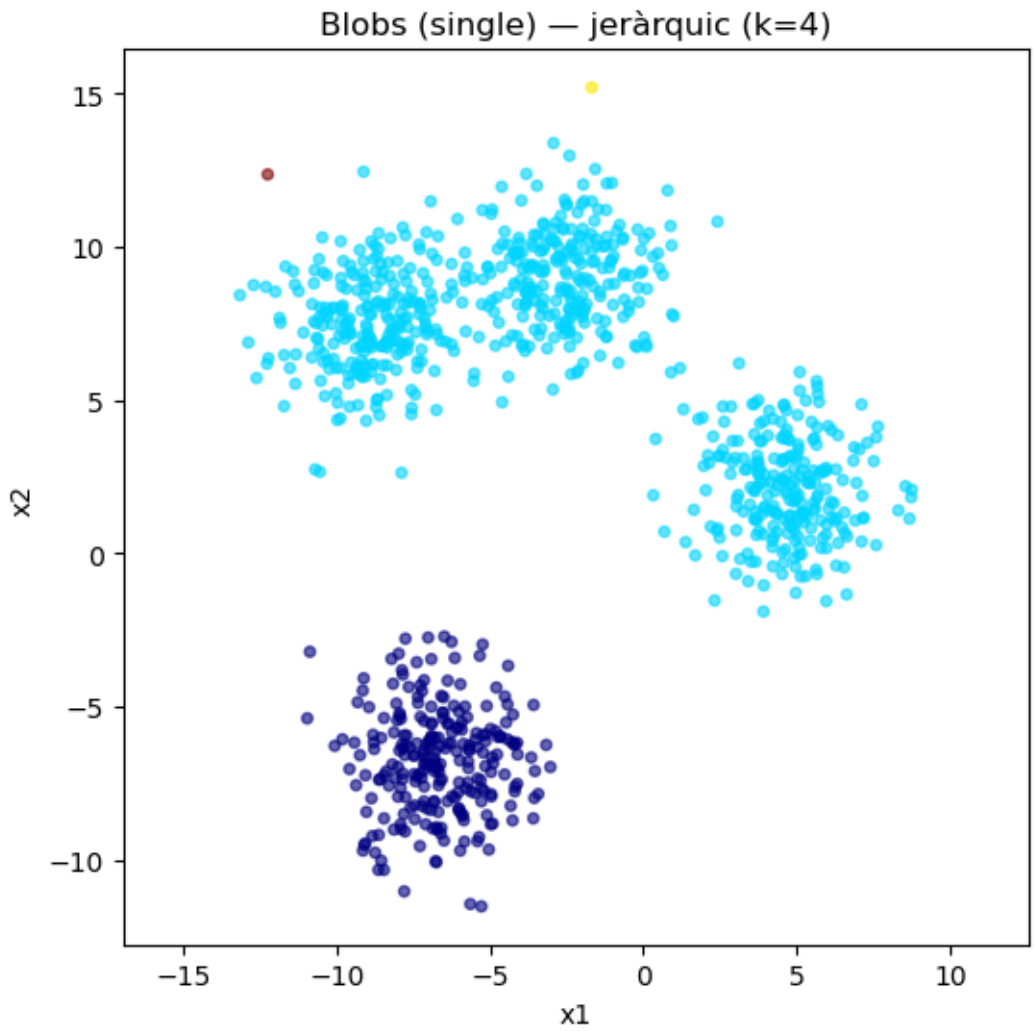
Implementació completa per al clustering jeràrquic aglomeratiu a Blobs, provant diversos criteris d'enllaç i mostrant dendrograma + scatter dels clústers.

[27]:
```
# ----------------------------------------------------------------------
# BLOBS (esperat k=4)
# ----------------------------------------------------------------------
X, y = X_blobs, y_blobs
res_blobs = run_hierarchical(
    df_blobs, k=4,
    methods=('single','complete','average','centroid'),
    metric='euclidean', name='Blobs'
)
```

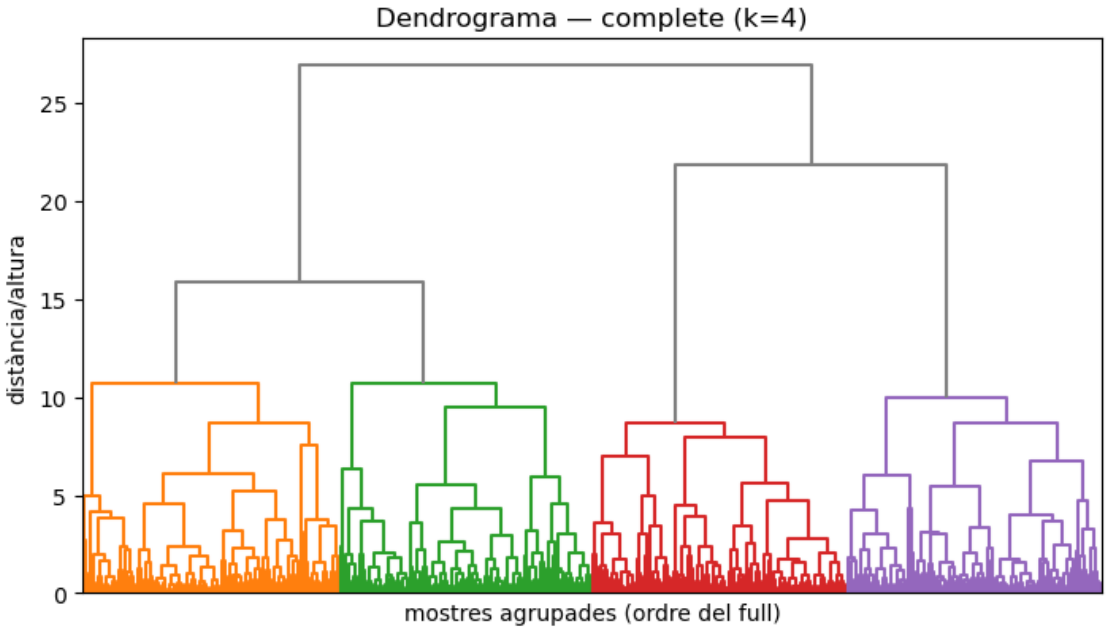
```
----------------------------------------------------------------------
[SINGLE] construint enllaç i dendrograma…
```



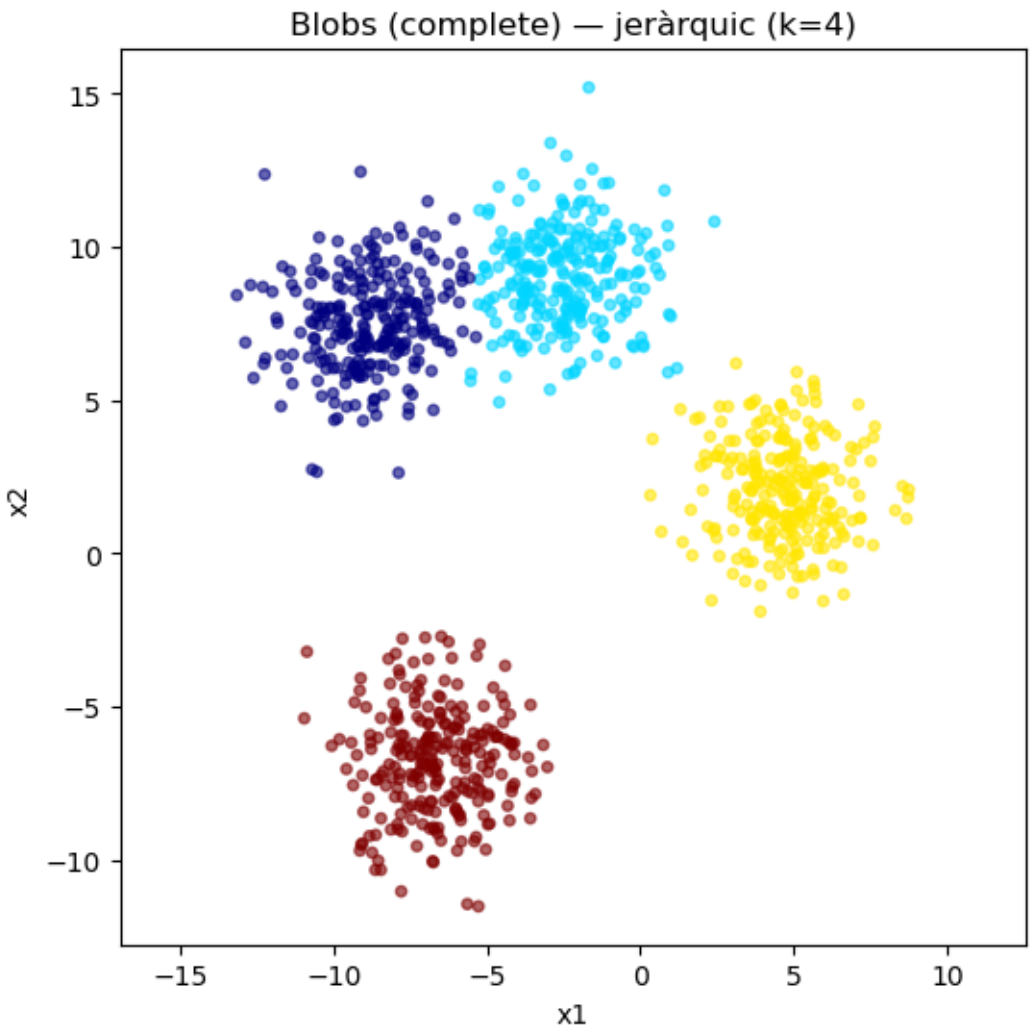

```
Recompte per clúster (k=4, Blobs (single)): {np.int32(2): 748, np.int32(1): 250,
np.int32(4): 1, np.int32(3): 1}
Silueta (euclidiana): 0.216
```



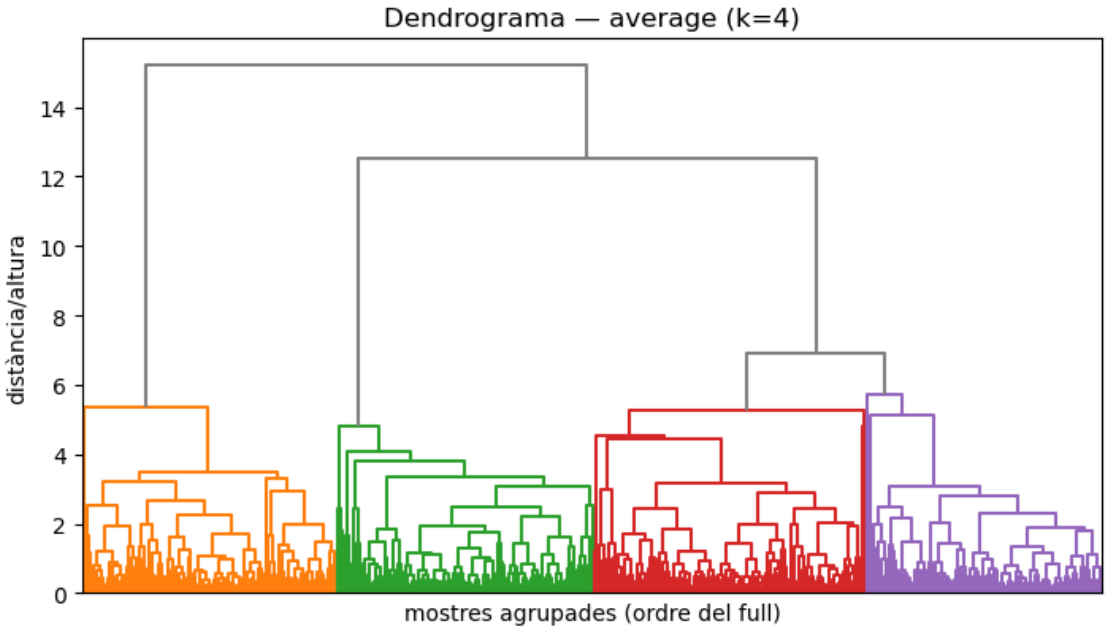

```
----------------------------------------------------------------------
[COMPLETE] construint enllaç i dendrograma…
```



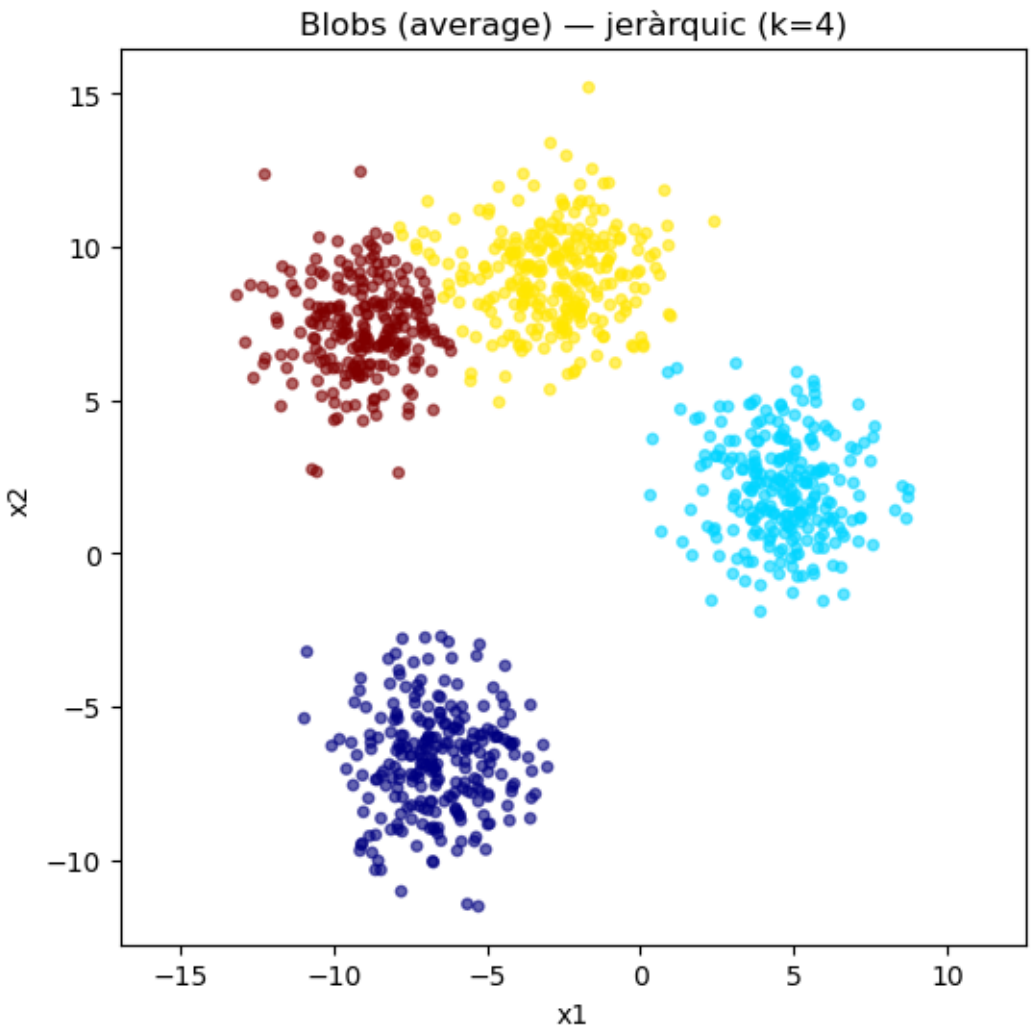

```
Recompte per clúster (k=4, Blobs (complete)): {np.int32(1): 253, np.int32(4):
250, np.int32(3): 250, np.int32(2): 247}
Silueta (euclidiana): 0.666
```



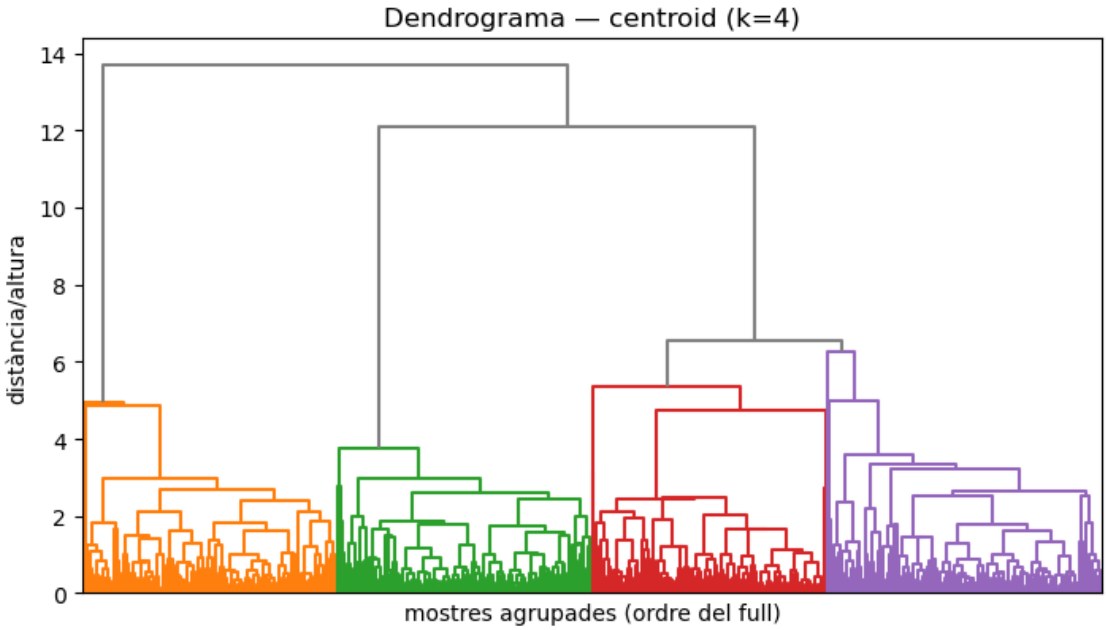

```
----------------------------------------------------------------------
[AVERAGE] construint enllaç i dendrograma…
```



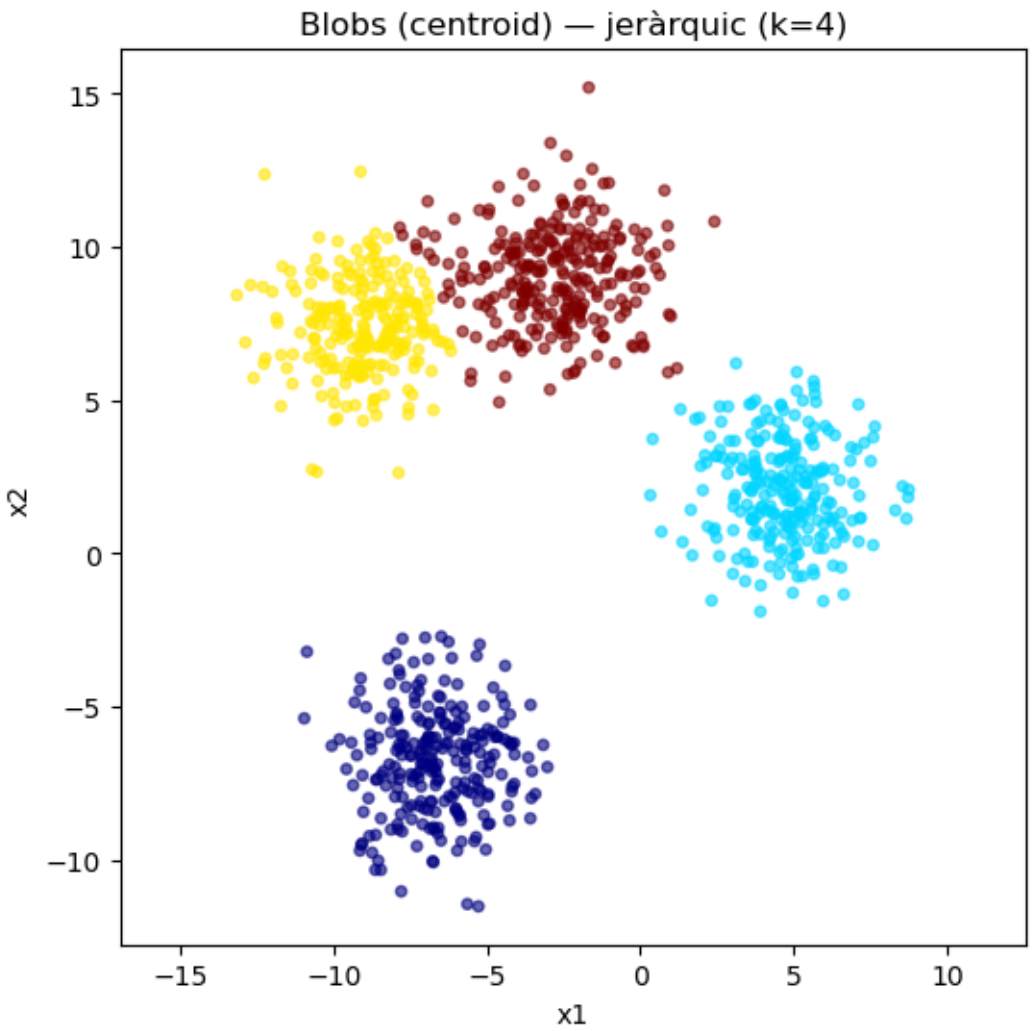

```
Recompte per clúster (k=4, Blobs (average)): {np.int32(4): 231, np.int32(1):
250, np.int32(2): 252, np.int32(3): 267}
Silueta (euclidiana): 0.661
```




```
----------------------------------------------------------------------
[CENTROID] construint enllaç i dendrograma…
```




```
Recompte per clúster (k=4, Blobs (centroid)): {np.int32(3): 231, np.int32(1):
250, np.int32(2): 250, np.int32(4): 269}
Silueta (euclidiana): 0.661
```



Anàlisi: Interpreta el dendrograma i comenta quin criteri d'enllaç s'ha comportat millor. Per què?

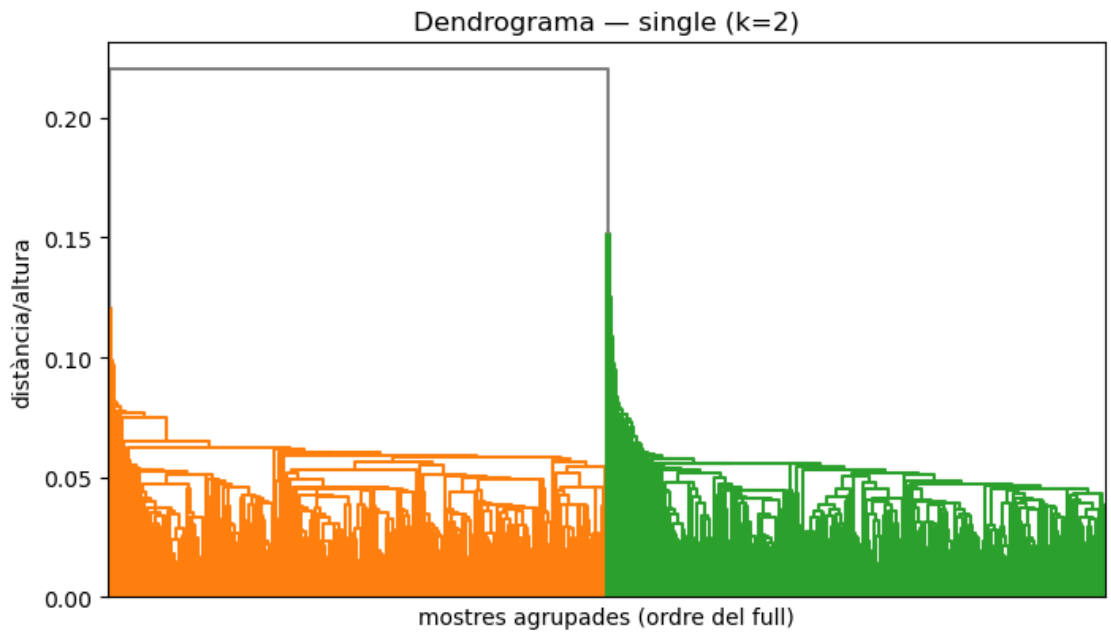
L'enllaç Complet és el millor (lleugerament per sobre d'average/centroid) perquè, en dades "esfèriques" com els blobs, usar la distància màxima entre punts dels dos grups penalitza els ponts esparsos i evita l'encadenament, mantenint els clústers compactes i ben separats. Average i centroid també funcionen bé aquí, però single falla per la seva tendència a unir grups a través de punts aïllats.

Implementació: Prova la implementació de clustering jeràrquic de scipy provant diferents criteris d'enllaç o linkage permetent identificar els clústers subjacents (mostrant-ne el resultat) i el seu dendrograma per al dataset Moons. Pots importar les llibreries necessàries per a això.

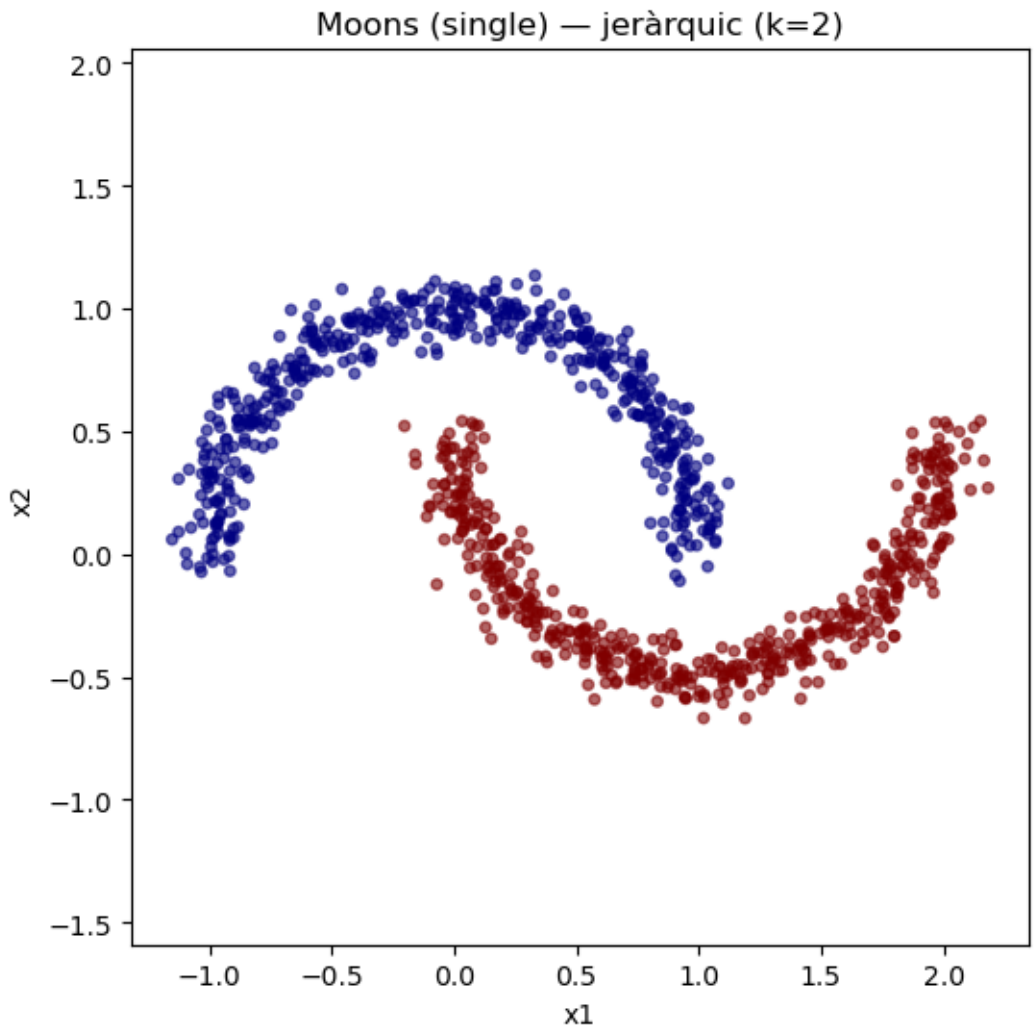
Implementació completa per al clustering jeràrquic aglomeratiu a Moons, provant diversos criteris d'enllaç i mostrant dendrograma + scatter dels clústers.

[28]:
```
# ----------------------------------------------------------------------
# MOONS (esperat k=2, forma no convexa)
# ----------------------------------------------------------------------
X, y = X_moons, y_moons
res_moons = run_hierarchical(
    df_moons, k=2,
    methods=('single','complete','average','centroid'),
    metric='euclidean', name='Moons'
)
```

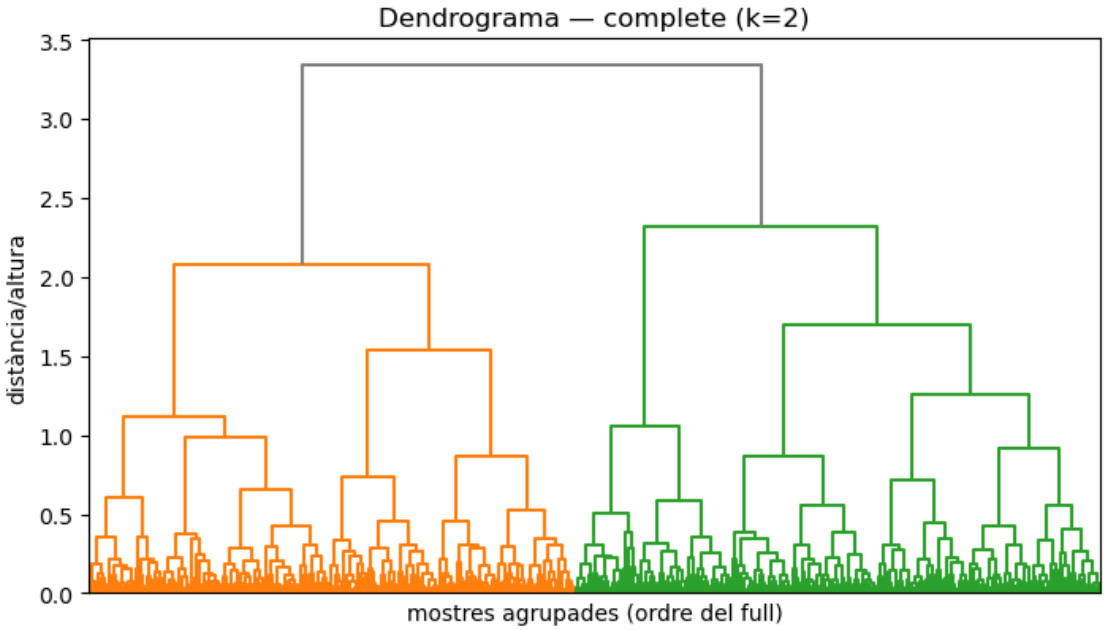
```
----------------------------------------------------------------------
[SINGLE] construint enllaç i dendrograma…
```



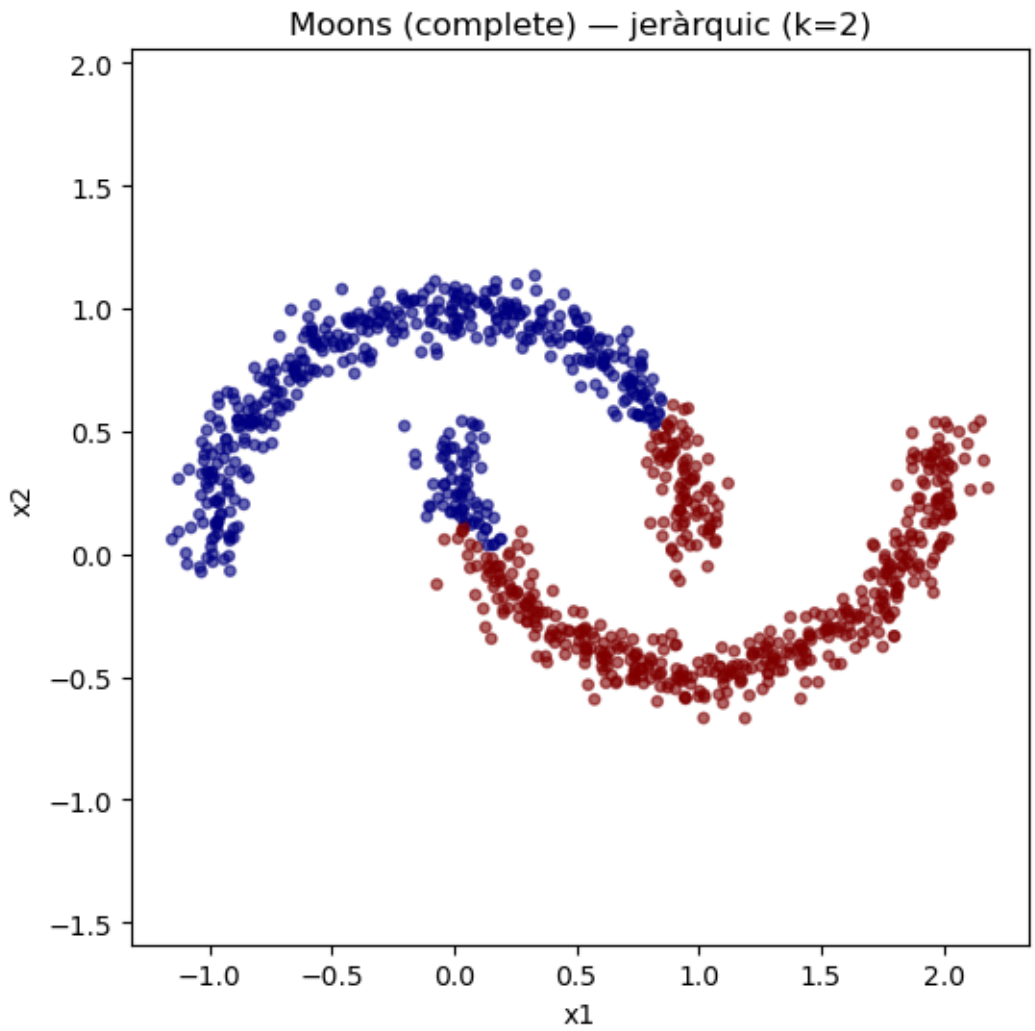

```
Recompte per clúster (k=2, Moons (single)): {np.int32(2): 500, np.int32(1): 500}
Silueta (euclidiana): 0.332
```



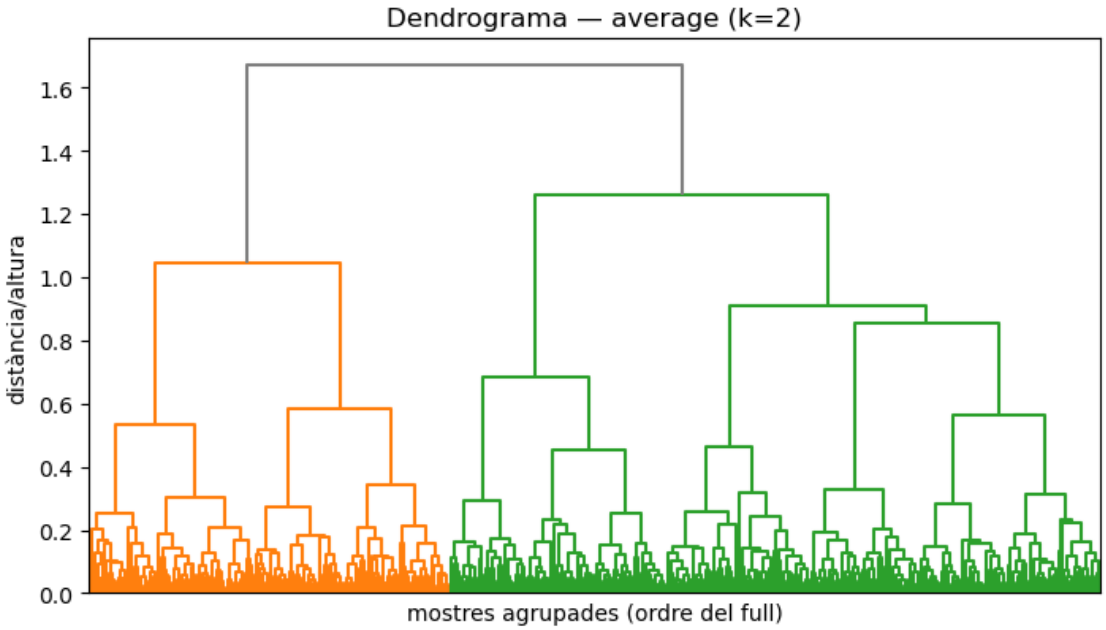

```
----------------------------------------------------------------------
[COMPLETE] construint enllaç i dendrograma…
```



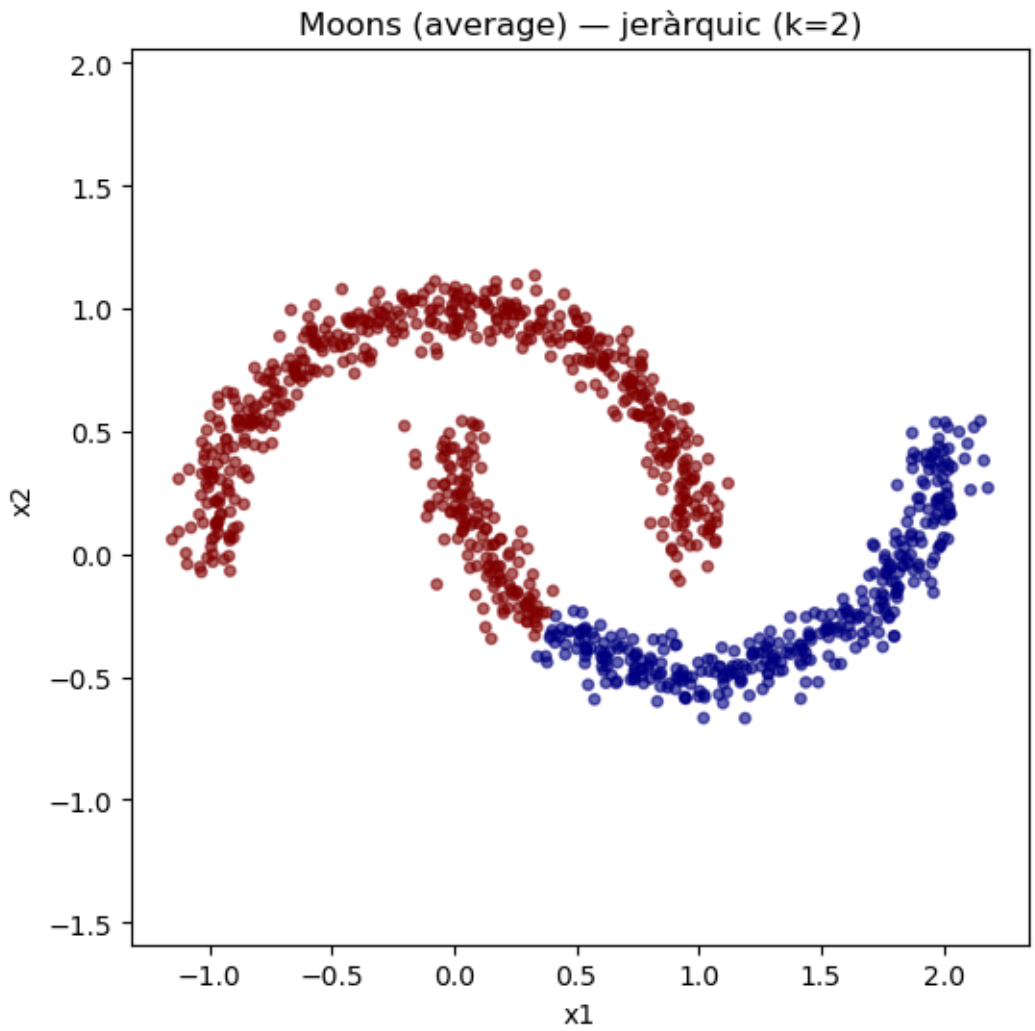

```
Recompte per clúster (k=2, Moons (complete)): {np.int32(1): 481, np.int32(2):
519}
Silueta (euclidiana): 0.472
```



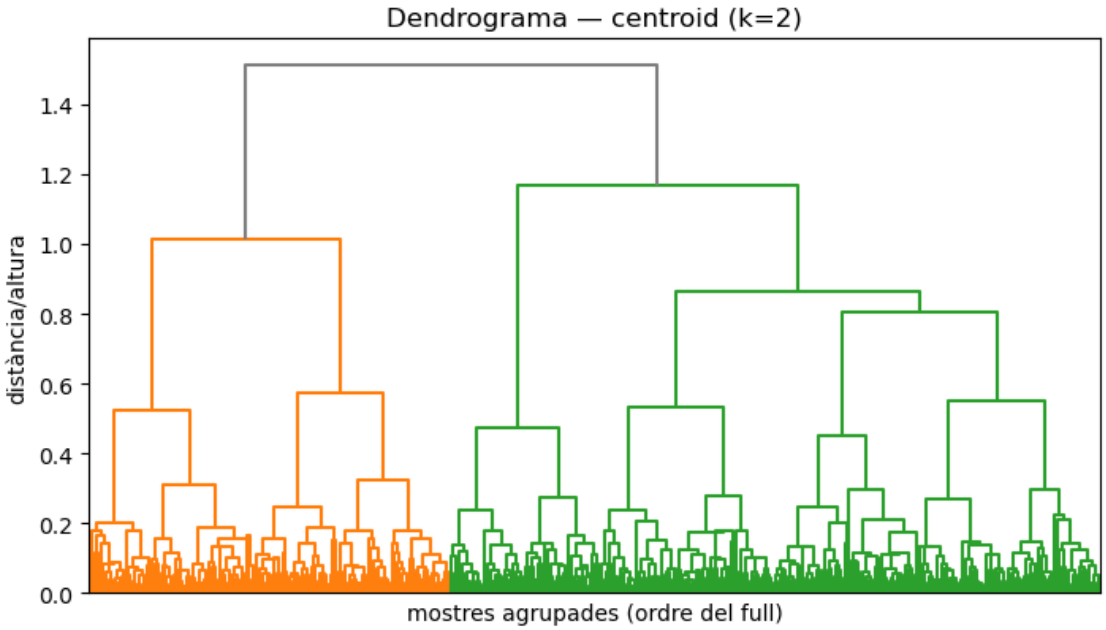

```
----------------------------------------------------------------------
[AVERAGE] construint enllaç i dendrograma…
```



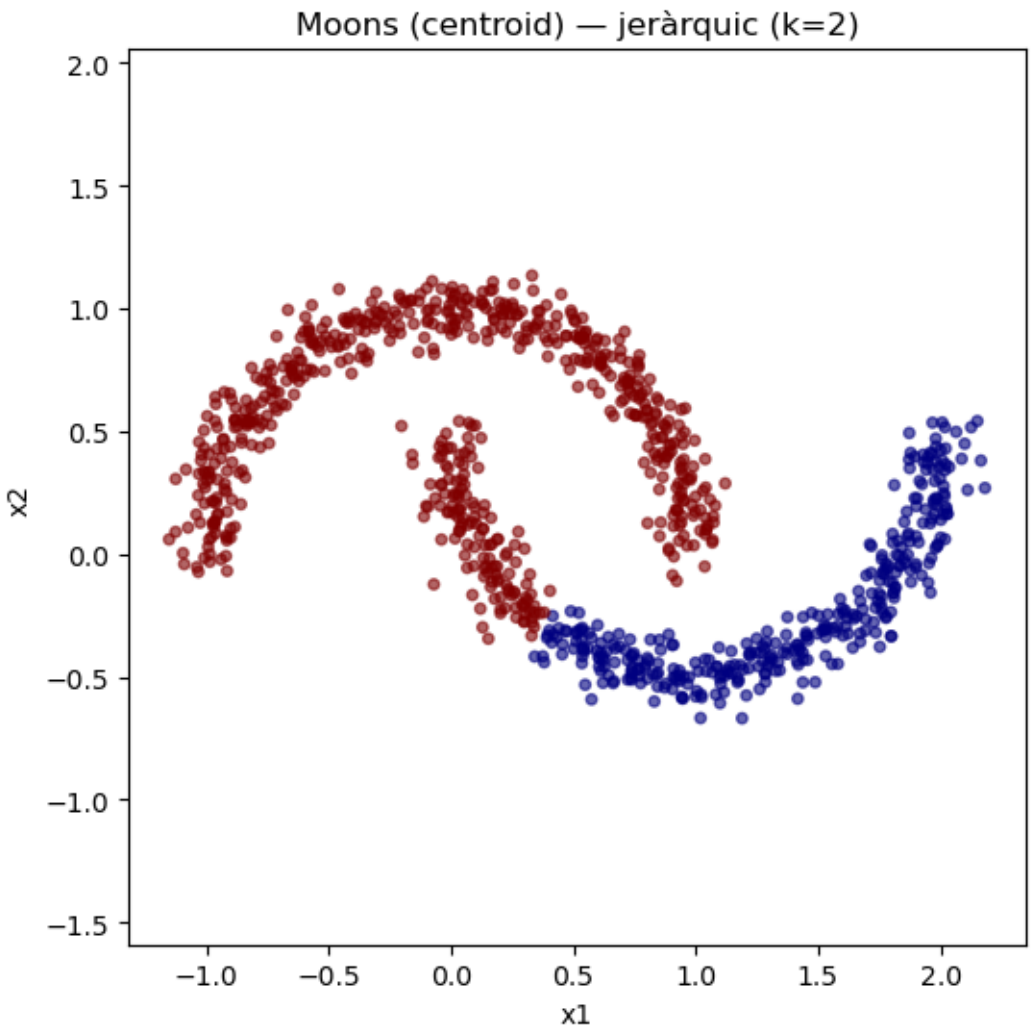

```
Recompte per clúster (k=2, Moons (average)): {np.int32(2): 643, np.int32(1):
357}
Silueta (euclidiana): 0.441
```




```
----------------------------------------------------------------------
[CENTROID] construint enllaç i dendrograma…
```




```
Recompte per clúster (k=2, Moons (centroid)): {np.int32(2): 643, np.int32(1):
357}
Silueta (euclidiana): 0.441
```



Anàlisi: Interpreta el dendrograma i comenta quin criteri d'enllaç s'ha comportat millor. Per què?

L'enllaç complet minimitza la distància màxima entre parelles de punts de dos clústers. Això afavoreix regions compactes i (quasi) convexes. En two moons, la frontera òptima no és recta; a la zona de contacte el criteri "força" una partició més recta i alguns punts canvien d'etiqueta.

Average i centroid pateixen el mateix biaix a convexitat (el centroid, a més, pot tenir inversions al dendrograma), i també "tallen" la lluna per una línia massa recta.

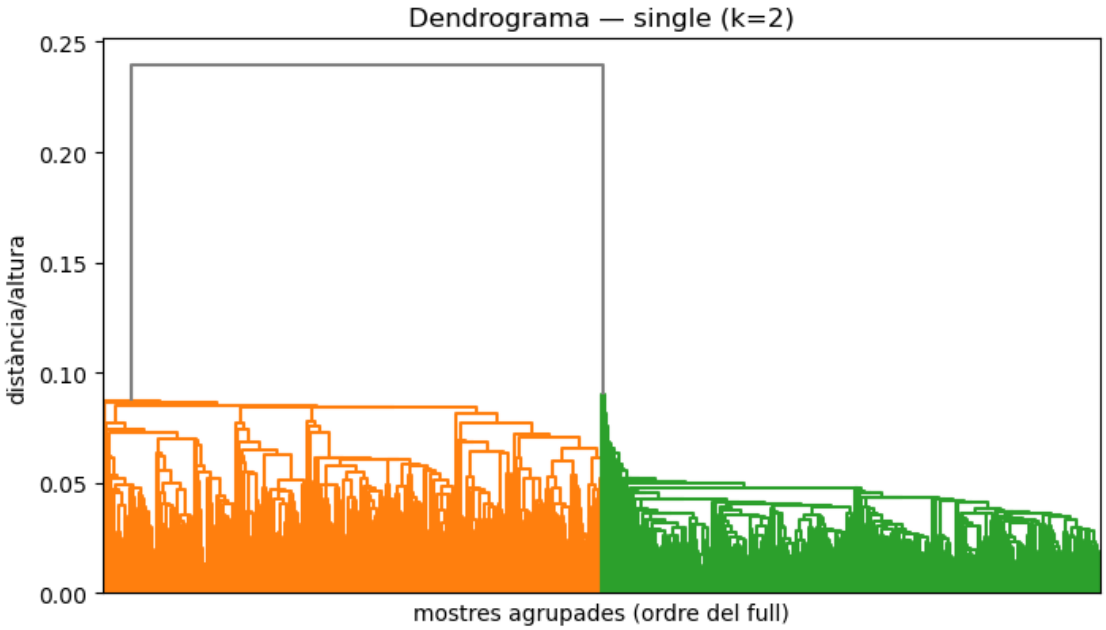
L'enllaç single, en canvi, uneix clústers si qualsevol parella de punts és propera; és equivalent a tallar l'MST pel seu aresta més llarga. Això preserva formes no convexes i, en dades netes com moons, és l'únic dels quatre que separa cada arc de manera natural. La contrapartida és el chaining i pitjor silueta, però visualment la separació és la desitjada.

Implementació: Prova la implementació de clustering jeràrquic de scipy provant diferents criteris d'enllaç o linkage permetent identificar els clústers subjacents (mostrant-ne el resultat) i el seu dendrograma per al dataset Circles. Pots importar les llibreries necessàries per a això.

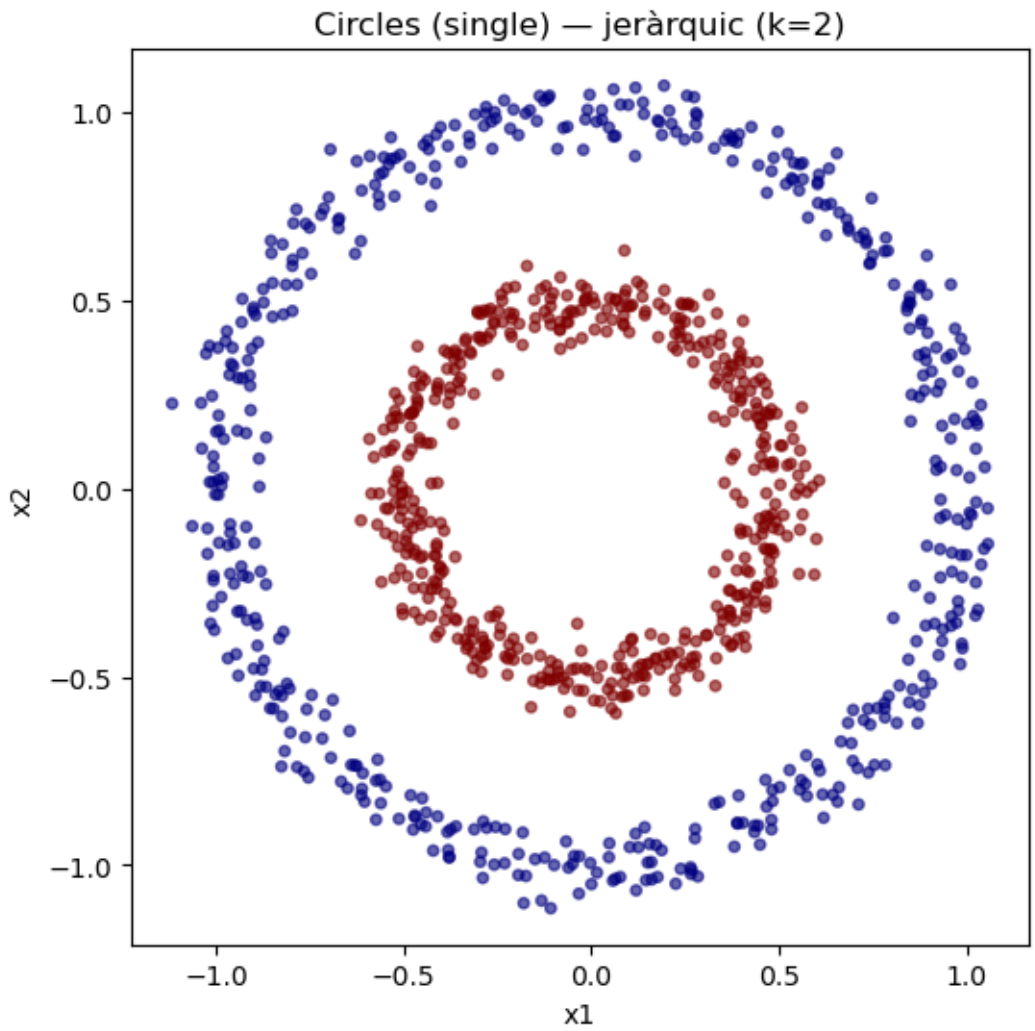
Implementació completa per al clustering jeràrquic aglomeratiu a Circles, provant diversos criteris d'enllaç i mostrant dendrograma + scatter dels clústers.

[29]:
```
# ----------------------------------------------------------------------
# CIRCLES (esperat k=2, anells concèntrics)
#   - single sol funcionar bé (manté formes fines/allargades)
#   - complete/average poden barrejar sectors oposats si la separació radial és␣
↪petita
#   - centroid sol trencar anells (assumeix clústers globulars)
# ----------------------------------------------------------------------
X, y = X_circles, y_circles
res_circles = run_hierarchical(
    df_circles, k=2,
    methods=('single','complete','average','centroid'),
    metric='euclidean', name='Circles'
)
```

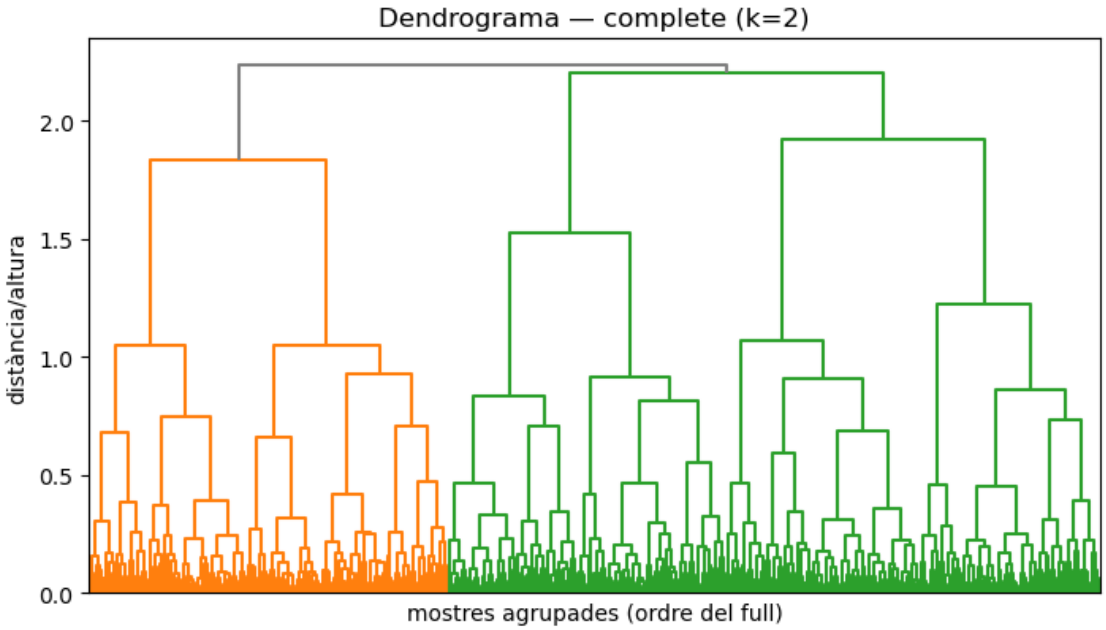
```
----------------------------------------------------------------------
[SINGLE] construint enllaç i dendrograma…
```



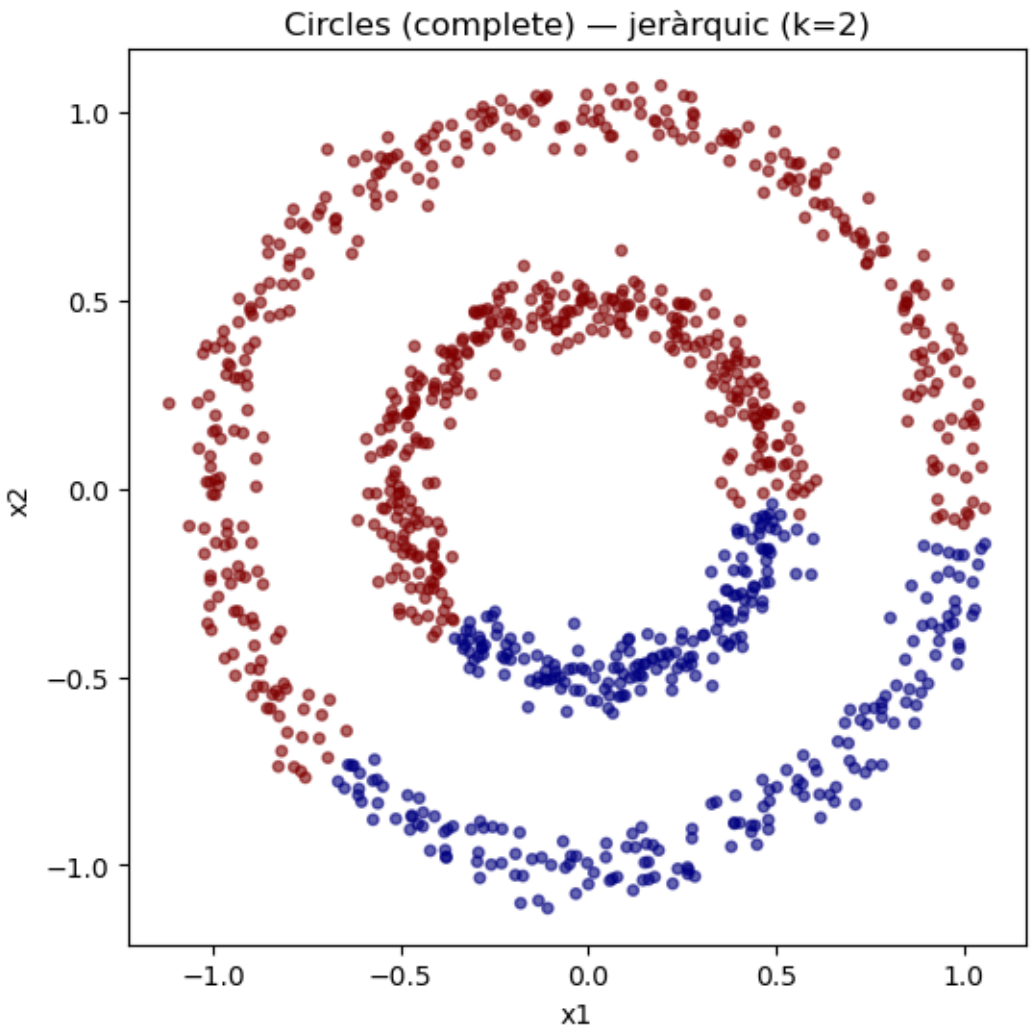

```
Recompte per clúster (k=2, Circles (single)): {np.int32(2): 500, np.int32(1):
500}
Silueta (euclidiana): 0.112
```



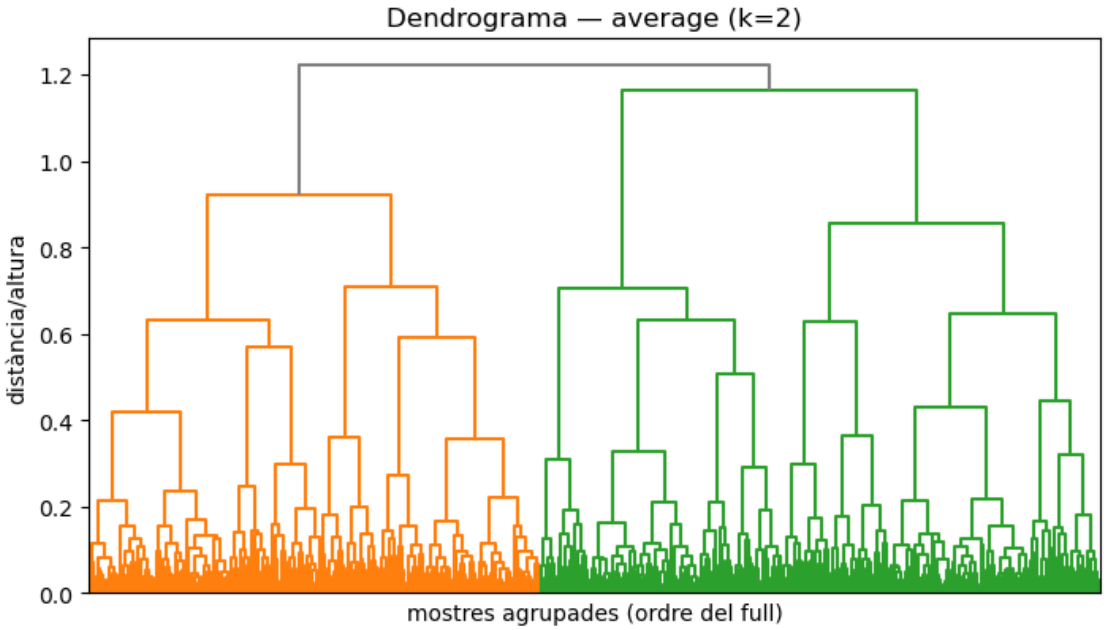

```
----------------------------------------------------------------------
[COMPLETE] construint enllaç i dendrograma…
```



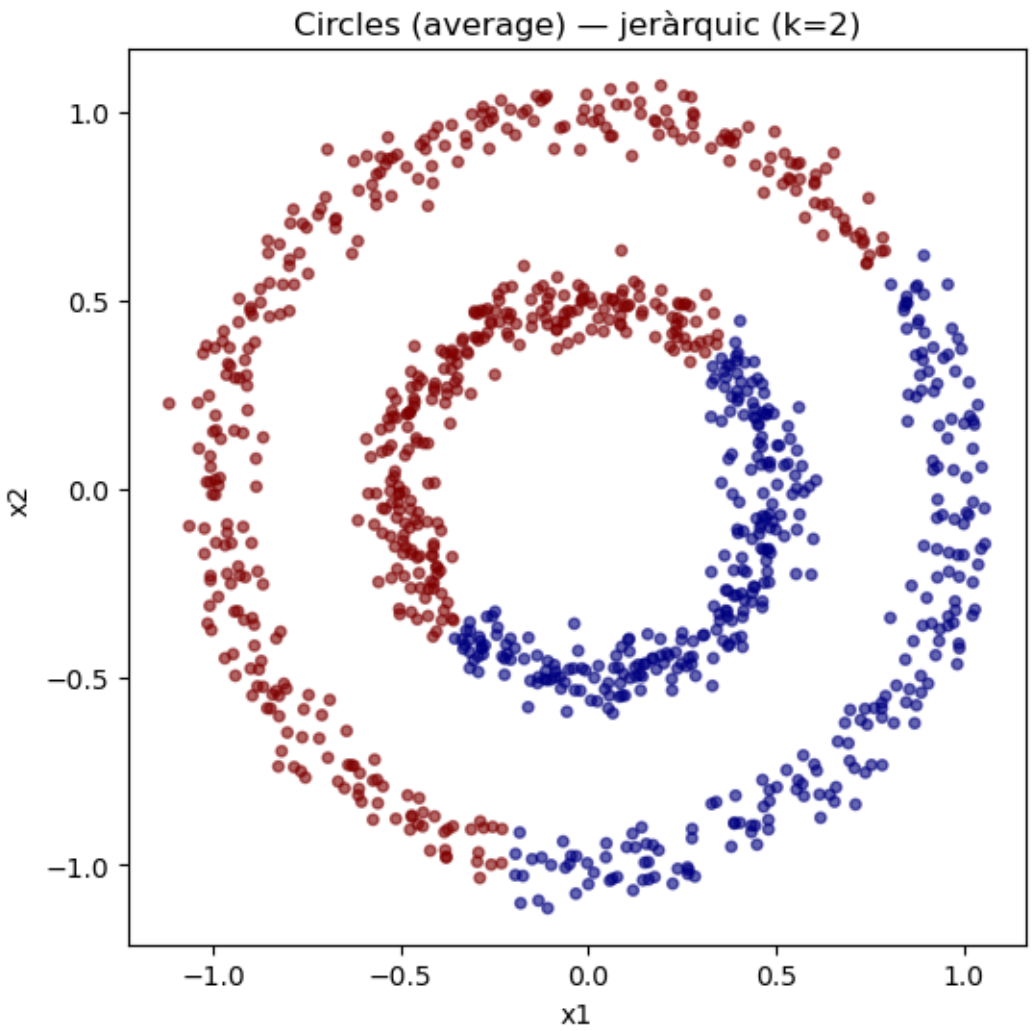

```
Recompte per clúster (k=2, Circles (complete)): {np.int32(2): 644, np.int32(1):
356}
Silueta (euclidiana): 0.318
```



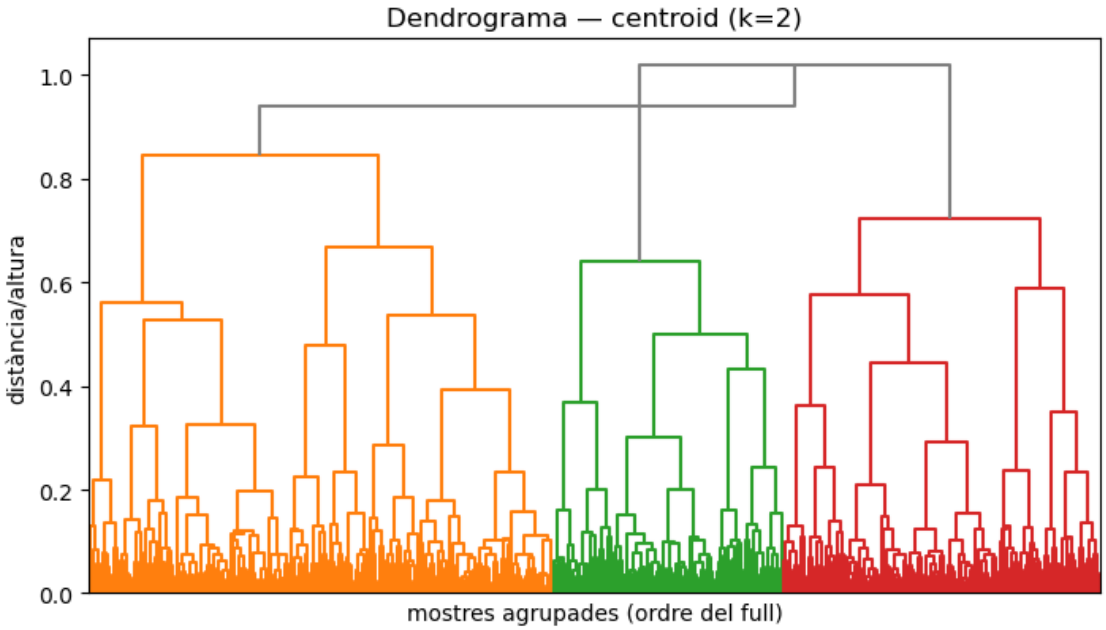

```
----------------------------------------------------------------------
[AVERAGE] construint enllaç i dendrograma…
```



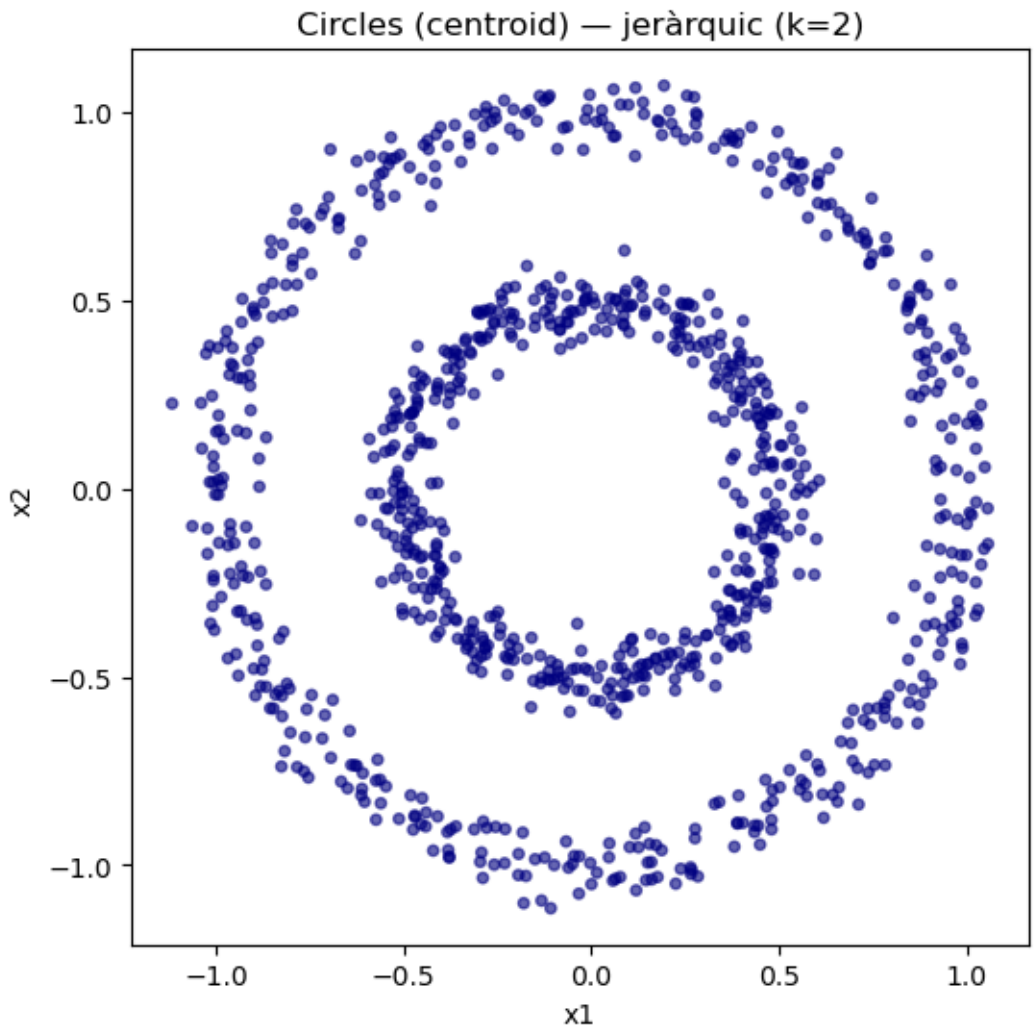

```
Recompte per clúster (k=2, Circles (average)): {np.int32(1): 446, np.int32(2):
554}
Silueta (euclidiana): 0.332
```




```
----------------------------------------------------------------------
[CENTROID] construint enllaç i dendrograma…
```




```
Recompte per clúster (k=2, Circles (centroid)): {np.int32(1): 1000}
Silueta (euclidiana): nan
```



Anàlisi: Interpreta el dendrograma i comenta quin criteri d'enllaç s'ha comportat millor. Per què?

L'enllaç millor és el single perqué és l'únic que recupera els dos anells. Tot i la silueta baixa, aquí la silueta euclidiana no és una bona mètrica per formes en anell (no convexes).

Pitjors per aquesta geometria: complete/average (tallen per semicercle) i centroid (col · lapsa en un clúster).

1.2 2. *Clustering* i reducció de dimensionalitat culinària: (6 punts)

En aquest exercici explorarem com mitjançant clustering podem descobrir patrons ocults en milers de receptes reals de Food.com. Partirem d'un conjunt d'unes 30.000 receptes amb el seu nom, descripció i ingredients, i el nostre objectiu serà agrupar-les per similitud per veure quins tipus de plats emergeixen sense supervisió: s'ajunten les postres? els plats amb all? les receptes mediterrànies?

Provarem diferents maneres de representar les receptes: des d'una codificació one-hot basada en els ingredients, fins a embeddings semàntics generats amb models de llenguatge (com MiniLM). Amb

això veurem com canvia l'estructura de les dades segons el tipus de representació i quin tipus de similitud capturen millor.

Finalment aplicarem tècniques de reducció de dimensionalitat amb UMAP i diversos mètodes de clustering que podràs provar (KMeans, DBSCAN, HDBSCAN, Mean-shift, jeràrquic…) per visualitzar i analitzar els grups resultants. Acabarem interpretant els clústers més grans a través dels seus ingredients i receptes representatives, comparant quin enfocament reflecteix millor la *lògica culinària* del conjunt.

[30]:
```
df = pd.read_csv('recipes.csv')
df.head(3)
```

[30]:
```
       id                                       name  \
0   56594                   Sun-Dried Tomato Palmiers
1  311362  Char Siu Pork Corn and Bok Choy Stir Fry
2  248335                            Black Cat Fudge

                                         description  \
0               from "INVITATION TO DINNER" cookbook
1  Here is another wok recipe I like stir fry's a…
2  These adorable fudge peices will be a huge hit…

                                         ingredients  \
0  ['garlic cloves', 'pesto sauce', 'frozen puff …
1  ['peanut oil', 'pork fillets', 'garlic cloves'…
2  ['semisweet chocolate', 'butter', 'light corn …

                                ingredients_raw_str serving_size  servings  \
0  ["2       garlic cloves, peeled and finely min…    1 (675 g)         1
1  ["2   tablespoons    peanut oil","600   g    p…    1 (364 g)         4
2  ["8   ounces    semisweet chocolate, coarsely …     1 (42 g)        24

                                               steps  \
0  ['Preheat the oven to 350 degrees F.', 'Stir t…
1  ['Heat 1 tabelspoon of oil in a wok and cook p…
2  ['Line 11x7 inches pan with foil, extending fo…

                                               tags        search_terms
0  ['60-minutes-or-less', 'time-to-make', 'course…       {'appetizer'}
1  ['60-minutes-or-less', 'time-to-make', 'course…  {'pork', 'dinner'}
2  ['time-to-make', 'course', 'preparation', 'occ…         {'dessert'}
```

Implementació: Crea una nova columna a l'DataFrame que contingui, en una sola cadena de text, tota la informació disponible de cada recepta. Concatena, en aquest ordre:

Títol (name)

Descripció (description)

Llista d'ingredients (ingredients)

Passos de preparació (steps)

Etiquetes (tags) i, si existeix, termes de cerca (search_terms)

Aquesta columna representarà una descripció textual completa de la recepta i s'usarà en el següent pas per generar els seus embeddings semàntics.

Tingues en compte:

Converteix llistes o conjunts a text llegible (per exemple, separant per comes).

Gestiona valors nuls per evitar errors de concatenació.

Verifica que el text resultant manté la informació rellevant i és coherent.

Creem funció principal i auxiliars per nova columna

[31]:
```
def build_full_recipe_text(df, out_col="text_complet"):
    """
    Construeix una nova columna de text amb una descripció completa per a cada␣
↪recepta:
      - Títol (name)
      - Descripció (description)
      - Llista d'ingredients (ingredients)
      - Passos de preparació (steps)
      - Etiquetes (tags) i, si existeixen, termes de cerca (search_terms)

    La idea és obtenir, per cada fila, una sola cadena de text que després
    podrem transformar en embeddings semàntics per fer clustering.

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame original de receptes. Ha de contenir, com a mínim, les
        columnes ['name', 'description', 'ingredients', 'steps', 'tags',␣
↪'search_terms'].
    out_col : str, opcional (default="text_complet")
        Nom de la nova columna on desarem el text concatenat.

    Returns
    -------
    df_out : pandas.DataFrame
        Còpia del DataFrame original amb una nova columna `out_col`
        que conté el text complet de cada recepta.
    """
    # Treballem sobre una còpia per no tocar el df original
    df_out = df.copy()

    def stringify_listlike(value, sep=", "):
        """
```

```
        Converteix objectes tipus llista/tupla/set (o el seu string) en una␣
↪cadena llegible.

        - Si és NaN -> retorna cadena buida.
        - Si ja és llista/tupla/set -> fa join amb el separador `sep`.
        - Si és un string que representa una llista/conjunt -> fem ast.
↪literal_eval
          i després fem join.
        - Si falla el parseig -> separem per comes com a fallback.
        """
        # Cas valors nuls de pandas (NaN), retornem valor "buit"
        if pd.isna(value):
            return ""

        # Si ja és un contenidor tipus llista/tupla/set
        if isinstance(value, (list, tuple, set)):
            items = list(value) # Afegim a ala llista
        else:
            # Si és string, intentem interpretar-lo com a estructura Python
            if isinstance(value, str):
                text = value.strip()
                try:
                    parsed = ast.literal_eval(text)
                    if isinstance(parsed, (list, tuple, set)):
                        items = list(parsed)
                    else:
                        # Si no és llista/conjunt, el tractem com a text pla
                        items = [str(value)]
                except (ValueError, SyntaxError):
                    # Si literal_eval surt amb exepció, provem de separar per␣
↪comes
                    items = [s.strip() for s in value.split(",") if s.strip()]
            else:
                # Qualsevol altre tipus: el convertim directament a string
                items = [str(value)]

        # Unió final (join) final, convertint cada element a string
        return sep.join(str(x) for x in items if str(x).strip())

    def build_row_text(row):
        """
        Construeix el text complet per a una sola recepta (una fila del␣
↪DataFrame).
        Controla valors nuls i dona un format mínimament "narratiu".
        """
        # Títol i descripció, controlant NaN
        name = "" if pd.isna(row.get("name")) else str(row.get("name")).strip()
```

```
        desc = "" if pd.isna(row.get("description")) else str(row.
↪get("description")).strip()

        # Ingredients i passos: venen com a string d'una llista -> els fem␣
↪llegibles
        ingredients_txt = stringify_listlike(row.get("ingredients"), sep=", ")
        # Per als passos, separar per punts sol donar un text més natural
        steps_txt = stringify_listlike(row.get("steps"), sep=". ")

        # Etiquetes i termes de cerca (poden ser llistes o conjunts)
        tags_txt = stringify_listlike(row.get("tags"), sep=", ")
        search_terms_txt = stringify_listlike(row.get("search_terms"), sep=", ")

        fragments = [] # Llista de fragments de columna

        if name:
            fragments.append(name)
        if desc:
            fragments.append(desc)
        if ingredients_txt:
            fragments.append(f"Ingredients: {ingredients_txt}")
        if steps_txt:
            fragments.append(f"Passos: {steps_txt}")

        # Combinar tags + search_terms en un únic bloc d'"Etiquetes"
        etiquetes_parts = []
        if tags_txt:
            etiquetes_parts.append(tags_txt)
        if search_terms_txt:
            etiquetes_parts.append(search_terms_txt)
        if etiquetes_parts:
            fragments.append("Etiquetes: " + ", ".join(etiquetes_parts))

        # Unim tots els fragments amb ". " perquè quedi una narrativa fluïda
        full_text = ". ".join(fragments)

        return full_text

    # Apliquem la funció fila a fila per construir la nova columna
    df_out[out_col] = df_out.apply(build_row_text, axis=1)

    return df_out
```

Llegim i transformem el df

[32]:
```
df = pd.read_csv("recipes.csv")
df = build_full_recipe_text(df, out_col="text_full")
```

```
print(df[["id","name","text_full"]].head(3))
```

```
       id                                       name  \
0   56594                   Sun-Dried Tomato Palmiers
1  311362  Char Siu Pork Corn and Bok Choy Stir Fry
2  248335                            Black Cat Fudge

                                          text_full
0  Sun-Dried Tomato Palmiers. from "INVITATION TO…
1  Char Siu Pork Corn and Bok Choy Stir Fry. Here…
2  Black Cat Fudge. These adorable fudge peices w…
```

Implementació: Genera els embeddings semàntics de les receptes utilitzant el model all-MiniLM-L6-v2 de la llibreria sentence-transformers. Aquest model és una versió lleugera de BERT que transforma frases completes en vectors numèrics que capturen el seu significat.

Usa com a entrada la columna amb tota la informació de la recepta (títol, descripció, ingredients, passos i etiquetes) per obtenir una representació semàntica de cadascuna.

Incrementa el paràmetre batch_size a encode() per aprofitar millor els recursos i activa show_progress_bar=True per visualitzar el progrés durant la generació d'embeddings.

Creem la funció de vectorització dels significats semàntics (semantic embeddings)/b>

[33]:
```
def build_recipe_embeddings(df, text_col="text_full",
                            model_name="all-MiniLM-L6-v2",
                            batch_size=128, show_progress_bar=True):
    """
    Genera embeddings semàntics per a cada recepta a partir del text complet.

    Utilitza el model 'all-MiniLM-L6-v2' de sentence-transformers, que␣
↪transforma
    frases/paràgrafs en vectors numèrics que capten el significat semàntic.

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame amb les receptes. Ha de contenir la columna `text_col`
        amb el text complet de cada recepta (títol, descripció, ingredients,␣
↪passos,
        etiquetes...).
    text_col : str, opcional (default="text_full")
        Nom de la columna que conté el text sobre el qual calcularem els␣
↪embeddings.
    model_name : str, opcional (default="all-MiniLM-L6-v2")
        Nom del model de sentence-transformers que volem carregar.
    batch_size : int, opcional (default=128)
        Mida del batch passat a `encode()`. Valors més grans aprofiten millor
        la GPU/CPU, sempre que càpiguen en memòria.
```

```
    Returns
    -------
    df_out : pandas.DataFrame
        Còpia del DataFrame original amb una nova columna 'embedding'
        on cada fila conté un vector (np.array) amb l'embedding de la recepta.
    embeddings : np.ndarray
        Matriu 2D de forma (n_receptes, dim_embedding) amb tots els vectors
        apilats. Útil per passar directament als algoritmes de clustering.
    """
    # Fem una còpia per no modificar el df original
    df_out = df.copy()

    # Ens assegurem que la columna de text existeix
    if text_col not in df_out.columns:
        raise KeyError(f"La columna '{text_col}' no existeix al DataFrame.")

    # 1) Preparem la llista de textos (Series -> llista de strings)
    #    - Omplim NaN amb cadena buida
    #    - Ens assegurem que tot és str
    texts = df_out[text_col].fillna("").astype(str).tolist()

    # 2) Carreguem el model de sentence-transformers
    #    (ja tenim SentenceTransformer importat a l'entorn)
    model = SentenceTransformer(model_name)

    # 3) Calculem els embeddings
    #    - batch_size gran per aprofitar recursos
    #    - show_progress_bar=True per veure el progrés
    embeddings = model.encode(
        texts,
        batch_size=batch_size,
        show_progress_bar=True
    )

    # 4) Afegim una nova columna al DataFrame on cada fila té el seu vector
    #    Convertim la matriu (n, d) en llista de np.array de llargada n
    df_out["embedding"] = list(embeddings)

    return df_out, embeddings
```

Apliquem la funció

[34]:
```
# Generem els embeddings semàntics
df, recipe_embeddings = build_recipe_embeddings(
    df,
    text_col="text_full",
```

```
    model_name="all-MiniLM-L6-v2",
    batch_size=128
)

# Mirem forma de la matriu d'embeddings
print(recipe_embeddings.shape)
print(df[["id", "name", "embedding"]].head(3))
```

```
Batches:   0%|          | 0/391 [00:00<?, ?it/s]

(50000, 384)
       id                                      name  \
0   56594                 Sun-Dried Tomato Palmiers
1  311362  Char Siu Pork Corn and Bok Choy Stir Fry
2  248335                           Black Cat Fudge

                                            embedding
0  [-0.057234544, 0.028758392, -0.06627667, 0.018…
1  [-0.100332916, 0.007798606, -0.011410531, 0.01…
2  [0.019577798, -0.028034406, 0.009474503, 0.025…
```

Implementació: Redueix els embeddings obtinguts a un espai de 2 dimensions utilitzant UMAP (Uniform Manifold Approximation and Projection), una tècnica de reducció de dimensionalitat que conserva l'estructura local de les dades en espais d'alta dimensió.

UMAP busca representar les dades en un espai més petit mantenint la relació entre punts similars, cosa que el fa ideal per visualitzar embeddings en 2D i detectar grups o patrons de manera intuïtiva. En comparació amb t-SNE, UMAP és més ràpid, escalable, permet la projecció de nous punts i preserva millor l'estructura global de les dades. Pots consultar una comparació visual detallada a aquesta pàgina.

Aplica UMAP sobre la matriu d'embeddings i guarda el resultat en una nova variable que contingui les coordenades 2D de cada recepta per a la seva posterior visualització o anàlisi.

Funció per calcular UMAP en 2D

[35]:
```
def compute_umap_2d(embeddings,
                    n_neighbors=15,
                    min_dist=0.1,
                    metric="cosine",
                    seed=SEED):
    """
    Redueix la dimensionalitat dels embeddings a 2 dimensions utilitzant UMAP.

    UMAP (Uniform Manifold Approximation and Projection) intenta projectar
    les dades d'alta dimensió a un espai de baixa dimensió (2D en aquest cas),
    tot preservant:
      - l'estructura local (punts propers continuen sent propers),
      - i parcialment l'estructura global.
```

```
    Parameters
    ----------
    embeddings : np.ndarray (shape: [n_mostres, d])
        Matriu amb els embeddings originals (sortida de SentenceTransformer).
    n_neighbors : int, opcional (default=15)
        Nombre de veïns utilitzats per construir el graf de veïnatge.
        Valors més petits → més focus en estructura local.
    min_dist : float, opcional (default=0.1)
        Distància mínima entre punts en l'espai de sortida.
        Valors petits (ex. 0.0) -> clústers més compactes.
    metric : str, opcional (default="cosine")
        Mètrica de distància a l'espai original. Amb embeddings semàntics,
        la "cosine" sol funcionar molt bé.
    seed : int, opcional
        Llavor per fer el resultat més reproduïble.

    Returns
    -------
    umap_coords : np.ndarray (shape: [n_mostres, 2])
        Matriu amb les coordenades 2D (x, y) de cada mostra/projecte UMAP.
    """
    # Creem l'objecte UMAP amb 2 components (2D)
    reducer = umap.UMAP(
        n_components=2,       # volem projecció a 2 dimensions
        n_neighbors=n_neighbors,
        min_dist=min_dist,
        metric=metric,
        random_state=seed
    )

    # Calculem la projecció 2D amb fit_transform
    #   - fit() aprèn la transformació
    #   - transform() aplica aquesta transformació
    #   - fit_transform() fa les dues coses en un sol pas
    umap_coords = reducer.fit_transform(embeddings)

    return umap_coords
```

Funció per afegir UMAP al DataFrame

[36]:
```
def add_umap_columns(df, umap_coords, x_col="umap_x", y_col="umap_y"):
    """
    Afegeix dues columnes amb les coordenades UMAP 2D al DataFrame de receptes.

    Parameters
    ----------
    df : pandas.DataFrame
```

```
        DataFrame original de receptes (ha de tenir el mateix nombre de files
        que files té `umap_coords`).
    umap_coords : np.ndarray (shape: [n_mostres, 2])
        Matriu amb les coordenades UMAP (x, y) per a cada recepta.
    x_col : str, opcional (default="umap_x")
        Nom de la columna per a la coordenada X.
    y_col : str, opcional (default="umap_y")
        Nom de la columna per a la coordenada Y.

    Returns
    -------
    df_out : pandas.DataFrame
        Còpia del DataFrame original amb les dues noves columnes UMAP.
    """
    # Comprovem que el número de files coincideix
    if len(df) != umap_coords.shape[0]:
        raise ValueError(
            f"Nombre de files del DataFrame ({len(df)}) "
            f"no coincideix amb les de umap_coords ({umap_coords.shape[0]})."
        )

    # Treballem sobre una còpia per no tocar el df original
    df_out = df.copy()

    # Columnes com a Series de pandas (1D) extretes de la matriu 2D
    df_out[x_col] = umap_coords[:, 0]
    df_out[y_col] = umap_coords[:, 1]

    return df_out
```

Ara hi afegim UMAP

[37]:
```
# 1) Calculem les coordenades UMAP 2D a partir de la matriu d'embeddings
umap_coords = compute_umap_2d(
    recipe_embeddings,
    n_neighbors=15,   # pots jugar amb aquests paràmetres
    min_dist=0.1,
    metric="cosine",
    seed=SEED
)

# Amb seed = SEED definida a la capçalera de la pràctica, el resultat serà␣
↪determinista
# per Umap (la matriu de vector reduida serà sempre la mateixa)

print("Forma umap_coords:", umap_coords.shape)  # (n_receptes, 2)
```

```
# 2) Afegim aquestes coordenades al DataFrame de receptes
df_umap = add_umap_columns(df, umap_coords,
                           x_col="umap_x", y_col="umap_y")

# Visualitzem per pantalla una mostra per control
print(df_umap[["id", "name", "umap_x", "umap_y"]].head(5))
```

```
C:\Users\user\anaconda3\envs\environment_uoc20251pec2-1\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(

Forma umap_coords: (50000, 2)
       id                                     name    umap_x     umap_y
0   56594                 Sun-Dried Tomato Palmiers  0.240205   6.116466
1  311362  Char Siu Pork Corn and Bok Choy Stir Fry  1.827638  13.057530
2  248335                            Black Cat Fudge  1.412497  -0.703809
3  102357              Hot Pickled Vegetable Medley -2.325199   6.088287
4  195095                               Shopska Salad -1.517166   7.068673
```

Implementación: Visualiza el resultado de la reducción a 2D con UMAP mediante un scatter plot. Representa cada receta como un punto en el plano y observa si se forman grupos o regiones con mayor densidad.

Para mejorar la legibilidad del gráfico:

Ajusta el parámetro alpha para controlar la transparencia de los puntos: valores más bajos (por ejemplo, 0.3) permiten visualizar mejor las zonas donde los puntos se solapan, haciendo más visibles las regiones densas.

Reduce el parámetro s (tamaño de los puntos) para evitar que se superpongan en exceso y poder distinguir mejor la estructura general del conjunto.

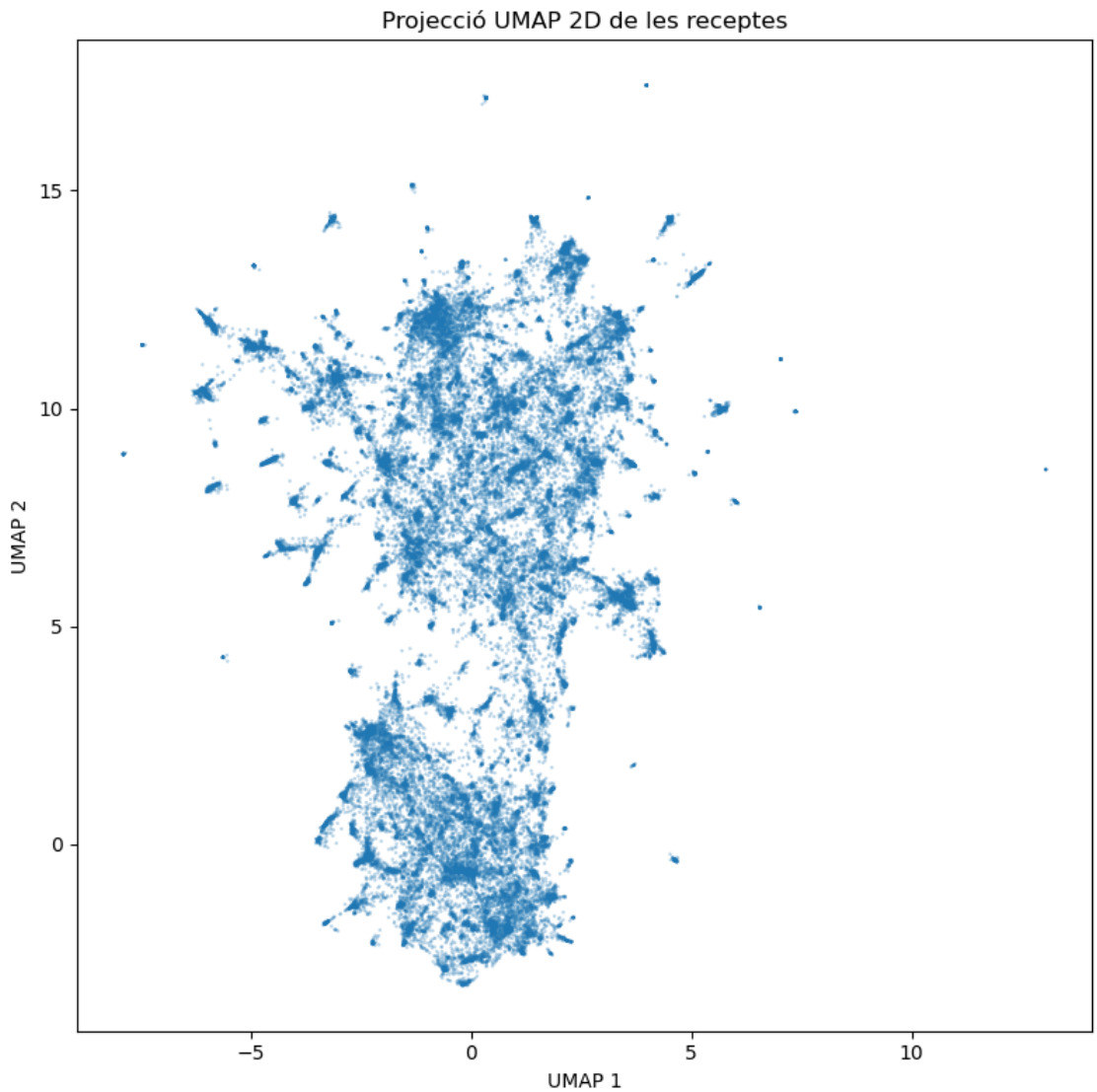
Descrivim la funció per dibuixar les coordenades del embedding passat reduït per UMAP

[38]:
```
def plot_umap_scatter(df,
                      x_col="umap_x",
                      y_col="umap_y",
                      alpha=0.3,
                      s=3,
                      figsize=(8, 8)):
    """
    Dibuixa un scatter plot de les coordenades UMAP 2D.

    Cada recepta es representa com un punt al pla (x = umap_x, y = umap_y),
    cosa que permet veure:
      - zones de més densitat (moltes receptes similars),
      - possibles "illes" o grups separats.

    Parameters
```

```
    ----------
    df : pandas.DataFrame
        DataFrame amb les receptes i les columnes UMAP.
    x_col : str, opcional (default="umap_x")
        Nom de la columna amb la coordenada X de la projecció UMAP.
    y_col : str, opcional (default="umap_y")
        Nom de la columna amb la coordenada Y de la projecció UMAP.
    alpha : float, opcional (default=0.3)
        Transparència dels punts. Valors baixos (0.2-0.4) ajuden a veure
        millor les zones on hi ha superposició.
    s : float, opcional (default=3)
        Mida dels punts. Amb molts punts (50k) convé que sigui petit (1-5).
    figsize : tupla, opcional (default=(8, 8))
        Mida de la figura en pulgades (ample, alt).

    Returns
    -------
    fig, ax : matplotlib Figure i Axes
        Objectes de la figura i els eixos per si es vol seguir personalitzant.
    """
    # Creem la figura i els eixos
    fig, ax = plt.subplots(figsize=figsize)

    # Scatter plot amb molts punts:
    # - s petit per evitar saturació
    # - alpha < 1 per veure densitats
    # - edgecolors='none' per no dibuixar contorn (més ràpid i net)
    ax.scatter(
        df[x_col],
        df[y_col],
        s=s,
        alpha=alpha,
        edgecolors="none"
    )

    # Etiquetes d'eixos i títol
    ax.set_xlabel("UMAP 1")
    ax.set_ylabel("UMAP 2")
    ax.set_title("Projecció UMAP 2D de les receptes")

    # Opcionalment, podem posar una graella molt suau
    ax.grid(False)

    plt.tight_layout()

    return fig, ax
```

"Plotejem" vectors de relació de les receptes reduïdes amb UMAP

[39]:
```
fig, ax = plot_umap_scatter(
    df_umap,
    x_col="umap_x",
    y_col="umap_y",
    alpha=0.3,   # transparència (pots provar 0.2, 0.4...)
    s=3          # mida dels punts (si ho veus molt carregat, baixa a 1-2)
)
plt.show()
```



Anàlisi: sobre quines dades fer el clustering: sobre les dades reduïdes en 2D o sobre els embeddings originals en alta dimensionalitat?

Justifica la teva elecció explicant les possibles conseqüències d'aplicar l'algorisme de clustering sobre una representació reduïda en comparació amb l'original.

Jo faria el clustering sobre els embeddings originals d'alta dimensionalitat, i utilitzaria UMAP només per visualitzar (és a dir: clustero a 384D, projectem a 2D i pintem els punts acolorits pel clúster).

Et resumeixo el perquè:

Per què clusterejar als embeddings originals?. Els embeddings (p.ex. 384 dimensions de all-MiniLM-L6-v2) estan construïts perquè distàncies / angles en aquest espai reflecteixin similitud semàntica.

Quan apliques KMeans, HDBSCAN, etc. en aquest espai, l'algorisme treballa amb tota la informació disponible: matisos entre receptes molt semblants, diferències subtils d'ingredients, de tipus de plat, etc.

La reducció a 2D és molt agressiva: condenses centenars de dimensions en només dues i això comporta una inevitable pèrdua d'informació.

UMAP està pensat sobretot per visualitzar. Preserva sobretot l'estructura local (qui és veí de qui), però pot distorsionar molt distàncies globals (dos clústers poden quedar més separats o més junts del que realment són a 384D), densitats (zones molt comprimides o molt estirades), formes dels grups (UMAP pot "trencar" o "ajuntar" regions segons els hiperparàmetres).

Això vol dir que, si fem clustering en 2D. Podem obtenir clústers artificials (creats per la manera com UMAP "aplana" l'espai), o bé barrejar receptes que en l'espai original no eren tant semblants. Per això, l'estratègia més "netament estadística" acostuma a ser Clustering en alta dimensió (embeddings originals, potser després de reduir a 10–50 dims amb UMAP/PCA si vols guanyar velocitat, però no fins a 2).

Projectar a 2D amb UMAP només per visualitzar i entendre els clústers (colorant cada punt segons l'etiqueta de clúster). Així, els clústers reflecteixen millor la similitud semàntica real, i UMAP només ens ajuda a visualitzar aquesta estructura en un pla 2D més intuïtiu.

Implementació: Visualitza les receptes en el seu espai original d'alta dimensionalitat utilitzant un dendrograma. Això et permetrà analitzar l'estructura de les dades i comparar si les agrupacions observades coincideixen amb les que apareixien a la projecció 2D d'UMAP.

Comenta què pots inferir en observar el dendrograma: si hi ha grups ben definits o si les fusions són graduals, i com es relaciona aquesta estructura jeràrquica amb les regions que apareixien a la visualització reduïda.

Creem la funció que genera un dendograma agafam el dataset amb els vectors del embedding semàntic

[40]:
```
def plot_dendrogram_high_dim(df,
                             emb_col="embedding",
                             label_col="name",
                             n_sample=300,
                             method="ward",
                             metric="euclidean",
                             truncate_mode="lastp",
                             p=30,
                             figsize=(12, 6),
                             seed=SEED):
```

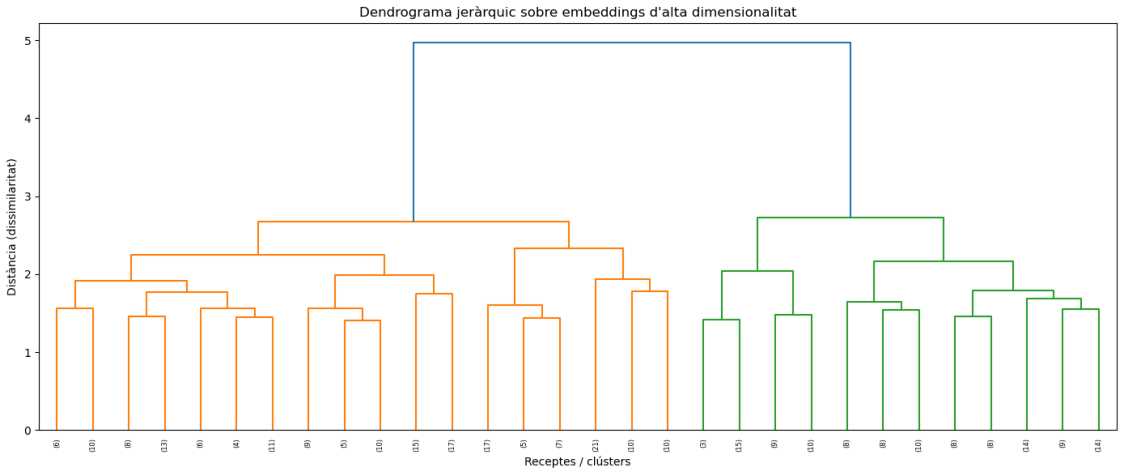
```
    """
    Construeix i visualitza un dendrograma a partir dels embeddings originals
    d'alta dimensionalitat.

    Per limitacions de temps i de llegibilitat, es fa servir només una mostra
    aleatòria de receptes (n_sample). Sobre aquesta mostra:
      - Es calcula el clustering jeràrquic amb `linkage`, enllaç.
      - Es dibuixa un dendrograma (normalment truncat) per veure l'estructura
        jeràrquica de les agrupacions.

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame amb les receptes. Ha de contenir la columna `emb_col`
        amb els vectors d'embedding (np.array).
    emb_col : str, opcional (default="embedding")
        Nom de la columna amb els embeddings.
    label_col : str, opcional (default="name")
        Nom de la columna que s'usarà com a etiqueta de cada fulla
        (nom de la recepta, id, etc.).
    n_sample : int, opcional (default=300)
        Nombre màxim de receptes a incloure al dendrograma. Si el DataFrame
        té menys files, s'usaran totes.
    method : str, opcional (default="Ward")
        Mètode d'enllaç per al clustering jeràrquic (ward, average, complete...
↪).
        `ward` és habitual quan es treballa amb distància euclidiana.
    metric : str, opcional (default="euclidean")
        Mètrica de distància a l'espai original dels embeddings.
    truncate_mode : str, opcional (default="lastp")
        Mode de truncament del dendrograma. `lastp` mostra només els `p`
        últims clústers fusionats.
    p : int, opcional (default=30)
        Nombre de clústers/fulles que es mostren al dendrograma truncat.
    figsize : tuple, opcional (default=(12, 6))
        Mida de la figura de Matplotlib.
    seed : int, opcional
        Llavor per fer que la mostra aleatòria sigui reproduïble.

    Returns
    -------
    Z : np.ndarray
        Matriu d'enllaç (linkage) del clustering jeràrquic.
    df_sample : pandas.DataFrame
        DataFrame amb la mostra de receptes utilitzada per construir el
        dendrograma (per poder-hi tornar o inspeccionar receptes concretes).
    """
```

```
    # 1) Seleccionem una mostra aleatòria de receptes
    if len(df) > n_sample:
        df_sample = df.sample(n=n_sample, random_state=seed)
    else:
        df_sample = df.copy()

    # 2) Convertim la columna d'embeddings (sèrie d'np.array) en matriu 2D
    #    shape: (n_mostres, dim_embedding)
    X = np.vstack(df_sample[emb_col].values)

    # 3) Calculem l'enllaç jeràrquic directament amb linkage.
    #    - Amb method="ward" implica distància euclidiana.
    #    - Si fas servir un altre mètode, la mètrica pot ser diferent.
    Z = linkage(X, method=method, metric=metric)

    # 4) Etiquetes de fulla (noms o ids de receptes)
    labels = df_sample[label_col].astype(str).values

    # 5) Dibuixem el dendrograma
    plt.figure(figsize=figsize)

    dendrogram(
        Z,
        labels=labels,
        leaf_rotation=90,      # girem les etiquetes per llegir-les millor
        leaf_font_size=6,      # lletra petita (hi ha moltes fulles)
        truncate_mode=truncate_mode,  # mostrem només els últims 'p' clústers
        p=p
    )

    plt.title("Dendrograma jeràrquic sobre embeddings d'alta dimensionalitat")
    plt.xlabel("Receptes / clústers")
    plt.ylabel("Distància (dissimilaritat)")
    plt.tight_layout()

    return Z, df_sample
```

Cridme a la funció

[41]:
```
Z, df_sample_dendro = plot_dendrogram_high_dim(
    df,                 # o df_umap, si hi tens també la columna "embedding"
    emb_col="embedding",
    label_col="name",   # o "id" si prefereixes etiquetes més curtes
    n_sample=300,
    method="ward",  # Dona millor visualització que altres
    metric="euclidean",
    truncate_mode="lastp",
```

```
    p=30,
    figsize=(14, 6),
    seed=SEED
)
plt.show()
```



Anàlisi: Atès el que s'ha observat en la reducció de dimensionalitat amb UMAP i en el dendrograma obtingut, decideix quin algorisme de clustering utilitzaries per identificar els grups de receptes. Explica amb quins arguments el triaries i justifica la teva resposta en funció de l'estructura observada a les dades (densitat, mida i forma dels grups).

Amb el que es veu al mapa UMAP i al dendrograma, triaria HDBSCAN com a algoritme principal de clustering aplicat sobre els embeddings originals d'alta dimensionalitat.

Arguments a partir de les dades:

A l'UMAP es veuen regions de densitat molt variable, amb "núvols" molt compactes, braços allargats i zones més disperses. Aquesta forma és lluny de clústers esfèrics i de mida similar, que és just el que assumeix KMeans o fins i tot un aglomeratiu amb tall fix.

El dendrograma mostra que les fusions són bastant graduals i que no hi ha un "salt" clar que indiqui un nombre natural de clústers. Això fa poc adequat haver de fixar prèviament un k (KMeans, aglomeratiu amb tall).

Per què HDBSCAN i no DBSCAN/altres?: Tant DBSCAN com HDBSCAN són basats en densitat i permeten trobar clústers de formes arbitràries i identificar punts sorollosos/outliers, cosa que té sentit amb aquest núvol de receptes on hi ha receptes molt aïllades. Però DBSCAN té el problema que necessita un únic valor d'eps i tolera malament la densitat variable que veiem a UMAP (clústers molt densos barrejats amb clústers més dispersos). HDBSCAN, en canvi, construeix una jerarquia de clústers de densitat i selecciona automàticament els més estables, no cal fixar el nombre de clústers, i s'adapta millor a densitats diferents.

Per tant, vista l'estructura: clústers irregulars, densitat variable, sense un k clar i amb outliers, l'algorisme més coherent és HDBSCAN sobre els embeddings originals, utilitzant UMAP només per

visualitzar els clústers resultants en 2D.

Implementació: Implementa l'algorisme elegit i analitza els seus possibles paràmetres per obtenir el resultat desitjat.

Creem una funció genèrica HDBSCAN per entrenar sobre els embeddings amb els paràmetres més adequats

[42]:
```
def fit_hdbscan_on_embeddings(embeddings,
                              min_cluster_size=20,
                              min_samples=4,
                              metric="euclidean",
                              cluster_selection_method="eom"):
    """
    Entrena l'algorisme HDBSCAN sobre els embeddings d'alta dimensionalitat.

    HDBSCAN és un mètode de clustering basat en densitat que:
      - detecta clústers de forma arbitrària,
      - suporta densitats diferents,
      - etiqueta com a soroll (label = -1) els punts que no encaixen
        bé en cap clúster.

    Parameters
    ----------
    embeddings : np.ndarray, shape (n_mostres, d)
        Matriu amb els embeddings originals de les receptes.
    min_cluster_size : int, opcional (default=80)
        Mida mínima de cada clúster. Valors grans - menys clústers però
        més grans; valors petits → més clústers, possiblement més sorollosos.
    min_samples : int, opcional (default=20)
        Nombre mínim de veïns perquè un punt sigui considerat "core".
        Valors grans -> HDBSCAN és més conservador (més soroll i clústers
        més robustos). Valors petits -> detecta més clústers, però pot ser
        més sensible a soroll.
    metric : str, opcional (default="euclidean")
        Mètrica de distància a l'espai original. Amb embeddings també es
        podria provar "cosine", però "euclidean" és un punt de partida habitual.
    cluster_selection_method : str, opcional (default="eom")
        Mètode per escollir els clústers finals a partir de la jerarquia:
          - "eom": (excess of mass) dona clústers més "globals".
          - "leaf": tendeix a clústers més petits i detallats.

    Returns
    -------
    clusterer : hdbscan.HDBSCAN
        Objecte HDBSCAN ja entrenat (fit).
    labels : np.ndarray, shape (n_mostres,)
        Etiquetes de clúster per a cada punt. El valor -1 indica soroll.
```

```
    """
    # Creem l'objecte HDBSCAN amb els hiperparàmetres indicats
    clusterer = hdbscan.HDBSCAN(
        min_cluster_size=min_cluster_size,
        min_samples=min_samples,
        metric=metric,
        cluster_selection_method=cluster_selection_method
    )

    # Entrenem el model i obtenim les etiquetes de clúster
    labels = clusterer.fit_predict(embeddings)

    return clusterer, labels
```

Afegim les etiquetes HDBSCAN al DataFrame de receptes

[43]:
```
def add_hdbscan_results_to_df(df,
                              embeddings,
                              min_cluster_size=20,
                              min_samples=4,
                              metric="euclidean",
                              cluster_selection_method="leaf"):
    """
    Aplica HDBSCAN sobre els embeddings i afegeix el resultat al DataFrame
    de receptes.

    Afegeix tres columnes:
      - 'cluster_hdbscan'      -> id de clúster (-1 = soroll)
      - 'cluster_prob'         -> probabilitat d'afiliació al clúster␣
↪(confiança)
      - 'cluster_outlier_score'-> score d'outlier (valors alts = més sospitosos)

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame amb la informació de les receptes.
    embeddings : np.ndarray, shape (n_mostres, d)
        Matriu amb els embeddings d'alta dimensionalitat (mateix ordre que df).
    min_cluster_size, min_samples, metric, cluster_selection_method :
        Vegeu la docstring de `fit_hdbscan_on_embeddings`.

    Returns
    -------
    df_out : pandas.DataFrame
        Còpia del df original amb les noves columnes de clustering.
    clusterer : hdbscan.HDBSCAN
        Objecte HDBSCAN ja entrenat.
```

```
    """
    # 1) Entrenem HDBSCAN sobre la matriu d'embeddings
    clusterer, labels = fit_hdbscan_on_embeddings(
        embeddings=embeddings,
        min_cluster_size=min_cluster_size,
        min_samples=min_samples,
        metric=metric,
        cluster_selection_method=cluster_selection_method
    )

    # 2) Fem una còpia del df per no modificar l'original
    df_out = df.copy()

    # 3) Afegim les etiquetes i els scores
    df_out["cluster_hdbscan"] = labels
    df_out["cluster_prob"] = clusterer.probabilities_
    df_out["cluster_outlier_score"] = clusterer.outlier_scores_

    return df_out, clusterer
```

Creem una funció per mostrar les estadístiques de sortida de HDBSCAN amb diferents paràmetres

[44]:
```
def summarize_hdbscan_labels(labels):
    """
    Calcula estadístiques bàsiques sobre les etiquetes de HDBSCAN:
      - nombre de clústers (sense comptar el soroll),
      - mida de cada clúster,
      - percentatge de punts etiquetats com a soroll.

    Parameters
    ----------
    labels : array-like, shape (n_mostres,)
        Etiquetes de clúster retornades per HDBSCAN (-1 = soroll).

    Returns
    -------
    summary : dict
        Diccionari amb les estadístiques principals.
    sizes_df : pandas.DataFrame
        Taula amb la mida de cada clúster (incloent -1 si es vol).
    """
    labels = np.asarray(labels)

    # Comptem quants punts hi ha de cada etiqueta
    counts = Counter(labels)

    n_total = len(labels)
```

```
    n_noise = counts.get(-1, 0)

    # Eliminem el soroll del recompte per comptar clústers "reals"
    cluster_counts = {k: v for k, v in counts.items() if k != -1}
    n_clusters = len(cluster_counts)

    # Preparem una taula amb la mida de cada clúster
    sizes_records = []
    for cl_id, size in sorted(cluster_counts.items()):
        sizes_records.append({"cluster": cl_id, "size": size})

    sizes_df = pd.DataFrame(sizes_records)

    summary = {
        "n_total_points": n_total,
        "n_clusters": n_clusters,
        "n_noise": n_noise,
        "noise_ratio": n_noise / n_total if n_total > 0 else np.nan,
        "cluster_size_min": sizes_df["size"].min() if not sizes_df.empty else 0,
        "cluster_size_max": sizes_df["size"].max() if not sizes_df.empty else 0,
        "cluster_size_median": sizes_df["size"].median() if not sizes_df.empty␣
↪else 0,
    }

    return summary, sizes_df
```

Entrenem HDBSCAN amb els paràmetres triats, Per limitacions de temps de càlcul, hem ajustat HDBSCAN sobre una mostra aleatòria de 20.000 receptes, mantenint l'espai d'embeddings original. Les conclusions es basen en aquesta mostra, que és representativa del conjunt."

[45]:
```
# 1) Triem una mostra, per ex. 20.000 receptes
sample_size = 20000
idx_sample = np.random.choice(recipe_embeddings.shape[0],
                              size=sample_size,
                              replace=False)

emb_sample = recipe_embeddings[idx_sample]
df_sample = df_umap.iloc[idx_sample].copy()

# 2) Apliquem HDBSCAN només a la mostra
df_hdb_sample, hdb = add_hdbscan_results_to_df(
    df=df_sample,
    embeddings=emb_sample,
    min_cluster_size=20,
    min_samples=4,
    metric="euclidean",
    cluster_selection_method="leaf"
```

```
)

summary, sizes_df = summarize_hdbscan_labels(df_hdb_sample["cluster_hdbscan"].
 ↪values)
print(summary)
```

```
C:\Users\user\anaconda3\envs\environment_uoc20251pec2-1\Lib\site-
packages\sklearn\utils\deprecation.py:132: FutureWarning: 'force_all_finite' was
renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
C:\Users\user\anaconda3\envs\environment_uoc20251pec2-1\Lib\site-
packages\sklearn\utils\deprecation.py:132: FutureWarning: 'force_all_finite' was
renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

```
{'n_total_points': 20000, 'n_clusters': 41, 'n_noise': 18165, 'noise_ratio':
0.90825, 'cluster_size_min': 20, 'cluster_size_max': 138, 'cluster_size_median':
34.0}
```

Ús de la funció mètriques d'etiquetes sobre l'entrenament final amb tots els punts

Implementació: Representa en un scatter plot la projecció 2D (UMAP) acoloreint cada punt segons el seu cluster. Usa una paleta qualitativa rica en colors (com tab20). Si hi ha més clústers que colors disponibles, reutilitza colors aplicant l'índex del clúster mòdul el nombre de colors de la paleta.

Fem el scatter UMAP acolorit pels clústers

Donem per suposat que tenim el DataFrame que conté:

umap_x, umap_y -> coordenades 2D,

cluster_hdbscan -> etiqueta de clúster (-1 = soroll).

El que faré és usar la paleta qualitativa tab20 de Matplotlib, assignar un color per a cada clúster, si hi ha més clústers que colors, reutilitzar colors amb mòdul.

[46]:
```
def plot_umap_clusters(df,
                       x_col="umap_x",
                       y_col="umap_y",
                       cluster_col="cluster_hdbscan",
                       palette_name="tab20",
                       alpha=0.6,
                       s=5,
                       figsize=(8, 8)):
    """
    Dibuixa un scatter plot de la projecció UMAP 2D acolorint cada punt
    segons el seu clúster (p.ex. resultat d'HDBSCAN).

    Parameters
    ----------
```

```
    df : pandas.DataFrame
        DataFrame amb:
          - coordenades UMAP (x_col, y_col)
          - columna amb l'etiqueta de clúster (cluster_col).
    x_col, y_col : str
        Noms de les columnes amb les coordenades UMAP 2D.
    cluster_col : str
        Nom de la columna amb les etiquetes de clúster.
        El valor -1 es considera soroll i es dibuixa amb un color neutre.
    palette_name : str, opcional (default="tab20")
        Nom de la paleta qualitativa de Matplotlib a utilitzar.
        Ha de tenir bastants colors (p.ex. 'tab20', 'tab10', 'tab20b'...).
    alpha : float, opcional
        Transparència dels punts (0-1). Valors més baixos eviten saturació.
    s : float, opcional
        Mida dels punts.
    figsize : tuple, opcional
        Mida de la figura (ample, alt) en polzades.

    Returns
    -------
    fig, ax : Figure i Axes de Matplotlib
        Per si es vol seguir personalitzant el gràfic.
    """
    # Ens assegurem que la columna de clúster existeix
    if cluster_col not in df.columns:
        raise KeyError(f"La columna '{cluster_col}' no existeix al DataFrame.")

    # 1) Preparem la paleta de colors
    cmap = plt.get_cmap(palette_name)
    n_colors = cmap.N  # nombre de colors diferents dins la paleta

    # 2) Identifiquem les etiquetes de clúster úniques
    labels = df[cluster_col].values
    unique_labels = np.unique(labels)

    # 3) Creem la figura
    fig, ax = plt.subplots(figsize=figsize)

    # 4) Pintem cada clúster per separat
    for lab in unique_labels:
        mask = labels == lab

        if lab == -1:
            # Soroll: color neutre (gris) i etiqueta específica
            color = "lightgrey"
            label = "soroll (-1)"
```

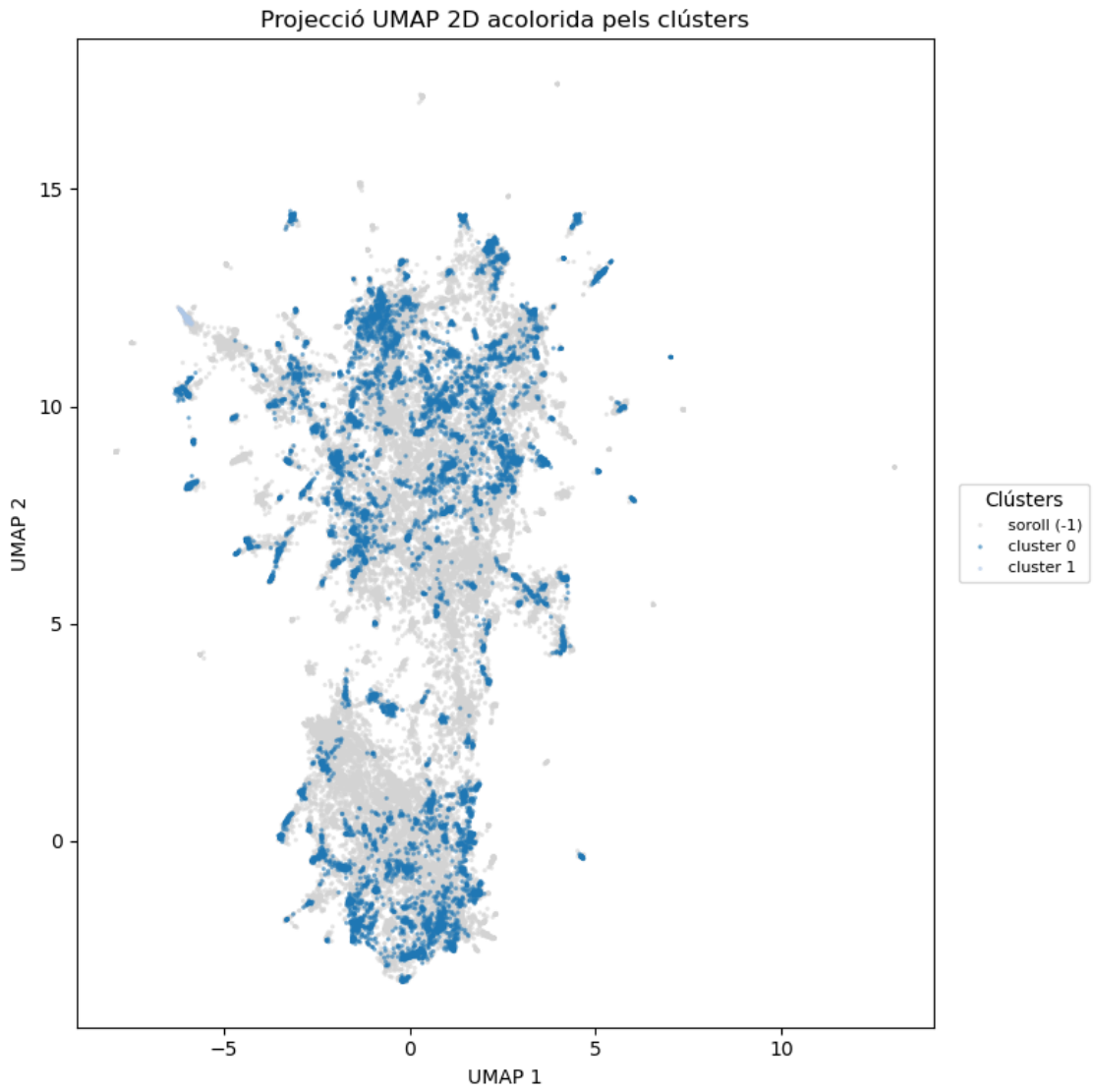
```
        else:
            # Assignem un color de la paleta fent servir el mòdul
            color = cmap(int(lab) % n_colors)
            label = f"cluster {lab}"

        ax.scatter(
            df.loc[mask, x_col],
            df.loc[mask, y_col],
            s=s,
            alpha=alpha,
            color=color,
            edgecolors="none",
            label=label
        )

    # 5) Retocs estètics
    ax.set_xlabel("UMAP 1")
    ax.set_ylabel("UMAP 2")
    ax.set_title("Projecció UMAP 2D acolorida pels clústers")
    ax.grid(False)

    # La llegenda pot ser molt llarga; si hi ha molts clústers,
    # la col·loquem fora de la zona de plot.
    ax.legend(
        loc="center left",
        bbox_to_anchor=(1.02, 0.5),
        fontsize=8,
        title="Clústers"
    )

    plt.tight_layout()

    return fig, ax
```

Fem

[47]:
```
# 1) df_umap ja té: ['embedding', 'umap_x', 'umap_y', ...]
df_hdb, hdb = add_hdbscan_results_to_df(
    df=df_umap,                 # IMPORTANT: fem servir df_umap
    embeddings=recipe_embeddings,
    min_cluster_size=50,
    min_samples=10,
    metric="euclidean",
    cluster_selection_method="eom"
)

# 2) Pintem UMAP acolorit pels clústers HDBSCAN
```

```python
fig, ax = plot_umap_clusters(
    df_hdb,
    x_col="umap_x",
    y_col="umap_y",
    cluster_col="cluster_hdbscan",
    palette_name="tab20",
    alpha=0.6,
    s=5
)
plt.show()
```

```
C:\Users\user\anaconda3\envs\environment_uoc20251pec2-1\Lib\site-
packages\sklearn\utils\deprecation.py:132: FutureWarning: 'force_all_finite' was
renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
C:\Users\user\anaconda3\envs\environment_uoc20251pec2-1\Lib\site-
packages\sklearn\utils\deprecation.py:132: FutureWarning: 'force_all_finite' was
renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



Implementació: Per interpretar el clustering i validar-ne la coherència, selecciona els 10 grups amb més mida i mostra, per a cadascun, les 5 receptes més properes al centroide (si el mètode el defineix).

HDBSCAN no defineix centroid, per aquest motiu, per a cada clúster (excloent el soroll -1), calculem la mitjana dels embeddings de totes les receptes del clúster → això serà el nostre "centroide", mesurem la distància de cada recepta del clúster a aquest centroide, agafem les 5 amb distància mínima.

[55]:
```
# Importem llibrería
from scipy.spatial.distance import cdist

def select_cluster_representatives(df,
                                   embeddings,
                                   cluster_col="cluster_hdbscan",
```

```
                                k_clusters=10,
                                n_examples=5,
                                metric="euclidean"):
    """
    Selecciona receptes representatives dels clústers HDBSCAN.

    Per als k clústers més grans (en nombre de receptes), troba,
    dins de cada clúster, les n receptes més properes al "centroide".
    Com que HDBSCAN no defineix centroid explícit, el definim com
    la mitjana dels embeddings de les receptes del clúster.

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame amb les receptes. Ha de contenir la columna de clúster
        (cluster_col) i, idealment, informació com 'id' i 'name' per poder
        interpretar les receptes seleccionades.
    embeddings : np.ndarray, shape (n_receptes, d)
        Matriu d'embeddings en l'espai on s'ha fet el clustering
        (p.ex. recipe_embeddings).
        L'ordre de les files ha de coincidir amb l'índex de df.
    cluster_col : str, opcional (default="cluster_hdbscan")
        Nom de la columna amb les etiquetes de clúster (-1 = soroll).
    k_clusters : int, opcional (default=10)
        Nombre de clústers més grans que volem analitzar.
    n_examples : int, opcional (default=5)
        Nombre de receptes representatives a seleccionar per clúster.
    metric : str, opcional (default="euclidean")
        Mètrica de distància per calcular la proximitat al centroide.

    Returns
    -------
    reps_df : pandas.DataFrame
        Taula amb una fila per (clúster, recepta seleccionada) i columnes:
          - 'cluster'        : id de clúster
          - 'cluster_size'   : mida total del clúster
          - 'rank'           : 1..n_examples (1 = més propera al centroide)
          - 'distance'       : distància al centroide
          - 'id', 'name', ...: qualsevol altra columna original de df
            que s'hagi mantingut en el merge (id, name, etc.)
    """

    # Comprovació bàsica de coherència de dimensions
    if len(df) != embeddings.shape[0]:
        raise ValueError(
            f"df té {len(df)} files però embeddings té "
            f"{embeddings.shape[0]} files. Han de coincidir."
```

```
    )

labels = df[cluster_col].values

# Comptem mides de cada etiqueta (incloent soroll)
counts = Counter(labels)

# Eliminem el soroll (-1) per triar clústers "reals"
cluster_ids = [cl for cl in counts.keys() if cl != -1]

# Ordenem clústers per mida descendent i ens quedem amb els k primers
cluster_ids_sorted = sorted(
    cluster_ids,
    key=lambda c: counts[c],
    reverse=True
)
top_clusters = cluster_ids_sorted[:k_clusters]

records = []

for cl_id in top_clusters:
    # Índex de les receptes que pertanyen a aquest clúster
    mask = (labels == cl_id)
    idx = np.where(mask)[0]

    # Embeddings d'aquest clúster
    emb_cl = embeddings[idx]

    # Centroide = mitjana dels embeddings del clúster
    centroid = emb_cl.mean(axis=0, keepdims=True)

    # Distàncies de cada recepta del clúster al centroide
    dists = cdist(emb_cl, centroid, metric=metric).ravel()

    # Ordenem per distància creixent i prenem les n més properes
    order = np.argsort(dists)
    top_local = order[:n_examples]

    for rank, pos in enumerate(top_local, start=1):
        global_idx = idx[pos]       # índex al df original
        dist = dists[pos]

        row = {
            "cluster": cl_id,
            "cluster_size": counts[cl_id],
            "rank": rank,
            "distance": dist,
```

```
                "df_index": df.index[global_idx],  # per referència
            }

            records.append(row)

    reps_df = pd.DataFrame(records)

    # Fem merge amb df per recuperar info de la recepta (id, name, ...)
    reps_df = reps_df.merge(
        df,
        left_on="df_index",
        right_index=True,
        how="left"
    )

    # Podem ordenar una mica el resultat
    reps_df = reps_df.sort_values(
        ["cluster", "rank"]
    ).reset_index(drop=True)

    return reps_df
```

[56]:
```
reps_df = select_cluster_representatives(
    df=df_hdb,
    embeddings=recipe_embeddings,
    cluster_col="cluster_hdbscan",
    k_clusters=10,
    n_examples=5,
    metric="euclidean"
)

# Mirem les primeres files
reps_df[["cluster", "cluster_size", "rank", "distance", "id", "name"]].head(20)
```

[56]:
```
   cluster  cluster_size  rank  distance      id  \
0        0          9631     1  0.591152  293295
1        0          9631     2  0.597453  432328
2        0          9631     3  0.599590  421935
3        0          9631     4  0.606704  200400
4        0          9631     5  0.607890  127237
5        1            84     1  0.388869  413784
6        1            84     2  0.394547  292029
7        1            84     3  0.395045  134851
8        1            84     4  0.415279  208220
9        1            84     5  0.416116  316753

                                   name
```

```
0              Beef Carrot Casserole
1         White Bean and Apple Chili
2                         Onion Dip
3        Veggie &amp; Rice Casserole
4                        White Chili
5            Tomato and Onion Salmon
6    Salmon With Sweet and Spicy Rub
7  Salmon With Olive Oil &amp; Herbs
8              Baked Seasoned Salmon
9                Savory Baked Salmon
```

Anàlisi: Observa els resultats del clustering i interpreta si els grups obtinguts tenen sentit des d'un punt de vista culinari. Observa si les receptes dins d'un mateix clúster comparteixen ingredients, estil de preparació o tipus de plat (per exemple, salses, postres, carns, amanides…). Explica breument per què creus que els clústers són o no coherents.

Si mirem les 5 receptes més properes al centroide de cada clúster, es veu que els grups tenen força sentit culinari: Les receptes representatives són, per exemple: "Beef Carrot Casserole", "White Bean and Apple Chili", "Onion Dip", "Veggie & Rice Casserole", "White Chili"… Totes són plats principals salats i més aviat contundents: cassoles, chilis, dips calents, plats d'olla, etc. Comparteixen:

ingredients típics de cuina casolana nord-americana (carns, llegums, verdures, formatges, salses cremoses…),

tècniques de cocció llargues (forn, cassola, guisat),

i el fet de ser plats "one-pot" o cassoles que es serveixen com a plat únic. Això justifica que HDBSCAN hagi creat un clúster molt gran d' "entrants i plats principals salats de cassola/guisat".

Implementació: Crearem una nova representació de les receptes utilitzant únicament els seus ingredients. Per a això, genera una matriu one-hot on cada fila representa una recepta i cada columna un ingredient diferent. En aquesta matriu, el valor serà 1 si la recepta conté aquest ingredient i 0 en cas contrari.

Pots utilitzar la classe MultiLabelBinarizer de sklearn.preprocessing per transformar automàticament les llistes d'ingredients en vectors binaris.

Usa la columna d'ingredients ja processada (llistes netes i sense valors nuls).

Guarda el resultat en una nova variable o DataFrame i assegura't que el nombre de columnes coincideix amb el nombre total d'ingredients únics.

Aquesta representació servirà com a nova base per aplicar tècniques de reducció de dimensionalitat i clustering.

Generem funcions i matriu ONE HOT

[49]:
```
def build_ingredient_ohe(df, ingredients_col="ingredients"):
    """
    Construeix una matriu one-hot d'ingredients a partir d'un DataFrame de␣
 ↪receptes.
```

```
    Cada recepta es representa com un vector binari on:
      - files  -> receptes
      - columnes -> ingredients únics
      - valor 1  -> la recepta conté aquest ingredient
      - valor 0  -> no el conté

    Utilitza MultiLabelBinarizer de sklearn, que està pensat precisament per
    transformar llistes de labels (multietiqueta) en vectors binaris.

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame original amb, com a mínim, la columna d'ingredients.
    ingredients_col : str, opcional (default="ingredients")
        Nom de la columna del df que conté les llistes d'ingredients
        (una llista per recepta).

    Returns
    -------
    X_ing_df : pandas.DataFrame
        Matriu one-hot d'ingredients:
          - shape = (n_receptes, n_ingredients_unics)
          - índex = mateix índex que df
          - columnes = noms d'ingredient (tal com surten al MultiLabelBinarizer)
    mlb : sklearn.preprocessing.MultiLabelBinarizer
        L'objecte MultiLabelBinarizer ja ajustat (fit).
        Serveix, per exemple, per:
          - consultar la llista d'ingredients únics (mlb.classes_)
          - fer inverse_transform sobre vectors binaris si cal.
    """

    # 1) Recuperem la sèrie amb les llistes d'ingredients
    #    Cada element ha de ser una llista (o conjunt) de strings.
    ingredient_lists = df[ingredients_col]

    # Opcional: petita comprovació bàsica
    # (si vols ser estricte, es pot deixar; si no, es pot comentar)
    if ingredient_lists.isnull().any():
        raise ValueError(f"La columna '{ingredients_col}' conté valors nuls. "
                         "Cal netejar-los abans de crear la matriu one-hot.")

    # 2) Creem el MultiLabelBinarizer.
    #    Per defecte retorna una matriu densa de 0/1 (np.ndarray).
    mlb = MultiLabelBinarizer()

    # 3) Ajustem (fit) i transformem d'una sola vegada
```

```
    #    Això recull tots els ingredients únics i construeix la matriu binària.
    X_ing = mlb.fit_transform(ingredient_lists)

    # 4) Convertim el resultat a DataFrame per treballar-hi còmodament amb␣
↪pandas
    ingredient_names = mlb.classes_          # array amb tots els ingredients␣
↪únics
    X_ing_df = pd.DataFrame(
        X_ing,
        columns=ingredient_names,
        index=df.index    # mantenim el mateix índex que df per poder fer join/
↪concat
    )

    return X_ing_df, mlb
```

[50]:
```
# Construïm la representació one-hot a partir de la columna 'ingredients'
X_ing_df, mlb_ing = build_ingredient_ohe(df, ingredients_col="ingredients")

# Fem una ullada ràpida
print("Forma de la matriu d'ingredients:", X_ing_df.shape)
print("Algunes columnes (ingredients):", X_ing_df.columns[:10].tolist())

# Comprovem que el nº de columnes coincideix amb el nº d'ingredients únics
print("Nombre d'ingredients únics segons mlb:", len(mlb_ing.classes_))
print("Nombre de columnes de X_ing_df:", X_ing_df.shape[1])
```

```
Forma de la matriu d'ingredients: (50000, 45)
Algunes columnes (ingredients): [' ', '%', "'", '*', ',', '-', '.', '0', '1',
'2']
Nombre d'ingredients únics segons mlb: 45
Nombre de columnes de X_ing_df: 45
```

Implementació: Aplica novament UMAP per reduir la nova representació one-hot d'ingredients a un espai de 2 dimensions. Aquesta reducció et permetrà visualitzar les receptes en funció de la similitud dels seus ingredients i detectar possibles agrupacions o patrons culinaris.

Usa els mateixos paràmetres d'UMAP que abans per poder comparar visualment ambdues projeccions.

Guarda el resultat en una nova variable amb les coordenades 2D de cada recepta.

[51]:
```
def compute_ingredient_umap_2d(X_ing,
                               n_neighbors=15,
                               min_dist=0.1,
                               metric="cosine",
                               seed=SEED):
    """
    Aplica UMAP sobre la representació one-hot d'ingredients per obtenir
```

```
    una projecció en 2 dimensions.

    Fem servir els mateixos paràmetres que a UMAP sobre els embeddings
    semàntics per poder comparar visualment les dues projeccions.

    Parameters
    ----------
    X_ing : pandas.DataFrame o np.ndarray, shape (n_receptes, n_ingredients)
        Matriu one-hot d'ingredients (files = receptes, columnes = ingredients).
    n_neighbors : int, opcional (default=15)
        Nº de veïns que UMAP utilitza per construir el "gràfic" de proximitat.
    min_dist : float, opcional (default=0.1)
        Distància mínima entre punts a la projecció 2D (controla la␣
↪"compactació").
    metric : str, opcional (default="cosine")
        Mètrica de distància. Amb vectors esparsos/one-hot, la cosine té força␣
↪sentit.
    seed : int, opcional
        Llavor per fer el resultat reproduïble.

    Returns
    -------
    umap_coords : np.ndarray, shape (n_receptes, 2)
        Coordenades 2D (UMAP1, UMAP2) per a cada recepta.
    umap_model : umap.UMAP
        Objecte UMAP ajustat, per si vols reutilitzar-lo més endavant.
    """
    # Ens assegurem que treballem amb un np.ndarray
    X_array = X_ing.values if hasattr(X_ing, "values") else X_ing

    umap_model = umap.UMAP(
        n_components=2,
        n_neighbors=n_neighbors,
        min_dist=min_dist,
        metric=metric,
        random_state=seed,
    )

    umap_coords = umap_model.fit_transform(X_array)

    return umap_coords, umap_model
```

[52]:
```
# 1) Partim del df original (o df ja filtrat) i de la matriu one-hot
#    X_ing_df l'hem obtingut abans amb build_ingredient_ohe(df, "ingredients")

ingredient_umap_coords, ingredient_umap_model = compute_ingredient_umap_2d(
    X_ing_df,
```

```
    n_neighbors=15,   # igual que abans
    min_dist=0.1,
    metric="cosine",
    seed=SEED
)

print("Forma projecció UMAP ingredients:", ingredient_umap_coords.shape)
# -> (n_receptes, 2)

# 2) Afegim aquestes coordenades 2D al DataFrame original.
#    Si ja tens una funció add_umap_columns, la podem reutilitzar amb nous noms.
df_ing_umap = add_umap_columns(
    df,                       # mateix df de sempre, amb info de receptes
    ingredient_umap_coords,
    x_col="umap_ing_x",       # noms diferents per no xafar el primer UMAP
    y_col="umap_ing_y"
)

# Fem una ullada ràpida
df_ing_umap[["id", "name", "umap_ing_x", "umap_ing_y"]].head()
```

```
C:\Users\user\anaconda3\envs\environment_uoc20251pec2-1\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(

Forma projecció UMAP ingredients: (50000, 2)
```

[52]:

	id	name	umap_ing_x	umap_ing_y
0	56594	Sun-Dried Tomato Palmiers	-5.221274	0.398941
1	311362	Char Siu Pork Corn and Bok Choy Stir Fry	-1.297400	-13.682735
2	248335	Black Cat Fudge	-7.001215	-15.875409
3	102357	Hot Pickled Vegetable Medley	-6.647655	-4.077013
4	195095	Shopska Salad	-4.675196	3.462325

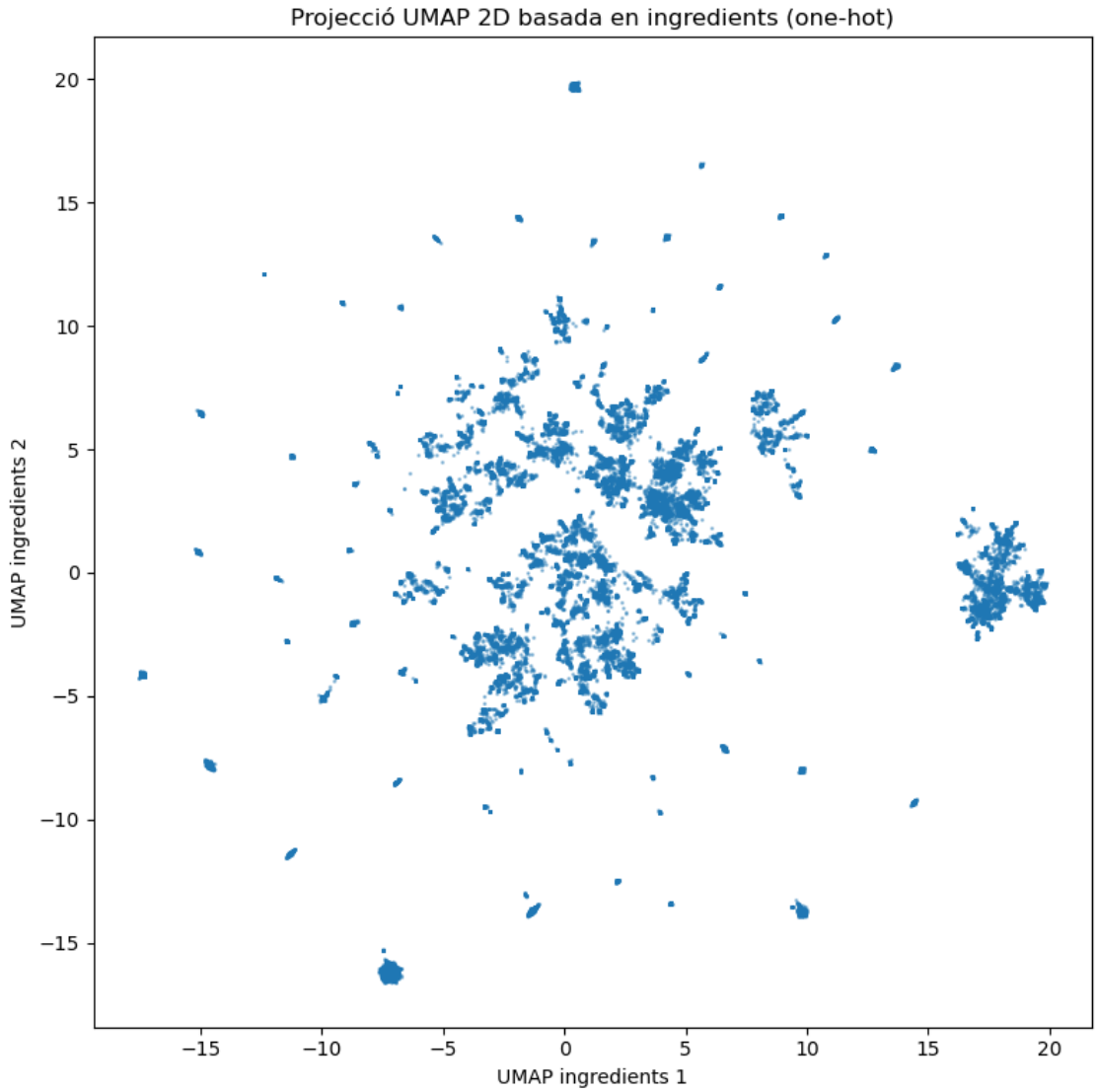
Implementació: Visualitza en un scatter plot la nova projecció en 2D obtinguda en aplicar UMAP sobre la representació one-hot dels ingredients. Cada punt ha de representar una recepta, posicionada segons la similitud dels seus ingredients.

[53]:
```
def plot_ingredient_umap(df,
                         x_col="umap_ing_x",
                         y_col="umap_ing_y",
                         s=1,
                         alpha=0.4,
                         figsize=(8, 8)):
    """
    Dibuixa un scatter plot de la projecció UMAP 2D basada en la
    representació one-hot d'ingredients.
```

```
    Cada punt representa una recepta i la seva posició al pla indica
    la similitud dels seus ingredients amb la resta de receptes:
      - receptes amb ingredients semblants queden a prop,
      - receptes amb ingredients molt diferents queden més allunyades.

    Parameters
    ----------
    df : pandas.DataFrame
        DataFrame que conté, com a mínim, les columnes amb les coordenades
        UMAP d'ingredients.
    x_col : str, opcional (default="umap_ing_x")
        Nom de la columna amb la coordenada UMAP 1 (eix X).
    y_col : str, opcional (default="umap_ing_y")
        Nom de la columna amb la coordenada UMAP 2 (eix Y).
    s : float, opcional (default=1)
        Mida dels punts del scatter.
    alpha : float, opcional (default=0.4)
        Transparència dels punts (0-1). Valors més baixos ajuden quan hi ha
        molts punts sobreposats.
    figsize : tuple, opcional (default=(8, 8))
        Mida de la figura en polzades (ample, alt).

    Returns
    -------
    fig, ax : matplotlib.figure.Figure, matplotlib.axes.Axes
        La figura i els eixos per si vols continuar personalitzant el gràfic.
    """

    # 1) Creem la figura i els eixos
    fig, ax = plt.subplots(figsize=figsize)

    # 2) Scatter de totes les receptes en funció dels ingredients
    ax.scatter(
        df[x_col],
        df[y_col],
        s=s,
        alpha=alpha
    )

    # 3) Retocs estètics
    ax.set_xlabel("UMAP ingredients 1")
    ax.set_ylabel("UMAP ingredients 2")
    ax.set_title("Projecció UMAP 2D basada en ingredients (one-hot)")
    ax.grid(False)

    plt.tight_layout()
```

```
    return fig, ax
```

[54]:
```
fig, ax = plot_ingredient_umap(
    df_ing_umap,
    x_col="umap_ing_x",
    y_col="umap_ing_y",
    s=1,
    alpha=0.4
)
plt.show()
```



Anàlisi: Compara la visualització obtinguda amb els embeddings semàntics (generats amb MiniLM) i la nova projecció obtinguda a partir de la representació one-hot dels ingredients. Descriu les

principals diferències que observis entre ambdues.

Justifica la teva resposta plantejant una hipòtesi sobre per què els punts s'organitzen de forma diferent en cada cas. Pots reflexionar sobre quin tipus d'informació captura cada representació i com això afecta l'estructura dels grups observats.

[ ]:

```

```

Implementació: A partir de la nova representació one-hot d'ingredients, decideix quin algorisme de clustering utilitzaries per agrupar les receptes. Explica si mantindries el mateix algorisme usat en el cas anterior o si en triaries un altre, justificant la teva decisió en funció de l'estructura observada a les dades.

Si l'algorisme triat requereix una mètrica de similitud o distància, indica quina utilitzaries (per exemple, Jaccard, cosinus o euclidiana) i argumenta per què resulta més adequada per a aquest tipus de dades binàries.

[ ]:

```

```

Implementació: Visualitza el conjunt de dades original (la projecció 2D obtinguda amb UMAP) acolorint cada punt segons el clúster al qual pertany.

[ ]:

```

```

Implementació: Per interpretar el clustering i comprovar-ne la coherència, selecciona els 10 clústers més grans i mostra, per a cadascun:

Les 5 receptes més properes al centroide del clúster (si el mètode el defineix). Si no hi ha centroide, fes servir el centroide empíric (mitjana del vector del clúster) o el medoide (recepta amb menor distància mitjana a la resta). Fes servir una mètrica adequada a la representació.

També, per a cada clúster, mostra els ingredients més repetits del clúster (per exemple, top-3 o top-5), opcionalment pots indicar-ne la freqüència relativa per caracteritzar el "tema" del grup. Pots agrupar i comptar amb utilitats tipus Counter o transformacions prèvies del one-hot.

[ ]:

```

```

Anàlisi: Analitza els resultats del clustering i valora si els clústers obtinguts tenen sentit des d'un punt de vista culinari. Observa els ingredients més freqüents i les receptes prototip de cada grup, i identifica si existeix un patró clar que relacioni els plats entre si.

[ ]:

```

```

Anàlisi: Compara els resultats obtinguts en aquest apartat (basats en la representació one-hot d'ingredients) amb els del primer apartat (embeddings semàntics generats amb MiniLM). Analitza si els grups que apareixen són similars o diferents, reflexiona sobre com influeix el tipus de representació i explica les principals diferències observades.

[ ]:

```

```

Anàlisi: Reflexiona sobre els possibles casos d'ús dels dos enfocaments de clustering aplicats:

Per a quin tipus de problema o aplicació seria més útil el clustering basat en els embeddings semàntics (MiniLM)?

I en quin tipus de situació resultaria més adequat el clustering obtingut amb la representació one-hot d'ingredients?

[ ]:
```

```

Anàlisi: Pensa en altres maneres de generar una representació de les receptes, diferents de les vistes a la pràctica (com els embeddings semàntics o els vectors one-hot d'ingredients). Descriu breument com construiries aquesta nova representació i quin tipus d'informació capturaria.

[ ]:
```

```